Post-Quantum Cryptography Developer's Handbook

A structured, in-depth path to mastering FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), and related PQC standards. Tailored for C/C++ developers with balanced theory and hands-on practice.

Target Audience: C/C++ developers with basic programming skills who want to understand and implement post-quantum cryptography.

Learning Approach: This plan follows a bottom-up approach: first understanding the mathematical foundations, then the algorithms, then practical implementation, and finally security analysis. Each section builds on previous ones.

ESSENTIALS EDITION.

This is a curated subset of the full PQC Developer's Handbook (1,220+ pages). It includes the introduction, glossary, key algorithm overviews (ML-KEM, ML-DSA, SLH-DSA, X-Wing), and the operational/migration material (cloud KMS integration, CVE registry, formal verification lessons, production deployment checklist). For mathematical foundations, deep algorithm internals, and full source code references, see the full edition.

Generated from `pqc-developers-handbook.md` v1.0 — see Document History in the full edition for change log.

Table of Contents

The Post-Quantum Transition at a Glance

Before diving into the technical details, it's important to understand the big picture: why post-quantum cryptography exists, what it replaces, and what remains secure.

Why the Transition?

The Quantum Threat: Shor's algorithm, when run on a sufficiently powerful quantum computer, can break RSA, Diffie-Hellman, and all elliptic curve cryptography in polynomial time. This affects:

- **Key Exchange:** RSA key transport, DH, ECDH
- **Digital Signatures:** RSA signatures, DSA, ECDSA, EdDSA
- **All public-key cryptography based on integer factorization or discrete logarithms**

Timeline Estimates:

- Cryptographically relevant quantum computers: 2030-2040 (estimates vary)
- "Harvest now, decrypt later" attacks: **happening today**
- NIST standardization: **Completed August 2024** (FIPS 203, 204, 205)
- NSA CNSA 2.0 deadlines: PQC required for National Security Systems by 2030-2035

How Shor's Algorithm Breaks Classical Cryptography

Shor's algorithm exploits the algebraic structure of factoring and discrete logarithms:

1. **The core insight:** Factoring an integer N reduces to finding the period of the function f(x) = a^x mod N. Similarly, discrete logarithms reduce to period finding.
2. **Quantum speedup:** A quantum computer can find periods using the Quantum Fourier Transform (QFT) in polynomial time — roughly $O(n^3)$ quantum gate operations for n-bit numbers. Classical computers require sub-exponential time for the same task.

3. **What it breaks:** Any cryptosystem whose security relies on:
    - Integer factorization (RSA)
    - Discrete logarithm in finite fields (DH, DSA)
    - Discrete logarithm on elliptic curves (ECDH, ECDSA, EdDSA)
4. **What it does NOT break:** Problems without the algebraic structure Shor exploits:
    - Finding shortest vectors in lattices (ML-KEM, ML-DSA) — no known period-finding reduction
    - Inverting hash functions (SLH-DSA) — unstructured search problem
    - Decoding random linear codes (McEliece, HQC) — no algebraic shortcut
5. **Grover's algorithm** provides only a quadratic speedup (√N vs N) for unstructured search problems. This affects symmetric ciphers and hash functions, but doubling key sizes is sufficient mitigation.

Current quantum hardware (April 2026): The largest announced machines have ~1,000 physical qubits. IBM's published roadmap to fault-tolerant scale runs **Kookaburra (2026)** — the first module combining qLDPC memory with a logical processing unit (LPU); **Cockatoo (2027)** — first inter-module entanglement (the L-bus that lets multiple Kookaburra-class chips share logical qubits); and **Starling (by 2029)** — the full fault-tolerant target at ~200 logical qubits. Breaking RSA-2048 still requires cryptographically relevant fault-tolerant scale (estimated thousands of logical qubits, see Q-Day section); none of the 2026–2029 milestones get there on their own, but each is a documented step on the path.

Q-Day estimates have accelerated sharply in the last year. Three papers in early 2026 rewrote the threat timeline:

1. **Gidney (May 2025, arXiv 2505.15917)** — revised the RSA-2048 factoring estimate down by 20× from his 2019 benchmark: **fewer than 1 million noisy qubits in under a week**, using ~1,730 logical qubits (Chevignard-Fouque-Schrottenloher approximate modular arithmetic).
2. **Google Quantum AI (March 2026)** — fewer than 500,000 physical qubits suffice to break the ECC used by Bitcoin and Ethereum in minutes. Spacetime volume ~10× smaller than the Chevignard/Litinski baseline.
3. **Google's own migration target** was moved up to **2029** for full PQC coverage of Google systems, reflecting internal assessments of the threat timeline.

These revisions do not change the math of whether to migrate — they sharpen *when*. "Harvest now, decrypt later" attacks on data with 5+ year confidentiality windows are already rational for an adversary, and the window for pre-quantum cryptography is narrower than the 2030–2040 estimates circulated in 2023–2024 suggested. The operative phrase is **"migrate now, finish before the end of the decade."**

What's Being Replaced?

CLASSICAL ALGORITHM	VULNERABILITY	POST-QUANTUM REPLACEMENT	STANDARD
RSA (key transport)	Shor's algorithm	**ML-KEM**	FIPS 203
Diffie-Hellman (DH)	Shor's algorithm	**ML-KEM**	FIPS 203
ECDH (X25519, P-256)	Shor's algorithm	**ML-KEM**	FIPS 203
RSA signatures	Shor's algorithm	**ML-DSA** or **SLH-DSA**	FIPS 204/205
DSA	Shor's algorithm	**ML-DSA** or **SLH-DSA**	FIPS 204/205
ECDSA (P-256, P-384)	Shor's algorithm	**ML-DSA** or **SLH-DSA**	FIPS 204/205
EdDSA (Ed25519)	Shor's algorithm	**ML-DSA** or **SLH-DSA**	FIPS 204/205

What Stays Safe?

Symmetric cryptography and **hash functions** are affected by Grover's algorithm, which provides only a quadratic speedup (not exponential like Shor's). The mitigation is simple: double the key/output size.

ALGORITHM TYPE	CLASSICAL SECURITY	POST-QUANTUM SECURITY	MITIGATION
AES-128	128-bit	64-bit	Use AES-256
AES-256	256-bit	128-bit	Already sufficient
SHA-256	256-bit	128-bit	Use SHA-384/SHA-512 or SHA3
SHA-512	512-bit	256-bit	Already sufficient
HMAC, HKDF	Secure	Secure	Use 256-bit keys

Bottom line: AES-256 and SHA-384/SHA-512 remain secure against quantum attacks.

Grover's Algorithm: How Much Threat, Really?

Grover's algorithm searches an unstructured space of N items in $O(\sqrt{N})$ queries, squaring the brute-force attack surface. For a symmetric key of k bits, this reduces effective security to $k/2$ bits. However, a practical Grover attack faces severe physical constraints:

FACTOR	IMPACT
Circuit depth	Each Grover iteration requires a full cipher evaluation as a quantum circuit. AES-128 needs ~6,400 T-gates per round × 10 rounds = ~64,000 T-gates, plus ancilla operations totaling ~10^6 logical gates per oracle call
Iterations required	$\sqrt{(2^{128})} = 2^{64}$ iterations for AES-128 — each must be run sequentially (Grover cannot be parallelized across quantum processors without losing the quadratic speedup)
MAXDEPTH parameter	NIST's security analysis assumes a maximum circuit depth of 2^{64} or 2^{96} sequential gates (the MAXDEPTH parameter). At 2^{64} depth, even AES-128 likely survives because each iteration exceeds simple gate depth budgets
Error correction	Logical qubits require ~1,000–10,000 physical qubits for error correction. A full Grover search on AES-128 would need millions of stable physical qubits for extended durations
Classical parallelism	Classical brute-force can parallelize linearly — 2^{30} processors give 2^{30} speedup. Grover gives only 2^{64} total speedup regardless of quantum resources

The AES-128 debate: Some researchers (Jaques et al., 2020) argue that the gate cost makes Grover-on-AES-128 impractical even with fault-tolerant quantum computers, because the total cost exceeds exhaustive classical search with realistic hardware. Others (Grassl et al.) show optimized circuits that keep the threat plausible. NIST's conservative stance: AES-128 provides **at most** 64 bits of post-quantum security — use AES-256 (128-bit post-quantum) for long-term safety.

Key takeaway: Grover's algorithm is a theoretical quadratic speedup, not a practical break. The real quantum threat to cryptography comes from Shor's algorithm (exponential speedup on structured problems), not Grover.

The Three NIST Standards

STANDARD	ALGORITHM	USE CASE	SECURITY BASIS
FIPS 203	ML-KEM	Key encapsulation (replacing DH/ ECDH)	Module-LWE (lattice)
FIPS 204	ML-DSA	Digital signatures (general purpose)	Module-LWE/SIS (lattice)
FIPS 205	SLH-DSA	Digital signatures (conservative)	Hash functions only

When to use which signature algorithm:

- **ML-DSA:** General purpose, smaller signatures, faster
- **SLH-DSA:** Maximum security assurance (no lattice assumptions), larger signatures

PQC Algorithm Families at a Glance

Beyond the three FIPS standards, PQC encompasses several distinct algorithm families based on different mathematical problems:

FAMILY	SECURITY BASIS	KEM EXAMPLES	SIGNATURE EXAMPLES	KEY CHARACTERISTICS
Lattice (structured)	Module-LWE/ SIS	ML-KEM	ML-DSA, FN-DSA (FALCON)	Fast, moderate sizes, NIST standardized
Hash-based	Hash function security	—	SLH-DSA, LMS, XMSS	Conservative security, large signatures, proven assumptions
Code-based	Decoding random codes	Classic McEliece, HQC	—	Decades of study, very large public keys (McEliece)
Lattice (unstructured)	Plain LWE	FrodoKEM	—	Most conservative lattice option, much larger keys
Multivariate	MQ problem	—	(none standardized)	Fast verify, but most candidates broken
Isogeny-based	Supersingular isogenies	—	SQIsign (research)	Smallest keys, but SIDH broken in 2022

Why lattice-based algorithms dominate: NIST selected primarily lattice-based schemes (ML-KEM, ML-DSA) because they offer the best balance of security, performance, and key/signature sizes. Hash-based signatures (SLH-DSA) provide a conservative alternative based on minimal assumptions. Code-based KEMs (HQC, selected March 2025) provide algorithm diversity.

Algorithm diversity matters: If a breakthrough were to weaken lattice assumptions, organizations need fallback options — this is why SLH-DSA (hash-based) and HQC (code-based) exist alongside ML-KEM and ML-DSA.

Size and Performance Trade-offs

Post-quantum algorithms have significantly larger keys and signatures than classical algorithms:

ALGORITHM	PUBLIC KEY	SIGNATURE/CIPHERTEXT	SECURITY LEVEL
RSA-2048	256 bytes	256 bytes	~112-bit
ECDSA P-256	64 bytes	64 bytes	~128-bit
ML-KEM-768	1,184 bytes	1,088 bytes (ciphertext)	~192-bit (Level 3)
ML-DSA-65	1,952 bytes	3,309 bytes	~192-bit (Level 3)
SLH-DSA-128f	32 bytes	17,088 bytes	~128-bit (Level 1)
SLH-DSA-128s	32 bytes	7,856 bytes	~128-bit (Level 1)

Key insight: PQC public keys and signatures are 10-50x larger than classical equivalents. This impacts:

- Certificate sizes
- TLS handshake latency
- Bandwidth requirements
- Storage requirements

The Hybrid Transition Strategy

During the transition period (2024-2035+), **hybrid cryptography** combines classical and post-quantum algorithms:

```
Hybrid Key Exchange:  ECDH + ML-KEM  →  shared_secret = KDF(ECDH_secret || ML-KEM_secret)
Hybrid Signature:     ECDSA + ML-DSA →  Both signatures must verify
```

Why hybrid?

1. **Defense in depth:** Secure if either algorithm remains unbroken
2. **Regulatory compliance:** Some systems still require classical crypto
3. **Backwards compatibility:** Gradual migration path
4. **Conservative approach:** PQC algorithms are newer, less battle-tested

Learning Path Summary

This curriculum takes you from foundations to deployment:

```
Module 1: Prerequisites     →  Environment setup, tools
Module 2: Math Foundations  →  Modular arithmetic, polynomials, NTT
Module 3: Lattice Theory    →  LWE, Ring-LWE, Module-LWE
Module 4: ML-KEM           →  Key encapsulation (FIPS 203)
Module 5: Signature Theory  →  Fiat-Shamir, security models
Module 6: ML-DSA           →  Lattice signatures (FIPS 204)
Module 7: SLH-DSA          →  Hash-based signatures (FIPS 205)
Module 8: Hybrid Crypto    →  Combining classical + PQC
Module 9: Protocols        →  TLS, SSH, S/MIME integration
Module 10: Future          →  Emerging algorithms, agility
Module 11: Migration       →  Legacy codebase migration guide
```

How to Use This Document

1. **Start with Module 1** - Set up your environment first. All code examples assume liboqs and OpenSSL 3.x are installed.
2. **Follow the modules sequentially** - Each module builds on the previous. Don't skip Module 2 foundations.
3. **Type the code yourself** - Don't just read; implement. The toy implementations help build intuition before using production libraries.
4. **Use the glossary** - Keep it open as a reference while reading. PQC has specialized terminology.
5. **Test incrementally** - After each implementation, test against known values. The debugging section provides tools for this.
6. **Choose your schedule** - Select Schedule A, B, or C based on your available time.
7. **Dive deep into one algorithm first** - Master ML-KEM (Module 4) before moving to ML-DSA (Module 6). They share concepts but differ in important ways.
8. **Bookmark the FIPS documents** - This guide explains concepts, but the standards are authoritative.

> **CITATIONS:**
>
> This document uses bracketed references like [Regev05] that point to the Bibliography section at the end. Full bibliographic details including DOIs and URLs are provided there.

Glossary of Terms

Understanding PQC requires familiarity with specialized terminology. Reference this section as you progress through the material.

Cryptographic Concepts

TERM	DEFINITION
KEM	Key Encapsulation Mechanism - a public-key algorithm that generates a random shared secret and encapsulates it for transmission
PKE	Public Key Encryption - encryption where anyone can encrypt with a public key, only the private key holder can decrypt
Digital Signature	Cryptographic proof that a message was created by a specific entity
IND-CPA	Indistinguishability under Chosen Plaintext Attack - security notion where adversary cannot distinguish encryptions of chosen plaintexts
IND-CCA2	Indistinguishability under Adaptive Chosen Ciphertext Attack - strongest security notion, adversary has access to decryption oracle
CCA Security	Chosen Ciphertext Attack security - protection against adversaries who can obtain decryptions of chosen ciphertexts
Random Oracle Model	Proof technique where hash functions are modeled as truly random functions
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
XOF	Extendable Output Function - hash function with variable-length output (e.g., SHAKE128, SHAKE256)

Lattice Cryptography Terms

TERM	DEFINITION
Lattice	Discrete additive subgroup of $\mathbb{R}^n$; geometrically, a regular grid of points
Basis	Set of linearly independent vectors that generate a lattice
SVP	Shortest Vector Problem - find the shortest non-zero vector in a lattice
CVP	Closest Vector Problem - find the lattice point closest to a target
LWE	Learning With Errors - problem of recovering secret from noisy linear equations
Ring-LWE	LWE variant using polynomial rings for efficiency
Module-LWE	LWE variant using modules over polynomial rings (used in ML-KEM/ML-DSA)
NTT	Number Theoretic Transform - finite field FFT for fast polynomial multiplication
CBD	Centered Binomial Distribution - noise distribution used in ML-KEM

ML-KEM Specific Terms

TERM	DEFINITION
Encapsulation Key (ek)	Public key used to encapsulate shared secrets
Decapsulation Key (dk)	Secret key used to decapsulate shared secrets (contains ek, the decryption key, and an implicit rejection seed)
Compress/ Decompress	Functions to reduce coefficient bit-width for smaller ciphertexts
Fujisaki-Okamoto Transform	Technique to convert IND-CPA encryption to IND-CCA2 KEM

ML-DSA Specific Terms

TERM	DEFINITION
Fiat-Shamir Transform	Technique to convert interactive proof to non-interactive signature
Rejection Sampling	Technique where signing may abort and retry to avoid leaking secret information
Hint	Auxiliary data in ML-DSA signatures to enable efficient verification

Hash-Based Signature Terms

TERM	DEFINITION
OTS	One-Time Signature - signature scheme that can only securely sign one message
WOTS+	Winternitz One-Time Signature Plus - efficient OTS using hash chains
XMSS	eXtended Merkle Signature Scheme - many-time signature using Merkle tree of OTS
FORS	Forest Of Random Subsets - few-time signature used in SPHINCS+
Hypertree	Multi-layer tree structure in SPHINCS+/SLH-DSA
Authentication Path	Sequence of sibling hashes from leaf to root in Merkle tree

Security Levels

NIST LEVEL	CLASSICAL EQUIVALENT	QUANTUM EQUIVALENT	EXAMPLE ALGORITHM
Level 1	AES-128	SHAKE256 with 128-bit output	ML-KEM-512
Level 2	SHA-256 collision	-	ML-DSA-44
Level 3	AES-192	SHAKE256 with 192-bit output	ML-KEM-768, ML-DSA-65
Level 4	SHA-384 collision	-	-
Level 5	AES-256	SHAKE256 with 256-bit output	ML-KEM-1024, ML-DSA-87

Hybrid Cryptography Terms

TERM	DEFINITION
Hybrid Cryptography	Combining classical and post-quantum algorithms to provide security against both conventional and quantum attacks
X-Wing	A hybrid KEM combining X25519 and ML-KEM-768, specified in IETF draft
Composite Signature	A signature scheme that combines multiple signature algorithms (e.g., ECDSA + ML-DSA)
Concatenation Combiner	Hybrid approach where outputs are concatenated: k = KDF(k_classical || k_pq)
Nested Combiner	Hybrid approach where one algorithm encrypts the other's output
Harvest Now, Decrypt Later	Attack where adversaries store encrypted data today to decrypt with future quantum computers
Defense in Depth	Security strategy using multiple layers of protection

Protocol Integration Terms

TERM	DEFINITION
sntrup761	Streamlined NTRU Prime, a lattice-based KEM used in OpenSSH hybrid key exchange
Hybrid KEX	Key exchange combining classical (e.g., X25519) and post-quantum algorithms
PQ-TLS	Post-quantum TLS, TLS 1.3 with hybrid or pure PQ key exchange
Named Groups	TLS extension specifying supported key exchange algorithms
Composite Certificate	X.509 certificate containing multiple public keys (classical + PQ)

Future Algorithms and Research Terms

TERM	DEFINITION
FALCON	Fast-Fourier Lattice-based Compact Signatures over NTRU - selected for NIST standardization as FN-DSA (FIPS 206 draft, expected late 2026/2027)
BIKE	Bit-Flipping Key Encapsulation - code-based KEM, NIST Round 4 candidate
HQC	Hamming Quasi-Cyclic - code-based KEM, selected by NIST for standardization (March 2025) as backup to ML-KEM
Classic McEliece	Code-based KEM with very large public keys, NIST Round 4 candidate
Cryptographic Agility	System design that allows algorithms to be changed without major modifications
Algorithm Deprecation	Process of phasing out cryptographic algorithms as they become insecure
NTRU	Number Theory Research Unit - lattice-based encryption scheme, precursor to modern lattice crypto
Isogeny-Based	Cryptography based on isogenies between elliptic curves (e.g., SIDH/SIKE, now broken)

Deployment, Compliance, and Operational Terms

TERM	DEFINITION
CBOM	Cryptographic Bill of Materials — OWASP CycloneDX 1.6 inventory of every cryptographic asset (algorithm, protocol, certificate, key, secret) and its location; foundation of PQC migration per OMB M-23-02
CNSA 2.0	NSA Commercial National Security Algorithm Suite 2.0 — successor to Suite B; mandates PQC for National Security Systems with phased deadlines (new acquisitions January 2027, full transition 2030–2035)
CRQC	Cryptographically Relevant Quantum Computer — a quantum computer large and reliable enough to run Shor's algorithm against RSA-2048 or ECC; timeline estimates per 2025-2026 research have accelerated to late 2020s
CSfC	Commercial Solutions for Classified — NSA program enabling classified-system operators to acquire commercial cryptographic products; CSfC PQC Addendum published 2025
FIPS 140-3	NIST Cryptographic Module Validation Program standard; mandatory for US federal systems. Transition deadline September 21, 2026 (140-2 moves to CMVP historical list)
HNDL	Harvest Now, Decrypt Later — the attack model in which an adversary records encrypted traffic today for later decryption once a CRQC becomes available; drives the urgency of PQC migration for data with 5+ year confidentiality windows
KAT	Known Answer Test — NIST-provided test vectors in `.req` / `.rsp` format used to validate algorithm implementation correctness; critical for FIPS compliance and interop testing
ACVP / CAVP	Automated Cryptographic Validation Protocol (JSON-format test vector service) / Cryptographic Algorithm Validation Program (underlying NIST process). Required input to FIPS 140-3 module validation

Recent Events and Incidents (2024-2026)

TERM	DEFINITION
KyberSlash	CVE-2024-37880 — timing attack against ML-KEM-512 on implementations where the Clang optimizer (versions through 18.x) converted constant-time `cmov` into a secret-dependent branch in `poly_frommsg`. Full key recovery demonstrated in < 10 min on Raspberry Pi 2
GoFetch	March 2024 microarchitectural attack against Apple M1/M2/M3 processors exploiting the Data Memory-Dependent Prefetcher (DMP) to extract ML-KEM and ML-DSA keys from constant-time implementations. Unmitigable on M1/M2 without kernel patches
Verification Theatre	ePrint 2026/192 (Nadim Kobeissi, February 2026) — catalog of 13 bugs that escaped formally-verified libraries (Cryspen libcrux / hpke-rs) deployed in Signal's SPQR post-quantum ratchet, including two FIPS 204 spec violations in the ML-DSA verifier

Signature and KEM Architectural Terms

TERM	DEFINITION
Composite Signature	Single-OID signature scheme combining a traditional algorithm with a post-quantum algorithm such that both must verify; standardized by `draft-ietf-lamps-pq-composite-sigs-16` with OID prototype arc `2.16.840.1.114027.80.8.1.*`
MTC	Merkle Tree Certificates — batch-signed alternative to per-domain X.509 certificates, chartered in IETF PLANTS WG (2026); replaces 14.7 KB composite certificate chains with ~736-byte inclusion proofs in TLS handshakes
PLANTS	IETF working group chartered 2026 to standardize Merkle Tree Certificates as a PQ-friendly replacement for Web PKI handshake-size bloat
SPQR	Sparse Post-Quantum Ratchet — Signal's October 2025 post-quantum upgrade to the Double Ratchet, forming the "Triple Ratchet" architecture; uses an ML-KEM "Braid Protocol" in place of the classical DH ratchet
HAWK	Lattice signature based on the Lattice Isomorphism Problem (module-LIP), submitted to NIST Additional Signatures Round 2; as of 2026 the sole remaining lattice candidate in Round 2, intended as a lattice-diversity backstop behind ML-DSA. Uses Falcon-like key derivation but the underlying hard problem is LIP, not NTRU.
Quorus	MPC-based threshold signature scheme for ML-DSA presented at NIST MPTS 2026; produces standard FIPS 204 signatures, making verifier-side rollout transparent

---

Module 4: ML-KEM Deep Dive

This module provides a comprehensive examination of ML-KEM (Module-Lattice Key Encapsulation Mechanism), the NIST-standardized quantum-resistant key encapsulation scheme. We'll explore how the theoretical foundations from Modules 2 and 3 come together in a practical, secure construction.

Module Prerequisites: Module 2 (Mathematical Foundations), Module 3 (Lattice Cryptography Theory)

Module Learning Objectives:

- Understand KEM syntax and IND-CCA2 security definitions
- Explain the K-PKE construction and its IND-CPA security
- Master the Fujisaki-Okamoto transform and implicit rejection
- Implement ML-KEM KeyGen, Encapsulation, and Decapsulation
- Apply compression techniques to optimize sizes
- Test implementations against KAT vectors

Estimated Time: 12-15 hours

Unit 4.1: KEM Concepts and Security Definitions

Prerequisites: Unit 3.4 (LWE to Public Key Encryption)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

- Define the KEM syntax (KeyGen, Encaps, Decaps)
- Distinguish KEMs from public key encryption
- State the IND-CCA2 security definition for KEMs
- Understand why KEMs are preferred over PKE in modern protocols

4.1.1 What Is a Key Encapsulation Mechanism?

The Key Exchange Problem

In cryptographic protocols, we often need to establish a shared secret key between two parties:

- TLS handshakes need session keys
- Secure messaging needs message keys
- File encryption needs content encryption keys

Traditional approach: Use public key encryption to send a chosen key. Better approach: Use a KEM to generate and share a random key.

KEM Syntax

A Key Encapsulation Mechanism consists of three algorithms:

KeyGen() → (ek, dk)

- Inputs: (internal randomness)
- Outputs: Encapsulation key ek (public), Decapsulation key dk (private)

Encaps(ek) → (c, K)

- Inputs: Encapsulation key ek
- Outputs: Ciphertext c, Shared secret K
- Note: The shared secret K is generated internally, not chosen by the caller

Decaps(dk, c) → K

- Inputs: Decapsulation key dk, Ciphertext c
- Outputs: Shared secret K (or failure indication)

Correctness Requirement: For any (ek, dk) ← KeyGen() and (c, K) ← Encaps(ek): Decaps(dk, c) = K

4.1.2 KEM vs Public Key Encryption

Why Not Just Use Encryption?

ASPECT	PUBLIC KEY ENCRYPTION	KEM
Sender chooses	The message to encrypt	Nothing (K is random)
Output	Ciphertext of message	Ciphertext + random shared secret
Typical use	Encrypt arbitrary data	Establish key for symmetric crypto
Security goal	Attacker can't learn message	Attacker can't learn shared secret

Advantages of KEMs:

1. **Simpler Security Analysis**
   - PKE security: "Can't learn message" is complex to define
   - KEM security: "K looks random" is cleaner
2. **No Padding Needed**
   - PKE often needs padding schemes (OAEP, PKCS#1)
   - KEMs output fixed-size random keys—no padding issues
3. **Natural Hybrid Encryption**
   - Use KEM to get K
   - Use K with symmetric encryption for actual data
   - This is exactly how TLS and other protocols work
4. **Better Bounds**
   - Security reductions for KEMs are often tighter

The Hybrid Encryption Pattern:

```
Sender:
1. (c, K) ← KEM.Encaps(recipient_ek)
2. ct ← AES-GCM.Encrypt(K, message)
3. Send (c, ct) to recipient

Recipient:
1. K ← KEM.Decaps(dk, c)
2. message ← AES-GCM.Decrypt(K, ct)
```

4.1.3 Security Definitions for KEMs

IND-CPA Security for KEMs

The IND-CPA game for KEMs tests whether an attacker can distinguish the real shared secret from random:

```
IND-CPA Game:
1. Challenger runs (ek, dk) ← KeyGen()
2. Challenger gives ek to adversary
3. Challenger runs (c*, K*₀) ← Encaps(ek)
4. Challenger picks random K*₁ of same length
5. Challenger flips coin b ∈ {0, 1}
6. Challenger gives (c*, K*_b) to adversary
7. Adversary outputs guess b'
8. Adversary wins if b' = b
```

Definition (IND-CPA Security): A KEM is IND-CPA secure if for all efficient adversaries A: |Pr[b' = b] - 1/2| ≤ negl(λ)

Informally: Even seeing the ciphertext, the adversary can't distinguish the real key from random.

IND-CCA2 Security for KEMs

IND-CCA2 (Chosen Ciphertext Attack) is stronger—the adversary can ask for decapsulation of any ciphertext except the challenge:

```
IND-CCA2 Game:
1. Challenger runs (ek, dk) ← KeyGen()
2. Challenger gives ek to adversary
3. Adversary can query decapsulation oracle: c → Decaps(dk, c)
4. Challenger creates challenge (c*, K*_b) as in IND-CPA
5. Adversary continues with decapsulation queries (but not on c*)
6. Adversary outputs guess b'
```

Why IND-CCA2 Matters:

In real protocols, attackers may:

- Modify ciphertexts and observe behavior
- Send malformed ciphertexts to probe for information
- Replay or mangle observed ciphertexts

IND-CCA2 security ensures the KEM remains secure even under such attacks.

ML-KEM Security:

- The underlying K-PKE is only IND-CPA secure

- The Fujisaki-Okamoto transform upgrades this to IND-CCA2
- ML-KEM achieves IND-CCA2 security

4.1.4 ML-KEM Overview

ML-KEM (FIPS 203)

ML-KEM is the NIST-standardized lattice-based KEM, formerly known as Kyber/CRYSTALS-Kyber.

Security Foundation: Module-LWE problem (covered in Unit 3.3)

Structure:

1. K-PKE: A CPA-secure encryption scheme from Module-LWE
2. FO Transform: Converts K-PKE to CCA2-secure KEM

Parameter Sets:

PARAMETER	ML-KEM-512	ML-KEM-768	ML-KEM-1024
Security Level	NIST 1	NIST 3	NIST 5
Equivalent to	AES-128	AES-192	AES-256
Module rank k	2	3	4
Encaps key	800 B	1,184 B	1,568 B
Decaps key	1,632 B	2,400 B	3,168 B
Ciphertext	768 B	1,088 B	1,568 B
Shared secret	32 B	32 B	32 B

Why These Sizes?

- Polynomial ring: R_q = Z_q[X]/(X^256 + 1), q = 3329
- Each polynomial has 256 coefficients
- Coefficients need ~12 bits ($\log_2(3329) \approx 11.7$)
- Module rank k determines how many polynomials per vector

4.1.5 Parameter Design Rationale: Why q = 3329 and n = 256?

ML-KEM's parameters are not arbitrary — they emerge from a chain of interlocking constraints:

Step 1: NTT requires roots of unity. The Number Theoretic Transform (NTT) is essential for fast polynomial multiplication. For an NTT of length n over Z_q, we need a primitive n-th root of unity in Z_q, which requires q ≡ 1 (mod n).

Step 2: The polynomial ring uses X^n + 1. The ring R_q = Z_q[X]/(X^n + 1) requires n to be a power of 2 (so X^n + 1 is irreducible over Z). ML-KEM uses n = 256, giving 256 coefficients per polynomial.

Step 3: Incomplete NTT requires 2n | (q-1). ML-KEM actually uses an *incomplete* NTT that reduces degree-255 polynomials to 128 degree-1 polynomials (not 256 scalars). This requires primitive 256th roots of unity but NOT 512th roots. The requirement is:

- q ≡ 1 (mod 256), i.e., 256 | (q-1)
- But q ≢ 1 (mod 512) — hence "incomplete" NTT

Step 4: q should be small for efficiency. Smaller q means coefficients fit in fewer bits, giving smaller keys and ciphertexts. We want the smallest prime q satisfying 256 | (q-1).

Checking: 256 × 13 + 1 = 3329 — **prime!** This is the smallest such prime, and it fits in 12 bits, allowing efficient implementation with 16-bit arithmetic.

Step 5: Module rank k scales security. Rather than increasing n (which would change the entire ring), ML-KEM scales security by increasing the module rank k:

- k = 2 → ML-KEM-512 (NIST Level 1, ~AES-128 equivalent)
- k = 3 → ML-KEM-768 (NIST Level 3, ~AES-192 equivalent)
- k = 4 → ML-KEM-1024 (NIST Level 5, ~AES-256 equivalent)

This means all three variants share the same NTT implementation, Barrett constants, and fundamental arithmetic — only the matrix dimensions change.

Step 6: Error distribution balances correctness and security. The secret and error vectors are sampled from the centered binomial distribution CBD(η):

- ML-KEM-512: $\eta_1=3$, $\eta_2=2$ (larger η_1 for secrets/errors provides tighter security at lower rank)
- ML-KEM-768/1024: $\eta_1=\eta_2=2$

Larger η provides more noise (harder problem) but increases decryption failure probability. The choice η = 2 for k ≥ 3 keeps failure probability negligible (< 2^{-140}).

Step 7: Compression parameters trade size for noise margin. Ciphertext compression (d_u, d_v) drops low-order bits to reduce sizes:

PARAMETER	ML-KEM-512	ML-KEM-768	ML-KEM-1024
d_u	10	10	11
d_v	4	4	5

More aggressive compression (fewer bits) reduces ciphertext size but increases the decryption noise margin, raising failure probability. These values were chosen to keep failure probability negligible while minimizing sizes.

The complete constraint chain:

```
NTT efficiency → n must be power of 2 → n = 256
              → q ≡ 1 (mod 256) and q prime
              → q = 3329 (smallest such prime)
Security scaling → k ∈ {2, 3, 4} for NIST Levels 1/3/5
Noise balance → η ∈ {2, 3} for correctness vs security
Compression → (d_u, d_v) for size vs failure probability
```

Worked Example: KEM Usage in a Protocol

Problem: Describe how ML-KEM would be used in a simplified TLS-like handshake.

Scenario:

- Client wants to establish secure connection to Server
- Server has ML-KEM key pair (ek, dk)
- Both sides need a shared session key

Protocol Flow:

```
Client                                    Server
   |                                        |
   |-------- ClientHello + supported ------>|
   |                                        |
   |<------ ServerHello + ek (public key) --|
   |                                        |
   | 1. (c, K) ← ML-KEM.Encaps(ek)          |
   | 2. session_key ← KDF(K)                |
   |                                        |
```

```
    |-------- Ciphertext c ----------------->|
    |                                        |
    |         1. K ← ML-KEM.Decaps(dk, c)    |
    |         2. session_key ← KDF(K)        |
    |                                        |
    |<======= Encrypted with session_key ===>|
```

Key Points:

1. Client doesn't choose the shared secret—ML-KEM generates it
2. Same K on both sides (by correctness)
3. Actual session key derived via KDF for domain separation
4. Attacker intercepting (ek, c) cannot compute K (by IND-CCA2)

Worked Example: Why IND-CPA Isn't Enough

Problem: Show a scenario where IND-CPA security fails but IND-CCA2 would protect.

Scenario: Malleability Attack

Suppose we use IND-CPA-secure K-PKE directly (without FO transform):

- Server decrypts and uses the key, or returns an error if decryption fails
- Attacker observes Server's behavior

Attack:

1. Attacker intercepts ciphertext c from legitimate encapsulation
2. Attacker modifies c to c' (using malleability of LWE encryption)
3. Attacker sends c' to Server
4. Server tries to decapsulate:
   - If K-PKE.Decrypt succeeds → uses some key K'
   - If K-PKE.Decrypt fails → returns error
5. Attacker learns 1 bit of information from this oracle

Repeating this attack many times can recover the original key!

Why IND-CCA2 Prevents This:

- With FO transform, Server always returns a key (implicit rejection)
- Attacker can't distinguish valid from invalid ciphertexts
- Modified ciphertexts produce unpredictable keys, not errors

C Implementation: KEM Interface

```
/*
 * Unit 4.1: KEM Interface Demonstration
 *
 * This code demonstrates the KEM API pattern using
 * a simplified (insecure) implementation.
 *
 * EDUCATIONAL PURPOSE ONLY - Not secure for any real use
 */

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <time.h>

/* ========== Simplified KEM Interface ========== */

#define EK_SIZE 32    /* Encapsulation key size (simplified) */
#define DK_SIZE 64    /* Decapsulation key size */
#define CT_SIZE 48    /* Ciphertext size */
#define SS_SIZE 32    /* Shared secret size */

typedef struct {
    uint8_t ek[EK_SIZE];    /* Public encapsulation key */
    uint8_t dk[DK_SIZE];    /* Private decapsulation key */
} kem_keypair_t;

typedef struct {
    uint8_t ct[CT_SIZE];    /* Ciphertext */
    uint8_t ss[SS_SIZE];    /* Shared secret */
} kem_encaps_result_t;

/* Simple XOR-based "encryption" (NOT SECURE - demo only) */
static void xor_bytes(uint8_t *out, const uint8_t *a, const uint8_t *b, size_t len) {
    for (size_t i = 0; i < len; i++) {
        out[i] = a[i] ^ b[i];
    }
}

/* Toy hash function (NOT SECURE - demo only) */
static void toy_hash(uint8_t *out, const uint8_t *in, size_t in_len) {
    uint32_t state = 0x12345678;
    for (size_t i = 0; i < in_len; i++) {
        state = state * 31 + in[i];
    }
    for (int i = 0; i < SS_SIZE; i++) {
        out[i] = (state >> (i % 4 * 8)) & 0xFF;
        state = state * 17 + i;
    }
}

/*
 * KeyGen: Generate KEM key pair
 */
int kem_keygen(kem_keypair_t *kp) {
```

```
    /* In real ML-KEM: sample matrix A, secrets s, compute t = As + e */
    /* Simplified: just random bytes */
    /* WARNING: rand() is NOT cryptographically secure. Use getrandom()/RAND_bytes() in
production. */

    for (int i = 0; i < EK_SIZE; i++) {
        kp->ek[i] = rand() & 0xFF;
    }

    /* dk contains ek plus additional secret material */
    memcpy(kp->dk, kp->ek, EK_SIZE);
    for (int i = EK_SIZE; i < DK_SIZE; i++) {
        kp->dk[i] = rand() & 0xFF;
    }

    return 0;  /* Success */
}

/*
 * Encaps: Create ciphertext and shared secret
 */
int kem_encaps(kem_encaps_result_t *result, const uint8_t *ek) {
    /* In real ML-KEM:
     * 1. Generate random m
     * 2. Derive K, r from m
     * 3. Encrypt m under ek using randomness r
     * 4. Hash to get final K
     */

    /* Simplified: random secret, "encrypt" with ek */
    uint8_t random_seed[SS_SIZE];
    for (int i = 0; i < SS_SIZE; i++) {
        random_seed[i] = rand() & 0xFF;
    }

    /* "Ciphertext" = seed XOR ek (padded) - NOT SECURE */
    memset(result->ct, 0, CT_SIZE);
    xor_bytes(result->ct, random_seed, ek, SS_SIZE);

    /* Add some additional "randomness" to ciphertext */
    for (int i = SS_SIZE; i < CT_SIZE; i++) {
        result->ct[i] = rand() & 0xFF;
    }

    /* Shared secret = hash of seed */
    toy_hash(result->ss, random_seed, SS_SIZE);

    return 0;  /* Success */
}

/*
 * Decaps: Recover shared secret from ciphertext
 */
int kem_decaps(uint8_t *ss, const uint8_t *ct, const uint8_t *dk) {
    /* In real ML-KEM:
     * 1. Decrypt ciphertext to get m'
     * 2. Re-derive K', r' from m'
     * 3. Re-encrypt and compare
     * 4. If match: output K'. If not: implicit rejection
     */
```

```
    /* Simplified: recover seed, hash it */
    uint8_t recovered_seed[SS_SIZE];

    /* ek is first EK_SIZE bytes of dk */
    xor_bytes(recovered_seed, ct, dk, SS_SIZE);

    /* Shared secret = hash of recovered seed */
    toy_hash(ss, recovered_seed, SS_SIZE);

    return 0;  /* Success */
}

/* ========== Demonstration ========== */

void print_hex(const char *label, const uint8_t *data, size_t len) {
    printf("%s: ", label);
    for (size_t i = 0; i < len && i < 16; i++) {
        printf("%02x", data[i]);
    }
    if (len > 16) printf("...");
    printf("\n");
}

void demo_kem_usage(void) {
    printf("=== KEM Usage Demonstration ===\n\n");

    kem_keypair_t keypair;
    kem_encaps_result_t encaps_result;
    uint8_t decaps_ss[SS_SIZE];

    /* Step 1: Key Generation (done once by recipient) */
    printf("Step 1: Key Generation\n");
    kem_keygen(&keypair);
    print_hex("  Encapsulation key (public)", keypair.ek, EK_SIZE);
    print_hex("  Decapsulation key (private)", keypair.dk, DK_SIZE);
    printf("\n");

    /* Step 2: Encapsulation (done by sender) */
    printf("Step 2: Encapsulation (sender side)\n");
    kem_encaps(&encaps_result, keypair.ek);
    print_hex("  Ciphertext", encaps_result.ct, CT_SIZE);
    print_hex("  Sender's shared secret", encaps_result.ss, SS_SIZE);
    printf("\n");

    /* Step 3: Decapsulation (done by recipient) */
    printf("Step 3: Decapsulation (recipient side)\n");
    kem_decaps(decaps_ss, encaps_result.ct, keypair.dk);
    print_hex("  Recipient's shared secret", decaps_ss, SS_SIZE);
    printf("\n");

    /* Step 4: Verify match */
    printf("Step 4: Verification\n");
    if (memcmp(encaps_result.ss, decaps_ss, SS_SIZE) == 0) {
        printf("  ✓ Shared secrets MATCH\n");
    } else {
        printf("  ✗ Shared secrets DO NOT MATCH (error!)\n");
    }
}
```

```
void demo_cca_attack_attempt(void) {
    printf("\n=== CCA Attack Demonstration ===\n\n");

    kem_keypair_t keypair;
    kem_encaps_result_t encaps_result;
    uint8_t modified_ct[CT_SIZE];
    uint8_t decaps_ss[SS_SIZE];

    kem_keygen(&keypair);
    kem_encaps(&encaps_result, keypair.ek);

    printf("Original shared secret:\n");
    print_hex("  K", encaps_result.ss, SS_SIZE);

    /* Attacker modifies ciphertext */
    memcpy(modified_ct, encaps_result.ct, CT_SIZE);
    modified_ct[0] ^= 0x01;  /* Flip one bit */

    printf("\nAttacker flips bit in ciphertext...\n");

    /* Decapsulate modified ciphertext */
    kem_decaps(decaps_ss, modified_ct, keypair.dk);

    printf("\nDecapsulation of modified ciphertext:\n");
    print_hex("  K'", decaps_ss, SS_SIZE);

    /* In this toy example, modification is detectable */
    if (memcmp(encaps_result.ss, decaps_ss, SS_SIZE) != 0) {
        printf("\n  Note: K ≠ K' (modified ciphertext gives different key)\n");
        printf("  In real ML-KEM with FO transform:\n");
        printf("  - This is 'implicit rejection'\n");
        printf("  - Attacker can't tell if ciphertext was valid\n");
    }
}

int main(void) {
    srand(time(NULL));

    printf("╔═══════════════════════════════════════════════════════════╗\n");
    printf("║  Unit 4.1: KEM Concepts and Security Definitions          ║\n");
    printf("╚═══════════════════════════════════════════════════════════╝\n\n");

    demo_kem_usage();
    demo_cca_attack_attempt();

    printf("\n");
    printf("═══════════════════════════════════════════════════════════\n");
    printf("Key Takeaways:\n");
    printf("  1. KEM generates random key (not sender-chosen message)\n");
    printf("  2. Three operations: KeyGen, Encaps, Decaps\n");
    printf("  3. Correctness: Decaps(dk, Encaps(ek)) = K\n");
    printf("  4. IND-CCA2 security protects against chosen-ciphertext attacks\n");
    printf("═══════════════════════════════════════════════════════════\n");

    return 0;
}
```

Compilation:

```
gcc -o unit4_1_kem unit4_1_kem.c
./unit4_1_kem
```

Expected Output (hex values vary per run due to `srand(time(NULL))`):

```
╔═══════════════════════════════════════════════════════════╗
║  Unit 4.1: KEM Concepts and Security Definitions          ║
╚═══════════════════════════════════════════════════════════╝

=== KEM Usage Demonstration ===

Step 1: Key Generation
  Encapsulation key (public): <hex>...
  Decapsulation key (private): <hex>...

Step 2: Encapsulation (sender side)
  Ciphertext: <hex>...
  Sender's shared secret: <hex>...

Step 3: Decapsulation (recipient side)
  Recipient's shared secret: <hex>...

Step 4: Verification
  ✓ Shared secrets MATCH

=== CCA Attack Demonstration ===

Original shared secret:
  K: <hex>...
Attacker flips bit in ciphertext...
Decapsulation of modified ciphertext:
  K': <hex>...
  Note: K ≠ K' (modified ciphertext gives different key)
  In real ML-KEM with FO transform:
  - This is 'implicit rejection'
  - Attacker can't tell if ciphertext was valid

═══════════════════════════════════════════════════════════
Key Takeaways:
  1. KEM generates random key (not sender-chosen message)
  2. Three operations: KeyGen, Encaps, Decaps
  3. Correctness: Decaps(dk, Encaps(ek)) = K
  4. IND-CCA2 security protects against chosen-ciphertext attacks
═══════════════════════════════════════════════════════════
```

Shared secrets always MATCH; modified ciphertext always produces a different key ($K \neq K'$).

Module 4 Summary: ML-KEM Deep Dive

Summary:

This module provided a comprehensive examination of ML-KEM:

UNIT	TOPIC	KEY CONCEPTS
4.1	KEM Concepts	KEM syntax, IND-CPA, IND-CCA2, KEM vs PKE
4.2	K-PKE	Module-LWE encryption, noise analysis, compression
4.3	FO Transform	Deterministic encryption, re-encryption, implicit rejection

Key Takeaways

1. **KEM Design:** KEMs generate random keys, simpler than PKE, perfect for hybrid encryption.
2. **K-PKE Foundation:** CPA-secure encryption from Module-LWE, with careful noise management.
3. **FO Transform:** Upgrades CPA to CCA2 security through deterministic encryption and implicit rejection.
4. **Constant-Time:** All secret-dependent operations must be constant-time for side-channel resistance.

Security Chain:

- Module-LWE hardness → K-PKE IND-CPA security → FO Transform → ML-KEM IND-CCA2 security

Implementation Requirements:

- ✓ Correct polynomial arithmetic (Units 2.3, 2.5)
- ✓ NTT for efficient multiplication
- ✓ Proper sampling (CBD, rejection sampling)
- ✓ Compression functions
- ✓ FO transform with implicit rejection
- ✓ Constant-time implementation throughout

Ready For: Module 5 covers digital signatures theory, and Module 6 applies similar techniques to ML-DSA.

Module 5 Summary: Digital Signatures Theory

Summary:

This module covered the theoretical foundations of post-quantum digital signatures:

UNIT	TOPIC	KEY CONCEPTS
5.1	Signature Security Definitions	EUF-CMA, SUF-CMA, attack models, nonce reuse
5.2	The Fiat-Shamir Transform	Sigma protocols, HVZK, special soundness, ROM
5.3	Lattice-Based Signatures	SIS, information leakage, rejection sampling

Key Takeaways

1. **Security Foundation:** EUF-CMA (Existential Unforgeability under Chosen Message Attack) is the standard security notion for signatures.
2. **From ID to Signature:** The Fiat-Shamir transform converts interactive identification protocols into non-interactive signatures.
3. **Lattice Challenge:** Unlike discrete log, lattice-based ID protocols leak information. Rejection sampling solves this at the cost of multiple signing attempts.
4. **ML-DSA Framework:** Fiat-Shamir with Aborts combines non-interactive signing with rejection sampling, forming the basis of ML-DSA.

Mathematical Prerequisites Satisfied:

- ✓ Sigma protocol structure (commit-challenge-respond)
- ✓ Special soundness and HVZK properties
- ✓ Random oracle model and its role in proofs
- ✓ SIS problem and lattice-based hardness
- ✓ Information-theoretic leakage analysis
- ✓ Rejection sampling technique

Ready For: Module 6 (ML-DSA Deep Dive) which covers the concrete implementation of these ideas in the NIST standard.

Module 6: ML-DSA Deep Dive

This module provides a comprehensive treatment of ML-DSA (Module-Lattice Digital Signature Algorithm), the NIST-standardized post-quantum signature scheme formerly known as CRYSTALS-Dilithium. Building on the theoretical foundations from Module 5, we'll explore every component of the algorithm in detail.

Prerequisites:

- Module 2: Mathematical Foundations (polynomial arithmetic, NTT)
- Module 3: Lattice Cryptography Theory (LWE, SIS problems)
- Module 5: Digital Signatures Theory (Fiat-Shamir, rejection sampling)

Module Learning Objectives:

- Master the complete ML-DSA algorithm at implementation level
- Understand all security-critical design decisions
- Implement constant-time ML-DSA operations in C
- Debug and test ML-DSA implementations effectively

Unit 6.1: ML-DSA Structure and Security Model

Prerequisites: Module 2 (Mathematical Foundations), Module 3 (Lattice Cryptography), Module 5 (Digital Signatures Theory)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

- Describe the complete ML-DSA algorithm architecture
- Explain how ML-DSA achieves EUF-CMA security
- Identify the relationship between ML-DSA and the underlying hard problems
- Compare ML-DSA parameter sets and their security levels

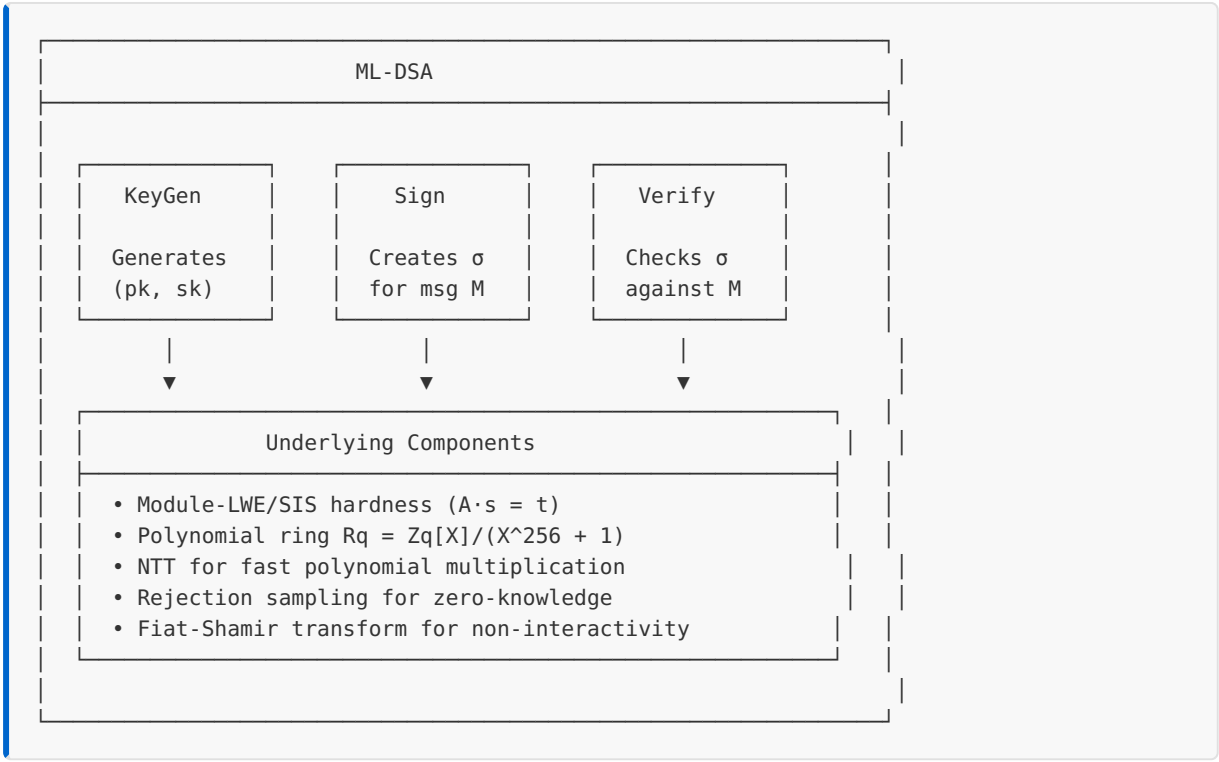
- Explain the role of each component in the signature scheme

6.1.1 ML-DSA Overview

ML-DSA is a digital signature scheme based on the Module-LWE and Module-SIS problems. It provides:

- **Post-quantum security:** Resistant to known quantum attacks
- **EUF-CMA security:** Existential unforgeability under chosen message attack
- **Deterministic signing:** (With optional hedged randomness)
- **Compact signatures:** Smaller than many other PQC signature schemes

High-Level Structure:



6.1.2 ML-DSA Algorithm Syntax

Key Generation: KeyGen() → (pk, sk)

```
Input: (none - uses randomness)
Output: Public key pk, Secret key sk

1. Sample seed ξ ← {0,1}^256
2. Expand ξ to get (ρ, ρ', K)
3. Generate matrix A ∈ Rq^(k×l) from ρ
4. Sample secret vectors s1 ∈ Sl^η, s2 ∈ Sk^η  (small coefficients)
5. Compute t = A·s1 + s2
6. Compress t into (t1, t0) where t = t1·2^d + t0
7. pk = (ρ, t1)
8. sk = (ρ, K, tr, s1, s2, t0)  where tr = H(pk)
```

Signing: Sign(sk, M) → σ

```
Input: Secret key sk, Message M
Output: Signature σ

1. Compute μ = H(tr || M)
2. Compute ρ' = H(K || rnd || μ)         // rnd is optional randomness
3. Initialize counter κ = 0
4. REPEAT:
   a. Sample y ∈ Sl^(γ1-1) from ExpandMask(ρ', κ)  // masking vector
   b. κ = κ + l                           // advance counter by l (one per polynomial in
y)
   c. Compute w = A·y
   d. Decompose w into (w1, w0)
   e. Compute challenge c̃ = H(μ || w1)
   f. Compute c ∈ Rq by expanding c̃     // challenge polynomial
   g. Compute z = y + c·s1
   h. IF ‖z‖∞ ≥ γ1 - β: CONTINUE        // rejection check 1
   i. Compute r0 = LowBits(w - c·s2)
   j. IF ‖r0‖∞ ≥ γ2 - β: CONTINUE       // rejection check 2
   k. Compute ct0 = c·t0
   l. Compute hint h = MakeHint(-ct0, w - c·s2 + ct0)
   m. IF ‖ct0‖∞ ≥ γ2 OR |h| > ω: CONTINUE  // rejection check 3
5. Return σ = (c̃, z, h)
```

Verification: Verify(pk, M, σ) → {accept, reject}

```
Input: Public key pk, Message M, Signature σ = (c̃, z, h)
Output: accept or reject

1. IF ‖z‖∞ ≥ γ1 - β: RETURN reject
2. Expand A from ρ in pk
3. Compute tr = H(pk), then μ = H(tr || M)
4. Expand c from c̃
5. Compute w'1 = UseHint(h, A·z - c·t1·2^d)
6. Compute c̃' = H(μ || w'1)
7. IF c̃ ≠ c̃': RETURN reject
8. RETURN accept
```

6.1.3 Parameter Sets

ML-DSA defines three parameter sets targeting different security levels:

PARAMETER	ML-DSA-44	ML-DSA-65	ML-DSA-87
NIST Level	2	3	5
Classical Security	~128 bits	~192 bits	~256 bits
q	8380417	8380417	8380417
n	256	256	256
(k, l)	(4, 4)	(6, 5)	(8, 7)
η	2	4	2
γ1	2^17	2^19	2^19
γ2	(q-1)/88	(q-1)/32	(q-1)/32
τ	39	49	60
β	τ·η	τ·η	τ·η
ω	80	55	75
d	13	13	13

Parameter Meanings:

- **q = 8380417:** The prime modulus, chosen as q = 2^23 - 2^13 + 1 for NTT efficiency
- **n = 256:** Polynomial degree (ring dimension)
- **(k, l):** Matrix dimensions - A is k×l, signatures have l polynomials
- **η:** Bound on secret key coefficients (s1, s2 ∈ {-η, ..., η})
- **γ1:** Bound for masking vector y coefficients
- **γ2:** Decomposition parameter for high/low bits
- **τ:** Number of ±1 coefficients in challenge polynomial c
- **β = τ·η:** Rejection bound derived from challenge weight and secret bound
- **ω:** Maximum number of 1s allowed in hint vector
- **d:** Number of dropped bits in t1 compression

Size Comparison:

PARAMETER SET	PUBLIC KEY	SECRET KEY	SIGNATURE
ML-DSA-44	1312 bytes	2560 bytes	2420 bytes
ML-DSA-65	1952 bytes	4032 bytes	3309 bytes
ML-DSA-87	2592 bytes	4896 bytes	4627 bytes

Compare with classical schemes:

- RSA-2048: 256 byte signatures, 256 byte public keys
- ECDSA-P256: 64 byte signatures, 64 byte public keys
- Ed25519: 64 byte signatures, 32 byte public keys

ML-DSA signatures are larger but provide quantum resistance.

6.1.4 Security Foundation

Module-SIS Connection:

The security of ML-DSA relies on the hardness of finding short vectors. Given:

- Public matrix A ∈ Rq^(k×l)
- Public vector t = A·s1 + s2

An attacker cannot find (s1, s2) because:

1. The values are hidden by the module structure
2. Finding short solutions to A·x ≈ t is the Module-SIS problem
3. The best known attacks are exponential in the security parameter

Fiat-Shamir Security:

The signature scheme achieves EUF-CMA security through:

1. **Commitment hiding:** The masking vector y hides s1 in z = y + c·s1
2. **Challenge binding:** The hash c̃ = H(μ || w1) binds challenge to message
3. **Rejection sampling:** Ensures z distribution is independent of s1

Security Reduction (Informal):

```
If an attacker can forge ML-DSA signatures
    ↓
They can solve the Module-SIS problem
    ↓
This contradicts the assumed hardness of M-SIS
    ↓
Therefore, ML-DSA is secure (in the ROM)
```

Concrete Security Claims:

PARAMETER SET	CLASSICAL	QUANTUM (GROVER)
ML-DSA-44	2^128	2^107
ML-DSA-65	2^192	2^153
ML-DSA-87	2^256	2^199

6.1.5 Comparison with Other Signature Schemes

ML-DSA vs SLH-DSA (SPHINCS+):

ASPECT	ML-DSA	SLH-DSA
Assumption	Module-LWE/SIS	Hash function security
Signatures	~2.4-4.6 KB	~7.9-49 KB
Public Keys	~1.3-2.6 KB	~32-64 bytes
Signing Speed	Fast	Slower
Verification	Fast	Moderate
Conservative	Lattice-based	Hash-based (more conservative)

ML-DSA vs FALCON:

ASPECT	ML-DSA	FALCON
Signatures	~2.4-4.6 KB	~0.7-1.3 KB
Complexity	Simpler	Complex (Gaussian sampling)
Side-channels	Easier to protect	Harder to protect
NIST Status	Standardized	Standardized

When to Choose ML-DSA:

✓ General-purpose signatures (documents, code signing) ✓ When signature size is acceptable ✓ When implementation simplicity matters ✓ When constant-time implementation is required ✓ For NIST compliance requirements

6.1.6 The Role of Each Component

Understanding the signature σ = (c̃, z, h):

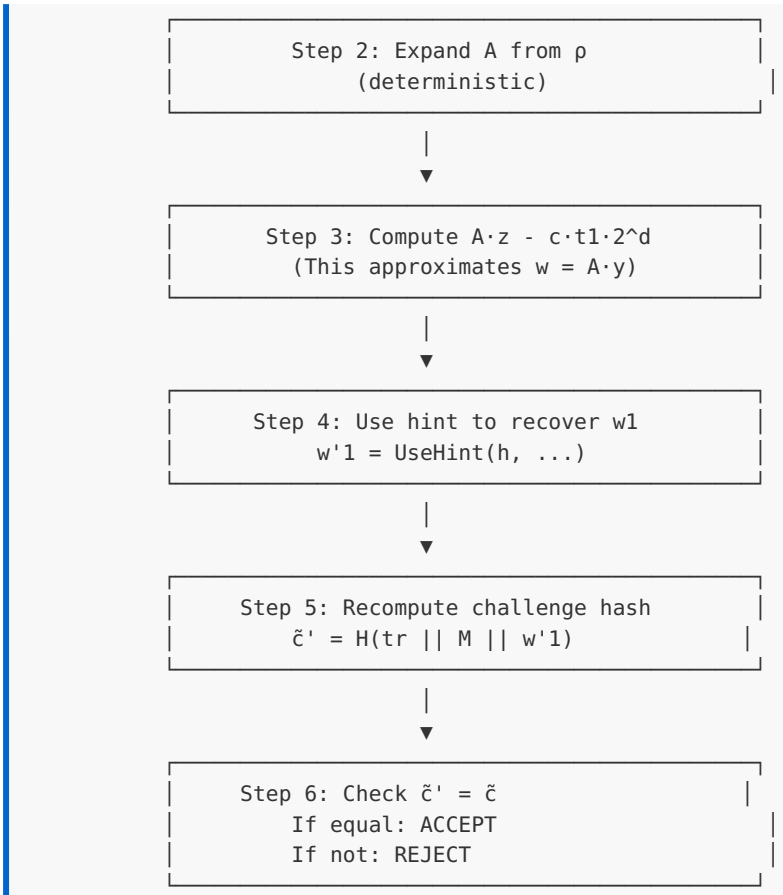
```
σ = (c̃, z, h)
     │   │  │
     │   │  └── Hint: Helps verifier recover w1 without knowing s2
     │   │
     │   └── Response: z = y + c·s1 (masked secret)
     │
     └── Challenge hash: Commitment to message and w1
```

Why each piece is needed:

1. **c̃ (challenge hash):**
   - Binds the challenge to the message
   - Makes signature non-malleable
   - Part of the Fiat-Shamir transform
2. **z (response vector):**
   - Proves knowledge of s1
   - Masked by random y to hide s1
   - Must pass rejection sampling checks
3. **h (hint):**
   - Allows verification without revealing s2
   - Encodes which coefficients need adjustment
   - Keeps signature size small (vs sending full w)

Verification Flow:

```
┌─────────────────────────────────────────────┐
│              Verifier receives              │
│         pk = (ρ, t1), M, σ = (c̃, z, h)      │
└─────────────────────────────────────────────┘
                    │
                    ▼
┌─────────────────────────────────────────────┐
│         Step 1: Check z is small            │
│            ‖z‖∞ < γ1 - β                    │
└─────────────────────────────────────────────┘
                    │
                    ▼
```



6.1.7 C Implementation: ML-DSA Structure

```
#include <stdint.h>
#include <stdio.h>
#include <string.h>

/*
 * ML-DSA Structure Demonstration
 * This code illustrates the data structures and flow of ML-DSA
 * NOT a secure implementation - for educational purposes only
 */

/* ML-DSA-65 parameters */
#define MLDSA_N 256
#define MLDSA_Q 8380417
#define MLDSA_K 6
#define MLDSA_L 5
#define MLDSA_ETA 4
#define MLDSA_GAMMA1 (1 << 19)  /* 2^19 */
#define MLDSA_GAMMA2 ((MLDSA_Q - 1) / 32)
#define MLDSA_TAU 49
#define MLDSA_BETA (MLDSA_TAU * MLDSA_ETA)
#define MLDSA_OMEGA 55
#define MLDSA_D 13

/* Polynomial in Rq */
typedef struct {
    int32_t coeffs[MLDSA_N];
```

```
} poly;

/* Vector of polynomials */
typedef struct {
    poly vec[MLDSA_L];
} polyvecl;

typedef struct {
    poly vec[MLDSA_K];
} polyveck;

/* Public key structure */
typedef struct {
    uint8_t rho[32];           /* Seed for matrix A */
    polyveck t1;               /* High bits of t */
} mldsa_pk;

/* Secret key structure */
typedef struct {
    uint8_t rho[32];           /* Seed for matrix A */
    uint8_t K[32];             /* Signing key seed */
    uint8_t tr[64];            /* H(pk) - public key hash */
    polyvecl s1;               /* Secret vector s1 */
    polyveck s2;               /* Secret vector s2 */
    polyveck t0;               /* Low bits of t */
} mldsa_sk;

/* Signature structure */
typedef struct {
    uint8_t c_tilde[48];       /* Challenge hash */
    polyvecl z;                /* Response vector */
    uint8_t h[MLDSA_OMEGA + MLDSA_K];  /* Hint */
} mldsa_sig;

/* Reduce coefficient to range [0, q) */
int32_t mod_q(int32_t a) {
    a = a % MLDSA_Q;
    if (a < 0) a += MLDSA_Q;
    return a;
}

/* Check if polynomial has coefficients bounded by B */
int poly_check_bound(const poly *p, int32_t bound) {
    for (int i = 0; i < MLDSA_N; i++) {
        int32_t c = p->coeffs[i];
        /* Reduce to centered representation */
        if (c > MLDSA_Q/2) c -= MLDSA_Q;
        if (c < -bound || c > bound) {
            return 0;  /* Exceeds bound */
        }
    }
    return 1;  /* Within bound */
}

/* Check if vector has coefficients bounded by B */
int polyvecl_check_bound(const polyvecl *v, int32_t bound) {
    for (int i = 0; i < MLDSA_L; i++) {
        if (!poly_check_bound(&v->vec[i], bound)) {
            return 0;
        }
```

```
    }
    return 1;
}

/* High bits: returns high part of decomposition */
int32_t highbits(int32_t r, int32_t alpha) {
    int32_t r_plus = r % MLDSA_Q;
    if (r_plus < 0) r_plus += MLDSA_Q;

    /* Center r in (-q/2, q/2] */
    int32_t r_centered = r_plus;
    if (r_centered > MLDSA_Q/2) r_centered -= MLDSA_Q;

    /* Compute r1 = high bits */
    int32_t r1 = (r_plus + alpha/2) / alpha;

    /* Handle wrap-around */
    if (r1 == (MLDSA_Q - 1) / alpha + 1) r1 = 0;

    return r1;
}

/* Low bits: returns low part of decomposition */
int32_t lowbits(int32_t r, int32_t alpha) {
    int32_t r1 = highbits(r, alpha);
    int32_t r0 = r - r1 * alpha;

    /* Center r0 */
    if (r0 > alpha/2) r0 -= alpha;
    if (r0 < -alpha/2) r0 += alpha;

    return r0;
}

/* MakeHint: compute hint bit */
int makehint(int32_t z, int32_t r) {
    int32_t r1 = highbits(r, 2 * MLDSA_GAMMA2);
    int32_t v1 = highbits(r + z, 2 * MLDSA_GAMMA2);
    return (r1 != v1) ? 1 : 0;
}

/* UseHint: recover high bits using hint */
int32_t usehint(int hint, int32_t r) {
    int32_t m = (MLDSA_Q - 1) / (2 * MLDSA_GAMMA2);
    int32_t r1 = highbits(r, 2 * MLDSA_GAMMA2);
    int32_t r0 = lowbits(r, 2 * MLDSA_GAMMA2);

    if (hint == 0) {
        return r1;
    }

    if (r0 > 0) {
        return (r1 + 1) % m;
    } else {
        return (r1 - 1 + m) % m;
    }
}

/* Demonstration: Print parameter info */
void print_parameters(void) {
```

```
    printf("ML-DSA-65 Parameters:\n");
    printf("====================\n");
    printf("Ring dimension n = %d\n", MLDSA_N);
    printf("Modulus q = %d (= 2^23 - 2^13 + 1)\n", MLDSA_Q);
    printf("Matrix dimensions (k, l) = (%d, %d)\n", MLDSA_K, MLDSA_L);
    printf("Secret coefficient bound η = %d\n", MLDSA_ETA);
    printf("Masking bound γ1 = %d (= 2^19)\n", MLDSA_GAMMA1);
    printf("Decomposition γ2 = %d (= (q-1)/32)\n", MLDSA_GAMMA2);
    printf("Challenge weight τ = %d\n", MLDSA_TAU);
    printf("Rejection bound β = τ·η = %d\n", MLDSA_BETA);
    printf("Max hint weight ω = %d\n", MLDSA_OMEGA);
    printf("Dropped bits d = %d\n", MLDSA_D);
    printf("\n");

    /* Size calculations */
    size_t pk_size = 32 + (MLDSA_K * MLDSA_N * 10) / 8;  /* 10 bits per t1 coeff */
    size_t sk_size = 32 + 32 + 64 +
                     (MLDSA_L * MLDSA_N * 4) / 8 +   /* s1: 4 bits per coeff */
                     (MLDSA_K * MLDSA_N * 4) / 8 +   /* s2: 4 bits per coeff */
                     (MLDSA_K * MLDSA_N * 13) / 8;   /* t0: 13 bits per coeff */
    size_t sig_size = 32 + (MLDSA_L * MLDSA_N * 20) / 8 + MLDSA_OMEGA + MLDSA_K;

    printf("Approximate sizes:\n");
    printf("  Public key:  ~%zu bytes\n", pk_size);
    printf("  Secret key:  ~%zu bytes\n", sk_size);
    printf("  Signature:   ~%zu bytes\n", sig_size);
}

/* Demonstration: Show decomposition */
void demo_decomposition(void) {
    printf("\nDecomposition Demo:\n");
    printf("===================\n");

    int32_t alpha = 2 * MLDSA_GAMMA2;
    printf("Using α = 2·γ2 = %d\n\n", alpha);

    int32_t test_values[] = {0, 100000, 1000000, 4000000, 8000000};

    for (int i = 0; i < 5; i++) {
        int32_t r = test_values[i];
        int32_t r1 = highbits(r, alpha);
        int32_t r0 = lowbits(r, alpha);

        printf("r = %8d: r1 (high) = %3d, r0 (low) = %8d\n", r, r1, r0);
        printf("  Verify: r1·α + r0 = %d·%d + %d = %d %s\n",
               r1, alpha, r0, r1 * alpha + r0,
               (mod_q(r1 * alpha + r0) == mod_q(r)) ? "✓" : "✗");
    }
}

/* Demonstration: Show hint mechanism */
void demo_hint(void) {
    printf("\nHint Mechanism Demo:\n");
    printf("====================\n");

    /* Hint is used when the verifier computes A·z - c·t1·2^d
     * which equals A·y - c·s2 + c·t0 (approximately)
     * The hint helps recover HighBits(A·y) from this value
     */
```

```
    printf("The hint encodes when adding a value changes the high bits.\n\n");

    int32_t r = 260000;  /* Some value */
    int32_t z_values[] = {0, 10000, 50000, -30000, -80000};

    printf("Base value r = %d\n", r);
    printf("HighBits(r) = %d\n\n", highbits(r, 2 * MLDSA_GAMMA2));

    for (int i = 0; i < 5; i++) {
        int32_t z = z_values[i];
        int h = makehint(z, r);
        int32_t r1_original = highbits(r, 2 * MLDSA_GAMMA2);
        int32_t r1_summed = highbits(r + z, 2 * MLDSA_GAMMA2);
        int32_t r1_recovered = usehint(h, r + z);

        printf("z = %7d: hint = %d, HighBits(r) = %d, HighBits(r+z) = %d, UseHint = %d\n",
               z, h, r1_original, r1_summed, r1_recovered);
    }
}

/* Demonstration: Rejection bound checking */
void demo_rejection_bounds(void) {
    printf("\nRejection Bound Demo:\n");
    printf("=====================\n");

    printf("In ML-DSA signing:\n");
    printf("1. Sample masking y with coefficients < γ1 = %d\n", MLDSA_GAMMA1);
    printf("2. Compute z = y + c·s1\n");
    printf("3. Reject if any coefficient of z has |z_i| ≥ γ1 - β = %d\n",
           MLDSA_GAMMA1 - MLDSA_BETA);
    printf("\n");

    printf("Why γ1 - β?\n");
    printf("  - s1 coefficients: |s1_i| ≤ η = %d\n", MLDSA_ETA);
    printf("  - Challenge c has τ = %d non-zero (±1) coefficients\n", MLDSA_TAU);
    printf("  - Product c·s1 has coefficients bounded by ~τ·η = β = %d\n", MLDSA_BETA);
    printf("  - If y_i ∈ [-(γ1-1), γ1-1], then z_i could be as large as γ1-1+β\n");
    printf("  - Rejection threshold γ1-β ensures z reveals nothing about s1\n");
    printf("\n");

    printf("Expected rejection probability: ~1/M where M ≈ 4-7\n");
    printf("This means ~4-7 loop iterations on average per signature.\n");
}

int main(void) {
    print_parameters();
    demo_decomposition();
    demo_hint();
    demo_rejection_bounds();

    printf("\n");
    printf("Key Structure Sizes (ML-DSA-65):\n");
    printf("================================\n");
    printf("Public key (pk):\n");
    printf("  - rho:  32 bytes (seed for A)\n");
    printf("  - t1:   %d polynomials × 10 bits × 256 coeffs / 8 = %d bytes\n",
           MLDSA_K, MLDSA_K * 256 * 10 / 8);
    printf("\n");
    printf("Secret key (sk):\n");
    printf("  - rho:  32 bytes\n");
```

```
    printf("  - K:    32 bytes (signing key seed)\n");
    printf("  - tr:   64 bytes (H(pk))\n");
    printf("  - s1:   %d polynomials (small coefficients)\n", MLDSA_L);
    printf("  - s2:   %d polynomials (small coefficients)\n", MLDSA_K);
    printf("  - t0:   %d polynomials (low bits of t)\n", MLDSA_K);
    printf("\n");
    printf("Signature (σ):\n");
    printf("  - c̃:    48 bytes (challenge hash)\n");
    printf("  - z:    %d polynomials × 20 bits × 256 coeffs / 8 = %d bytes\n",
           MLDSA_L, MLDSA_L * 256 * 20 / 8);
    printf("  - h:    ≤ ω + k = %d + %d = %d bytes\n",
           MLDSA_OMEGA, MLDSA_K, MLDSA_OMEGA + MLDSA_K);

    return 0;
}
```

Expected Output:

```
ML-DSA-65 Parameters:
=====================
Ring dimension n = 256
Modulus q = 8380417 (= 2^23 - 2^13 + 1)
Matrix dimensions (k, l) = (6, 5)
Secret coefficient bound η = 4
Masking bound γ1 = 524288 (= 2^19)
Decomposition γ2 = 261888 (= (q-1)/32)
Challenge weight τ = 49
Rejection bound β = τ·η = 196
Max hint weight ω = 55
Dropped bits d = 13

Approximate sizes:
  Public key:  ~1952 bytes
  Secret key:  ~4032 bytes
  Signature:   ~3309 bytes

Decomposition Demo:
===================
Using α = 2·γ2 = 523776

r =        0: r1 (high) =   0, r0 (low) =        0
  Verify: r1·α + r0 = 0·523776 + 0 = 0 ✓
r =   100000: r1 (high) =   0, r0 (low) =   100000
  Verify: r1·α + r0 = 0·523776 + 100000 = 100000 ✓
r =  1000000: r1 (high) =   2, r0 (low) =   -47552
  Verify: r1·α + r0 = 2·523776 + -47552 = 1000000 ✓
r =  4000000: r1 (high) =   8, r0 (low) =  -190208
  Verify: r1·α + r0 = 8·523776 + -190208 = 4000000 ✓
r =  8000000: r1 (high) =  15, r0 (low) =   143360
  Verify: r1·α + r0 = 15·523776 + 143360 = 8000000 ✓

Hint Mechanism Demo:
====================
The hint encodes when adding a value changes the high bits.

Base value r = 260000
HighBits(r) = 0
```

```
z =       0: hint = 0, HighBits(r) = 0, HighBits(r+z) = 0, UseHint = 0
z =   10000: hint = 0, HighBits(r) = 0, HighBits(r+z) = 1, UseHint = 0
z =   50000: hint = 0, HighBits(r) = 0, HighBits(r+z) = 1, UseHint = 0
z =  -30000: hint = 0, HighBits(r) = 0, HighBits(r+z) = 0, UseHint = 0
z =  -80000: hint = 0, HighBits(r) = 0, HighBits(r+z) = 0, UseHint = 0

Rejection Bound Demo:
====================
In ML-DSA signing:
1. Sample masking y with coefficients < γ1 = 524288
2. Compute z = y + c·s1
3. Reject if any coefficient of z has |z_i| ≥ γ1 - β = 524092

Why γ1 - β?
  - s1 coefficients: |s1_i| ≤ η = 4
  - Challenge c has τ = 49 non-zero (±1) coefficients
  - Product c·s1 has coefficients bounded by ~τ·η = β = 196
  - If y_i ∈ [-(γ1-1), γ1-1], then z_i could be as large as γ1-1+β
  - Rejection threshold γ1-β ensures z reveals nothing about s1

Expected rejection probability: ~1/M where M ≈ 4-7
This means ~4-7 loop iterations on average per signature.

Key Structure Sizes (ML-DSA-65):
================================
Public key (pk):
  - rho:  32 bytes (seed for A)
  - t1:   6 polynomials × 10 bits × 256 coeffs / 8 = 1920 bytes

Secret key (sk):
  - rho:  32 bytes
  - K:    32 bytes (signing key seed)
  - tr:   64 bytes (H(pk))
  - s1:   5 polynomials (small coefficients)
  - s2:   6 polynomials (small coefficients)
  - t0:   6 polynomials (low bits of t)

Signature (σ):
  - c̃:    48 bytes (challenge hash)
  - z:    5 polynomials × 20 bits × 256 coeffs / 8 = 3200 bytes
  - h:    ≤ ω + k = 55 + 6 = 61 bytes
```

6.1.8 Exercises

Exercise 6.1.1 *(Beginner)*

Explain why ML-DSA uses γ1 = 2^19 for ML-DSA-65 but γ1 = 2^17 for ML-DSA-44, despite ML-DSA-44 having smaller secrets (η = 2 vs η = 4).

Solution

The choice of γ1 involves a tradeoff between:

1. **Security:** Larger γ1 means y can hide larger perturbations from c·s1
2. **Signature size:** Larger γ1 requires more bits to encode z coefficients

3. **Rejection rate:** Larger γ1 relative to the secret expansion reduces rejections

   For ML-DSA-44:

- η = 2, τ = 39, so β = 78
- γ1 = 2^17 = 131072
- Ratio γ1/β ≈ 1681

   For ML-DSA-65:

- η = 4, τ = 49, so β = 196
- γ1 = 2^19 = 524288
- Ratio γ1/β ≈ 2675

   ML-DSA-65 uses larger η, which means c·s1 has larger coefficients. To maintain a similar rejection rate and security margin, γ1 must increase proportionally. The larger γ1 means z requires 20 bits per coefficient instead of 18, contributing to larger signatures.

   Key insight: γ1 scales with the expected magnitude of c·s1 to maintain consistent rejection probabilities and security margins across parameter sets.

Exercise 6.1.2 *(Intermediate)*

For ML-DSA-65, calculate:

1. The maximum possible magnitude of a coefficient in c·s1
2. The probability that a random coefficient in z exceeds the bound γ1 - β

Solution

Part 1: Maximum magnitude of c·s1 coefficient

The challenge polynomial c has exactly τ = 49 coefficients that are ±1, rest are 0. The secret s1 has coefficients bounded by |s1_i| ≤ η = 4.

When multiplying polynomials in Rq = Zq[X]/(X^256 + 1):

- Coefficient i of product = Σ_j c_j · s1_{i-j mod 256} (with sign changes for wraparound)
- Each coefficient of the product sums at most τ = 49 terms
- Each term has magnitude at most η = 4
- Maximum magnitude: τ · η = 49 · 4 = 196 = β

Part 2: Probability z coefficient exceeds bound

$z_i = y_i + (c \cdot s1)_i$

y_i is uniform in $[-(\gamma1-1), \gamma1-1] = [-524287, 524287]$ $(c \cdot s1)_i$ has magnitude at most $\beta = 196$

For z_i to exceed $\gamma1 - \beta = 524092$:

- Need $y_i + (c \cdot s1)_i > 524092$ or < -524092
- Worst case: y_i near boundary AND $(c \cdot s1)_i$ pushes it over

The coefficients $(c \cdot s1)_i$ are concentrated around 0 (sum of 49 random $\pm\eta$ values).

Simplified analysis (assuming $(c \cdot s1)_i$ distributed roughly uniformly in $[-\beta, \beta]$):

- y_i uniform in range of size $2 \cdot (\gamma1-1) = 1048574$
- z_i must land in "rejection zone" of size $\sim 2\beta = 392$ on each side
- Approximate rejection probability per coefficient: $2\beta / (2\gamma1) = \beta/\gamma1 = 196/524288 \approx 0.00037$

For $L \cdot n = 5 \cdot 256 = 1280$ coefficients:

- Probability at least one exceeds: $\approx 1 - (1 - 0.00037)^{1280} \approx 0.38$

This rough estimate suggests ~38% rejection from z bound alone, consistent with ML-DSA requiring ~4-7 attempts on average (other rejection checks also contribute).

Key insight: The rejection rate is tuned to balance signature size (smaller $\gamma1$ = smaller z encoding) against signing speed (more rejections = more attempts).

Exercise 6.1.3 *(Advanced)*

The hint h in ML-DSA encodes at most ω positions where high bits need adjustment. Why does having too many hints indicate a problem, and why is $|h| \leq \omega$ a rejection criterion?

Solution

Why hints are needed:

During verification, the verifier computes:

```
A·z - c·t1·2^d = A·(y + c·s1) - c·(t - s2)·(2^d / 2^d adjustment)
              ≈ A·y + c·(A·s1 - t + s2)
              = A·y - c·s2 + c·t0  (approximately)
              = w - c·s2 + c·t0
```

The verifier needs HighBits(w) = HighBits(A·y) to check the challenge. But they compute HighBits(w - c·s2 + c·t0), which may differ.

The hint records which polynomial coefficients need adjustment.

Why many hints indicate a problem:

1. **Honest signing:** When signing correctly, c·t0 and c·s2 have small coefficients (bounded by design). They rarely cause high-bit changes.
2. **Expected hint count:** For honest signers, the expected number of hints is small. The parameter ω is set to accommodate honest signatures with high probability.
3. **Forgery attempts:** An attacker trying to forge might produce signatures where the computed values don't align well, requiring many hint corrections.

Why |h| ≤ ω is checked:

- ω = 55 for ML-DSA-65 (ω = 80 for ML-DSA-44, ω = 75 for ML-DSA-87)
- Honest signatures almost never need more than ω hints
- If |h| > ω, either:
  - The signature was generated incorrectly
  - An attacker is trying something unusual
- Rejecting these cases is safe (doesn't reject honest signatures) and may catch some attacks

Key insight: The hint bound serves as a "sanity check" - honestly generated signatures naturally have few hints, so requiring |h| ≤ ω filters out malformed signatures while accepting all valid ones.

Exercise 6.1.4 *(Advanced)*

Implement the `HighBits` and `LowBits` decomposition functions for ML-DSA-65. Verify that for any input r, the reconstruction r1·α + r0 equals r (mod q).

Solution

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define Q 8380417
#define GAMMA2 ((Q - 1) / 32)  /* 261888 */
#define ALPHA (2 * GAMMA2)      /* 523776 */

/*
 * Power2Round: Decompose r into (r1, r0) where r = r1·2^d + r0
 * Used for t = t1·2^d + t0 decomposition
 */
void power2round(int32_t r, int32_t *r1, int32_t *r0, int d) {
    /* Ensure r is positive */
    r = r % Q;
    if (r < 0) r += Q;

    /* r0 = r mod 2^d (centered) */
    *r0 = r & ((1 << d) - 1);
    if (*r0 > (1 << (d-1))) {
        *r0 -= (1 << d);
    }

    /* r1 = (r - r0) / 2^d */
    *r1 = (r - *r0) >> d;
}

/*
 * Decompose: Split r into high and low parts for signing
 * Returns (r1, r0) such that r = r1·α + r0 with r0 small
 */
void decompose(int32_t r, int32_t *r1, int32_t *r0) {
    /* Ensure r is in [0, q) */
    r = r % Q;
    if (r < 0) r += Q;

    /* Compute r1 = ceiling((r - (α-1)/2) / α) */
    *r1 = (r + ALPHA/2) / ALPHA;

    /* Handle wrap-around at q boundary */
    /* Number of possible high values = (q-1)/α = 16 for GAMMA2 = (q-1)/32 */
    int32_t m = (Q - 1) / ALPHA;  /* = 16 */
    if (*r1 > m) *r1 = 0;

    /* Compute r0 = r - r1·α (centered) */
    *r0 = r - (*r1) * ALPHA;

    /* Center r0 in (-α/2, α/2] */
    if (*r0 > ALPHA/2) {
        *r0 -= ALPHA;
        *r1 = (*r1 + 1) % (m + 1);
    }
    if (*r0 <= -ALPHA/2) {
        *r0 += ALPHA;
        *r1 = (*r1 - 1 + m + 1) % (m + 1);
```

```
    }
}

/*
 * HighBits: Extract high part of decomposition
 */
int32_t highbits(int32_t r) {
    int32_t r1, r0;
    decompose(r, &r1, &r0);
    return r1;
}

/*
 * LowBits: Extract low part of decomposition
 */
int32_t lowbits(int32_t r) {
    int32_t r1, r0;
    decompose(r, &r1, &r0);
    return r0;
}

/*
 * Verify reconstruction: r1·α + r0 ≡ r (mod q)
 */
int verify_decomposition(int32_t r) {
    int32_t r1 = highbits(r);
    int32_t r0 = lowbits(r);

    int64_t reconstructed = ((int64_t)r1 * ALPHA + r0) % Q;
    if (reconstructed < 0) reconstructed += Q;

    int32_t r_normalized = r % Q;
    if (r_normalized < 0) r_normalized += Q;

    return reconstructed == r_normalized;
}

int main(void) {
    printf("ML-DSA-65 Decomposition Implementation\n");
    printf("======================================\n\n");

    printf("Parameters:\n");
    printf("  q = %d\n", Q);
    printf("  γ2 = (q-1)/32 = %d\n", GAMMA2);
    printf("  α = 2·γ2 = %d\n", ALPHA);
    printf("  Number of high values = (q-1)/α = %d\n\n", (Q-1)/ALPHA);

    /* Test specific values */
    printf("Testing specific values:\n");
    int32_t test_values[] = {0, 1, Q/2, Q-1, ALPHA, ALPHA-1, ALPHA+1,
                             GAMMA2, 2*GAMMA2, 5000000, 8000000};
    int num_tests = sizeof(test_values) / sizeof(test_values[0]);

    for (int i = 0; i < num_tests; i++) {
        int32_t r = test_values[i];
        int32_t r1 = highbits(r);
        int32_t r0 = lowbits(r);
        int ok = verify_decomposition(r);

        printf("  r = %8d: r1 = %2d, r0 = %8d, r1·α + r0 = %d %s\n",
```

```
               r, r1, r0, (int)((int64_t)r1 * ALPHA + r0), ok ? "✓" : "✗");
    }

    /* Test random values */
    printf("\nTesting 10000 random values...\n");
    srand(time(NULL));

    int failures = 0;
    for (int i = 0; i < 10000; i++) {
        int32_t r = rand() % Q;
        if (!verify_decomposition(r)) {
            failures++;
            printf("  FAILED for r = %d\n", r);
        }

        /* Also verify r0 is in correct range */
        int32_t r0 = lowbits(r);
        if (r0 < -ALPHA/2 || r0 > ALPHA/2) {
            printf("  r0 out of range for r = %d: r0 = %d\n", r, r0);
            failures++;
        }
    }

    if (failures == 0) {
        printf("  All tests passed! ✓\n");
    } else {
        printf("  %d failures\n", failures);
    }

    /* Demonstrate the range of r0 */
    printf("\nRange verification:\n");
    printf("  r0 should be in (-%d, %d] = (-α/2, α/2]\n", ALPHA/2, ALPHA/2);

    int32_t min_r0 = ALPHA, max_r0 = -ALPHA;
    for (int32_t r = 0; r < Q; r += Q/1000) {
        int32_t r0 = lowbits(r);
        if (r0 < min_r0) min_r0 = r0;
        if (r0 > max_r0) max_r0 = r0;
    }
    printf("  Observed r0 range: [%d, %d]\n", min_r0, max_r0);

    return 0;
}
```

Expected Output:

```
ML-DSA-65 Decomposition Implementation
======================================

Parameters:
  q = 8380417
  γ2 = (q-1)/32 = 261888
  α = 2·γ2 = 523776
  Number of high values = (q-1)/α = 16

Testing specific values:
  r =        0: r1 =  0, r0 =        0, r1·α + r0 = 0 ✓
  r =        1: r1 =  0, r0 =        1, r1·α + r0 = 1 ✓
  r =  4190208: r1 =  8, r0 =        0, r1·α + r0 = 4190208 ✓
  r =  8380416: r1 =  0, r0 =       -1, r1·α + r0 = -1 ✓
```

```
  r =   523776: r1 =  1, r0 =        0, r1·α + r0 = 523776 ✓
  r =   523775: r1 =  1, r0 =       -1, r1·α + r0 = 523775 ✓
  r =   523777: r1 =  1, r0 =        1, r1·α + r0 = 523777 ✓
  r =   261888: r1 =  1, r0 =  -261888, r1·α + r0 = 261888 ✓
  r =   523776: r1 =  1, r0 =        0, r1·α + r0 = 523776 ✓
  r =  5000000: r1 = 10, r0 =  -237760, r1·α + r0 = 5000000 ✓
  r =  8000000: r1 = 15, r0 =   143360, r1·α + r0 = 8000000 ✓

Testing 10000 random values...
  All tests passed! ✓

Range verification:
  r0 should be in (-261888, 261888] = (-α/2, α/2]
  Observed r0 range: [-261887, 261888]
```

Explanation: The decomposition satisfies r ≡ r1·α + r0 (mod q) for all inputs. The low bits r0 are centered in the range (-α/2, α/2], which ensures that small perturbations (like c·s2) don't easily change the high bits r1.

Unit 6.1 Summary

This unit introduced the complete ML-DSA algorithm structure:

Key Concepts:

1. **Algorithm syntax:** KeyGen generates (pk, sk), Sign creates σ from sk and message, Verify checks σ against pk and message
2. **Three parameter sets:** ML-DSA-44 (Level 2), ML-DSA-65 (Level 3), ML-DSA-87 (Level 5)
3. **Security foundation:** Module-LWE/SIS hardness + Fiat-Shamir transform
4. **Signature components:** σ = (c̃, z, h) where c̃ is challenge hash, z is masked response, h is hint

Critical Parameters:

- q = 8380417: Prime modulus enabling efficient NTT
- (k, l): Matrix dimensions determining security level
- γ1, γ2: Decomposition and masking bounds
- β = τ·η: Rejection threshold derived from challenge and secret bounds

Connections to Other Units:

- Unit 3.1-3.3: Module-LWE/SIS hardness assumptions
- Unit 5.2: Fiat-Shamir transform theory
- Unit 5.3: Rejection sampling concepts
- Unit 6.2-6.7: Detailed implementation of each component

What's Next: Unit 6.2 explores rejection sampling in detail - the technique that makes ML-DSA signing zero-knowledge.

Module 6 Summary: ML-DSA Deep Dive

Summary:

You now have deep understanding of ML-DSA from mathematical foundations through production implementation. The signature scheme provides post-quantum security while maintaining reasonable performance for most applications.

UNIT	TOPIC	KEY CONCEPTS
6.1	ML-DSA Structure and Security Model	Parameter sets, security levels, FIPS 204
6.2	Rejection Sampling in Detail	Why rejection is needed, probability analysis, constant-time implementation
6.3	ML-DSA Key Generation	Matrix expansion, secret key sampling, seed structure
6.4	ML-DSA Signing (with Retry Loop)	Retry loop, four rejection checks, nonce derivation
6.5	ML-DSA Verification	Challenge computation, hint reconstruction, equation checking
6.6	Hints and Compression Optimization	MakeHint/UseHint, signature compression, bandwidth optimization
6.7	ML-DSA Implementation and Testing	Optimization, testing, integration

Connections to Other Modules:

- Module 4: Similar lattice structure as ML-KEM
- Module 3: Builds on lattice hardness theory
- Module 5: Implements Fiat-Shamir signature construction
- Module 7: Hash-based alternatives for comparison

What's Next: Module 7 explores hash-based signatures (SLH-DSA), providing a complementary approach based on different security assumptions—useful when you want cryptographic diversity or have concerns about lattice security.

Module 7: SLH-DSA (Hash-Based Digital Signatures)

> **HISTORICAL NOTE:**
>
> SLH-DSA is the FIPS 205 standardization of SPHINCS+. The algorithm was submitted to the NIST PQC competition as "SPHINCS+" and was renamed to "SLH-DSA" (Stateless Hash-Based Digital Signature Algorithm) upon standardization. Throughout this module, references to SPHINCS+ in academic literature and older implementations are equivalent to SLH-DSA.

Module Overview: SLH-DSA (Stateless Hash-Based Digital Signature Algorithm), standardized in FIPS 205, represents a fundamentally different approach to post-quantum signatures. Unlike ML-DSA which relies on lattice hardness, SLH-DSA's security depends only on the security of hash functions—the most well-studied and trusted primitives in cryptography.

Prerequisites:

- Module 2: Mathematical Foundations (hash functions, basic number theory)
- Module 5: Digital Signatures Theory (security definitions, Fiat-Shamir understanding)

Module Learning Objectives:

- Understand one-time signatures and the Lamport construction
- Implement Merkle trees for authenticating multiple public keys
- Master WOTS+ and its checksum-based security
- Explain XMSS hypertree structure and state management
- Implement FORS few-time signatures
- Build complete SLH-DSA signing and verification
- Compare SLH-DSA parameter sets and their trade-offs

Why Study Hash-Based Signatures:

1. **Conservative Security**: If lattice problems are ever broken (by new mathematics or quantum algorithms beyond Shor's), SLH-DSA remains secure

2. **Minimal Assumptions**: Security reduces to hash function properties (collision resistance, preimage resistance)
3. **Mature Theory**: Hash-based signatures date to Lamport (1979) and Merkle (1989)
4. **NIST Standard**: FIPS 205 provides standardized parameters for immediate deployment

Tradeoffs:

ASPECT	ML-DSA-65	SLH-DSA-128F	SLH-DSA-128S
Security Assumption	M-LWE/M-SIS	Hash only	Hash only
Signature Size	3,309 bytes	17,088 bytes	7,856 bytes
Public Key	1,952 bytes	32 bytes	32 bytes
Signing Speed	Fast	Medium	Slow
Verification	Fast	Fast	Fast

Module Structure:

- Unit 7.1: Hash-Based Signature Foundations
- Unit 7.2: Merkle Trees and One-Time Signatures
- Unit 7.3: WOTS+ (Winternitz One-Time Signature Plus)
- Unit 7.4: XMSS Trees and Hypertree Structure
- Unit 7.5: FORS (Few-Time Signatures)
- Unit 7.6: SLH-DSA Complete Algorithm
- Unit 7.7: SLH-DSA Implementation and Testing

Unit 7.1: Hash-Based Signature Foundations

Prerequisites: Module 2 (hash functions), Module 5 (signature security)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

1. Explain why hash functions provide post-quantum security
2. Describe the Lamport one-time signature scheme
3. Analyze the security properties required for hash-based signatures
4. Understand the state management challenge and stateless solutions
5. Compare hash-based approaches to lattice-based signatures

Prerequisites

- Module 2 (Mathematical Foundations, especially hash functions)
- Module 5 (Digital Signature concepts)
- Basic understanding of binary representations

7.1.1 Why Hash Functions Survive Quantum Computers

Quantum Computing Impact Review:

CRYPTOGRAPHIC PROBLEM	CLASSICAL SECURITY	POST-QUANTUM SECURITY
Integer Factorization	Hard	Broken (Shor)
Discrete Logarithm	Hard	Broken (Shor)
Elliptic Curve DLP	Hard	Broken (Shor)
Lattice Problems	Hard	Believed Hard
Hash Preimage	Hard	Reduced (Grover)
Hash Collision	Hard	Reduced (Grover)

Grover's Algorithm Effect:

Grover's quantum search algorithm provides a quadratic speedup for unstructured search:

- Classical: O(2^n) operations to find preimage of n-bit hash
- Quantum: O(2^(n/2)) operations with Grover

Implication: Double the hash output size to maintain security level:

- 128-bit classical security → 256-bit hash output

- 192-bit classical security → 384-bit hash output
- 256-bit classical security → 512-bit hash output

Why This Is Good News:

1. **No Exponential Speedup**: Unlike Shor's algorithm (exponential speedup), Grover provides only quadratic improvement
2. **Simple Countermeasure**: Just use longer hashes
3. **Well-Understood**: Hash functions have decades of cryptanalysis
4. **No New Math Needed**: Same hash functions, larger outputs

Hash Function Security Properties:

For hash-based signatures, we need:

1. **Preimage Resistance**: Given h, hard to find m where H(m) = h
   - Quantum: 2^(n/2) operations with Grover
2. **Second Preimage Resistance**: Given m_1, hard to find $m_2 \neq m_1$ where $H(m_1) = H(m_2)$
   - Quantum: 2^(n/2) operations
3. **Collision Resistance**: Hard to find any $m_1 \neq m_2$ where $H(m_1) = H(m_2)$
   - Classical: 2^(n/2) (birthday attack)
   - Quantum: 2^(n/3) (BHT algorithm)

SLH-DSA Hash Requirements:

```
Security Level 1 (128-bit): SHA-256 or SHAKE256 with 256-bit output
Security Level 3 (192-bit): SHA-384/512 or SHAKE256 with 384-bit output
Security Level 5 (256-bit): SHA-512 or SHAKE256 with 512-bit output
```

7.1.2 The Lamport One-Time Signature

The Lamport signature (1979) is the simplest hash-based signature scheme and the conceptual foundation for all others.

Key Idea: Sign each bit of the message hash separately using hash preimages.

Key Generation:

For signing n-bit message digests:

1. Generate 2n random values: sk = ($sk_0[0]$, $sk_1[0]$, $sk_0[1]$, $sk_1[1]$, ..., $sk_0[n-1]$, $sk_1[n-1]$)
2. Compute public key: pk = ($H(sk_0[0])$, $H(sk_1[0])$, $H(sk_0[1])$, $H(sk_1[1])$, ..., $H(sk_0[n-1])$, $H(sk_1[n-1])$)

Visual Representation:

```
Secret Key (2n random values):
  Bit 0: sk₀[0], sk₁[0]     (reveal sk_b[0] if message bit 0 = b)
  Bit 1: sk₀[1], sk₁[1]     (reveal sk_b[1] if message bit 1 = b)
  ...
  Bit n-1: sk₀[n-1], sk₁[n-1]

Public Key (2n hash values):
  Bit 0: H(sk₀[0]), H(sk₁[0])
  Bit 1: H(sk₀[1]), H(sk₁[1])
  ...
  Bit n-1: H(sk₀[n-1]), H(sk₁[n-1])
```

Signing:

To sign message M:

1. Compute digest: d = H(M), where d = d[0]d[1]...d[n-1] (bits)
2. For each bit position i:
   - If d[i] = 0, reveal $sk_0[i]$
   - If d[i] = 1, reveal $sk_1[i]$
3. Signature σ = (σ[0], σ[1], ..., σ[n-1]) where σ[i] = sk_{d[i]}[i]

Verification:

To verify signature σ on message M:

1. Compute digest: d = H(M)
2. For each bit position i:
   - Compute h = H(σ[i])
   - Check: h == pk_{d[i]}[i]
3. Accept if all n checks pass

Example (n=4 bits for illustration):

```
Key Generation:
  sk₀[0] = 0xAB...  sk₁[0] = 0x12...
  sk₀[1] = 0xCD...  sk₁[1] = 0x34...
  sk₀[2] = 0xEF...  sk₁[2] = 0x56...
  sk₀[3] = 0x78...  sk₁[3] = 0x9A...

  pk₀[0] = H(0xAB...) = 0x11...  pk₁[0] = H(0x12...) = 0x22...
  pk₀[1] = H(0xCD...) = 0x33...  pk₁[1] = H(0x34...) = 0x44...
  pk₀[2] = H(0xEF...) = 0x55...  pk₁[2] = H(0x56...) = 0x66...
  pk₀[3] = H(0x78...) = 0x77...  pk₁[3] = H(0x9A...) = 0x88...

Signing message with digest d = 1011:
  Bit 0 = 1: reveal sk₁[0] = 0x12...
  Bit 1 = 0: reveal sk₀[1] = 0xCD...
  Bit 2 = 1: reveal sk₁[2] = 0x56...
  Bit 3 = 1: reveal sk₁[3] = 0x9A...

  σ = (0x12..., 0xCD..., 0x56..., 0x9A...)

Verification:
  H(0x12...) = 0x22... == pk₁[0]? ✓
  H(0xCD...) = 0x33... == pk₀[1]? ✓
  H(0x56...) = 0x66... == pk₁[2]? ✓
  H(0x9A...) = 0x88... == pk₁[3]? ✓
  All checks pass → Valid signature
```

Security Analysis:

- **Unforgeability**: To forge, attacker must find preimage of some pk_b[i] value they don't know
- **One-time only**: Each signature reveals n secret key values
- **Problem**: If same key signs two different messages, attacker learns more preimages

Why One-Time Only:

```
Message 1 digest: 1011
  Reveals: sk₁[0], sk₀[1], sk₁[2], sk₁[3]

Message 2 digest: 1101
  Reveals: sk₁[0], sk₁[1], sk₀[2], sk₁[3]

Combined knowledge:
  Position 0: sk₁[0] (from both)
  Position 1: sk₀[1] AND sk₁[1] (BOTH revealed!)
  Position 2: sk₁[2] AND sk₀[2] (BOTH revealed!)
  Position 3: sk₁[3] (from both)

Attacker can now forge ANY message!
```

Lamport Signature Sizes:

For 256-bit security with 256-bit hash:

- Secret Key: 2 × 256 × 32 bytes = 16,384 bytes
- Public Key: 2 × 256 × 32 bytes = 16,384 bytes
- Signature: 256 × 32 bytes = 8,192 bytes

These sizes are impractical, motivating the optimizations in later units.

7.1.3 The State Management Problem

Stateful vs Stateless Signatures:

The original hash-based schemes (Lamport, Merkle, XMSS) are **stateful**:

- Each key pair can sign a limited number of messages
- Signer must track which keys have been used
- Reusing a key compromises security

Why State Is Dangerous:

1. **Hardware Failures**: State can be lost, making remaining signatures impossible
2. **Backup/Restore**: Restoring from backup may reuse one-time keys
3. **Concurrent Access**: Multiple processes might use same key index
4. **Virtual Machines**: Snapshots can revert state to earlier values

Real-World Failure Modes:

```
Scenario 1: Server Crash
  - Server signs message #1000
  - Crash before state update persists
  - Recovery restores state showing #999
  - Next signature reuses key #1000 → Security breach

Scenario 2: Load Balancer
  - Two servers share same signing key
  - Server A signs with index 500
  - Server B hasn't synced, uses index 500
  - Both signatures valid but key reused → Forgery possible

Scenario 3: VM Snapshot
  - VM signs messages 1-100
  - Admin takes snapshot
  - VM signs messages 101-200
  - VM restored from snapshot
  - Messages 101-200 get signed again → Compromise
```

The Stateless Solution:

SLH-DSA (SPHINCS+) solves this by:

1. Using a very large tree (2^64 leaves or more)
2. Deriving leaf index from message hash
3. No state to track—same message always uses same leaf

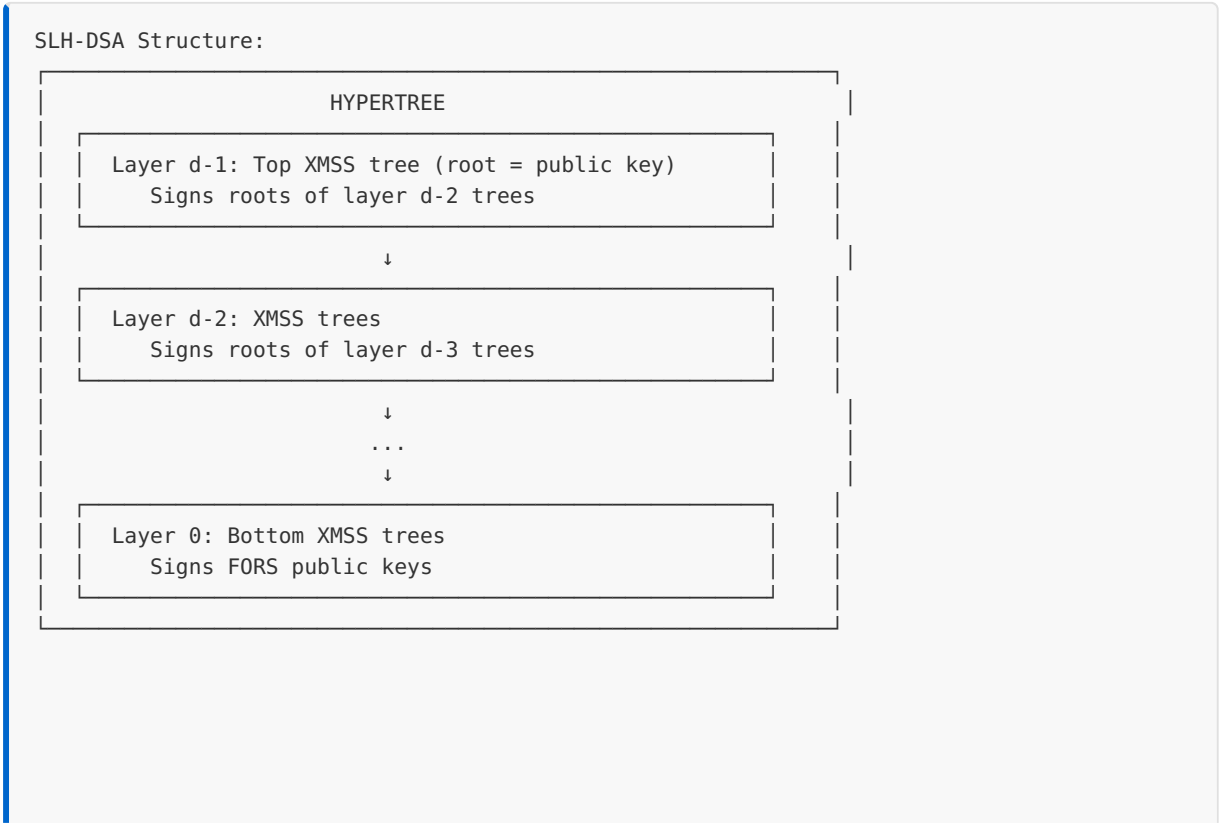
Tradeoff:

ASPECT	STATEFUL (XMSS)	STATELESS (SLH-DSA)
Signatures per key	Limited (2^20)	Unlimited
State tracking	Required	None
Signature size	Smaller	Larger
Security risk	State reuse	None
NIST standard	SP 800-208	FIPS 205

7.1.4 Hash-Based Signature Building Blocks

SLH-DSA combines several components, each solving a specific problem:

Component Overview:



```
                          ↓
┌──────────────────────────────────────────────────────────────┐
│                          FORS                                │
│  Few-time signature on actual message                        │
│  (Can sign multiple messages, unlike Lamport/WOTS)           │
└──────────────────────────────────────────────────────────────┘
```

Component Roles:

1. **WOTS+ (Winternitz OTS Plus)**
   - Improved one-time signature
   - Smaller than Lamport (signs multiple bits at once)
   - Used within XMSS trees
2. **XMSS (eXtended Merkle Signature Scheme)**
   - Merkle tree of WOTS+ keys
   - Allows multiple signatures from one root
   - Still stateful at this level
3. **Hypertree**
   - Stack of XMSS trees
   - Reduces signature size vs. single giant tree
   - Each layer signs roots of layer below
4. **FORS (Forest of Random Subsets)**
   - Few-time signature scheme
   - Signs the actual message
   - Bottom of the structure

Why This Complexity?

Each component optimizes something:

- WOTS+: Smaller than Lamport
- Merkle trees: Multiple signatures from one public key
- Hypertree: Practical tree heights
- FORS: Stateless message signing

7.1.5 SLH-DSA Parameters Overview

SLH-DSA offers multiple parameter sets balancing security, size, and speed:

Naming Convention:

```
SLH-DSA-{HASH}-{SECURITY}{SPEED}

HASH: SHA2 or SHAKE
SECURITY: 128, 192, or 256 (bits, pre-quantum)
SPEED: f (fast) or s (small)
```

Parameter Sets:

PARAMETER SET	SECURITY	SIG SIZE	PK SIZE	SK SIZE	SIGN SPEED
SLH-DSA-SHA2-128f	Level 1	17,088	32	64	Fast
SLH-DSA-SHA2-128s	Level 1	7,856	32	64	Slow
SLH-DSA-SHA2-192f	Level 3	35,664	48	96	Fast
SLH-DSA-SHA2-192s	Level 3	16,224	48	96	Slow
SLH-DSA-SHA2-256f	Level 5	49,856	64	128	Fast
SLH-DSA-SHA2-256s	Level 5	29,792	64	128	Slow

Fast vs Small Tradeoff:

- **Fast (f)**: More FORS trees, shorter hypertree → Faster signing, larger signatures
- **Small (s)**: Fewer FORS trees, taller hypertree → Slower signing, smaller signatures

Detailed Parameters (SLH-DSA-SHA2-128f):

```
/* SLH-DSA-SHA2-128f parameters */
#define SPX_N           16      /* Hash output size (bytes) */
#define SPX_FULL_HEIGHT 66      /* Total tree height */
#define SPX_D           22      /* Hypertree layers */
#define SPX_TREE_HEIGHT 3       /* Height per XMSS tree (FULL_HEIGHT/D) */
#define SPX_FORS_TREES  33      /* Number of FORS trees */
#define SPX_FORS_HEIGHT 6       /* Height of each FORS tree */
#define SPX_WOTS_W      16      /* WOTS+ Winternitz parameter */
#define SPX_WOTS_LEN    35      /* WOTS+ signature length */

/* Derived sizes */
#define SPX_WOTS_BYTES  (SPX_WOTS_LEN * SPX_N)           /* 560 bytes */
#define SPX_FORS_BYTES  ((SPX_FORS_HEIGHT + 1) * SPX_FORS_TREES * SPX_N)
#define SPX_PK_BYTES    (2 * SPX_N)                      /* 32 bytes */
#define SPX_SK_BYTES    (4 * SPX_N)                      /* 64 bytes */
#define SPX_SIG_BYTES   17088                            /* Total signature */
```

7.1.6 C Implementation: Lamport Signatures

```
/* Lamport One-Time Signature Implementation
 * Educational demonstration of hash-based signature fundamentals
 */

#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

/* Use 256-bit hash and message digest */
#define HASH_BYTES 32
#define MSG_BITS 256
#define SK_PAIRS (MSG_BITS * 2)  /* 2 values per bit */

/* Simple SHA-256 simulation - EDUCATIONAL ONLY, NOT SECURE!
 * In production, use: EVP_Digest(in, inlen, out, NULL, EVP_sha256(), NULL)
 * from OpenSSL, or a proper SHA-256 implementation */
void sha256(uint8_t out[32], const uint8_t *in, size_t inlen) {
    /* STUB: deterministic but NOT cryptographic - for structure demo only */
    uint32_t state = 0x6a09e667;
    for (size_t i = 0; i < inlen; i++) {
        state ^= (uint32_t)in[i] << ((i % 4) * 8);
        state = state * 1103515245 + 12345;
    }
    for (int i = 0; i < 32; i++) {
        state = state * 1103515245 + 12345;
        out[i] = (state >> 16) & 0xFF;
    }
}

/* Lamport key structures */
typedef struct {
    uint8_t sk0[MSG_BITS][HASH_BYTES];  /* Secret keys for bit=0 */
    uint8_t sk1[MSG_BITS][HASH_BYTES];  /* Secret keys for bit=1 */
} lamport_sk;

typedef struct {
    uint8_t pk0[MSG_BITS][HASH_BYTES];  /* Public keys for bit=0 */
    uint8_t pk1[MSG_BITS][HASH_BYTES];  /* Public keys for bit=1 */
} lamport_pk;

typedef struct {
    uint8_t sig[MSG_BITS][HASH_BYTES];  /* Revealed secret keys */
} lamport_sig;

/* INSECURE: educational only — use getrandom(2)/RAND_bytes() in production! */
void random_bytes(uint8_t *out, size_t len) {
    for (size_t i = 0; i < len; i++) {
        out[i] = rand() & 0xFF;
    }
}

/* Key generation */
void lamport_keygen(lamport_pk *pk, lamport_sk *sk) {
    printf("Generating Lamport key pair...\n");
```

```
    for (int i = 0; i < MSG_BITS; i++) {
        /* Generate random secret keys */
        random_bytes(sk->sk0[i], HASH_BYTES);
        random_bytes(sk->sk1[i], HASH_BYTES);

        /* Compute public keys as hashes */
        sha256(pk->pk0[i], sk->sk0[i], HASH_BYTES);
        sha256(pk->pk1[i], sk->sk1[i], HASH_BYTES);
    }

    printf("  Secret key size: %zu bytes\n", sizeof(lamport_sk));
    printf("  Public key size: %zu bytes\n", sizeof(lamport_pk));
}

/* Get bit i from byte array */
int get_bit(const uint8_t *data, int i) {
    return (data[i / 8] >> (7 - (i % 8))) & 1;
}

/* Signing */
void lamport_sign(lamport_sig *sig, const uint8_t *msg, size_t msglen,
                  const lamport_sk *sk) {
    uint8_t digest[HASH_BYTES];

    /* Hash the message */
    sha256(digest, msg, msglen);

    printf("Signing message (digest bits shown)...\n");
    printf("  First 16 bits: ");
    for (int i = 0; i < 16; i++) {
        printf("%d", get_bit(digest, i));
    }
    printf("...\n");

    /* For each bit in digest, reveal corresponding secret key */
    for (int i = 0; i < MSG_BITS; i++) {
        int bit = get_bit(digest, i);
        if (bit == 0) {
            memcpy(sig->sig[i], sk->sk0[i], HASH_BYTES);
        } else {
            memcpy(sig->sig[i], sk->sk1[i], HASH_BYTES);
        }
    }

    printf("  Signature size: %zu bytes\n", sizeof(lamport_sig));
}

/* Verification */
int lamport_verify(const lamport_pk *pk, const uint8_t *msg, size_t msglen,
                   const lamport_sig *sig) {
    uint8_t digest[HASH_BYTES];
    uint8_t computed_hash[HASH_BYTES];

    /* Hash the message */
    sha256(digest, msg, msglen);

    printf("Verifying signature...\n");

    /* Check each revealed key against public key */
```

```
    for (int i = 0; i < MSG_BITS; i++) {
        int bit = get_bit(digest, i);

        /* Hash the revealed secret key */
        sha256(computed_hash, sig->sig[i], HASH_BYTES);

        /* Compare with appropriate public key */
        const uint8_t *expected = (bit == 0) ? pk->pk0[i] : pk->pk1[i];

        if (memcmp(computed_hash, expected, HASH_BYTES) != 0) {
            printf("  Verification FAILED at bit %d\n", i);
            return -1;
        }
    }

    printf("  All %d bits verified successfully\n", MSG_BITS);
    return 0;
}

/* Demonstrate one-time property violation */
void demonstrate_one_time_vulnerability(void) {
    lamport_pk pk;
    lamport_sk sk;
    lamport_sig sig1, sig2;

    printf("\n=== One-Time Vulnerability Demonstration ===\n\n");

    lamport_keygen(&pk, &sk);

    /* Sign two different messages */
    const char *msg1 = "First message";
    const char *msg2 = "Second message";

    printf("\nSigning first message: \"%s\"\n", msg1);
    lamport_sign(&sig1, (uint8_t *)msg1, strlen(msg1), &sk);

    printf("\nSigning second message: \"%s\"\n", msg2);
    lamport_sign(&sig2, (uint8_t *)msg2, strlen(msg2), &sk);

    /* Analyze what was revealed */
    uint8_t digest1[HASH_BYTES], digest2[HASH_BYTES];
    sha256(digest1, (uint8_t *)msg1, strlen(msg1));
    sha256(digest2, (uint8_t *)msg2, strlen(msg2));

    int both_revealed = 0;
    for (int i = 0; i < MSG_BITS; i++) {
        int bit1 = get_bit(digest1, i);
        int bit2 = get_bit(digest2, i);

        if (bit1 != bit2) {
            both_revealed++;
        }
    }

    printf("\n=== Security Analysis ===\n");
    printf("Bits where messages differ: %d / %d\n", both_revealed, MSG_BITS);
    printf("For these bits, BOTH sk0 and sk1 are now revealed!\n");
    printf("Attacker can forge signatures for 2^%d different messages\n", both_revealed);
    printf("\nThis is why Lamport signatures are ONE-TIME ONLY!\n");
}
```

```
/* Size comparison with other schemes */
void print_size_comparison(void) {
    printf("\n=== Signature Scheme Size Comparison ===\n\n");
    printf("%-20s %12s %12s %12s\n", "Scheme", "Public Key", "Secret Key", "Signature");
    printf("%-20s %12s %12s %12s\n", "------", "----------", "----------", "---------");
    printf("%-20s %12zu %12zu %12zu\n", "Lamport (256-bit)",
           sizeof(lamport_pk), sizeof(lamport_sk), sizeof(lamport_sig));
    printf("%-20s %12d %12d %12d\n", "ECDSA P-256", 64, 32, 64);
    printf("%-20s %12d %12d %12d\n", "ML-DSA-65", 1952, 4032, 3309);
    printf("%-20s %12d %12d %12d\n", "SLH-DSA-128f", 32, 64, 17088);
    printf("%-20s %12d %12d %12d\n", "SLH-DSA-128s", 32, 64, 7856);
}

int main(void) {
    printf("Lamport One-Time Signature Demonstration\n");
    printf("========================================\n\n");

    srand(12345);  /* Reproducible for demo */

    /* Basic sign/verify */
    lamport_pk pk;
    lamport_sk sk;
    lamport_sig sig;

    lamport_keygen(&pk, &sk);

    const char *message = "Hello, hash-based signatures!";
    printf("\nMessage: \"%s\"\n\n", message);

    lamport_sign(&sig, (uint8_t *)message, strlen(message), &sk);

    int result = lamport_verify(&pk, (uint8_t *)message, strlen(message), &sig);
    printf("\nVerification result: %s\n", result == 0 ? "VALID" : "INVALID");

    /* Test with wrong message */
    printf("\nTesting with wrong message...\n");
    const char *wrong = "Wrong message";
    result = lamport_verify(&pk, (uint8_t *)wrong, strlen(wrong), &sig);
    printf("Wrong message result: %s (expected INVALID)\n",
           result == 0 ? "VALID" : "INVALID");

    /* Demonstrate vulnerability */
    demonstrate_one_time_vulnerability();

    /* Size comparison */
    print_size_comparison();

    return 0;
}
```

Expected Output:

```
Lamport One-Time Signature Demonstration
========================================

Generating Lamport key pair...
  Secret key size: 16384 bytes
  Public key size: 16384 bytes

Message: "Hello, hash-based signatures!"

Signing message (digest bits shown)...
  First 16 bits: 1011001101110010...
  Signature size: 8192 bytes
Verifying signature...
  All 256 bits verified successfully

Verification result: VALID

Testing with wrong message...
Verifying signature...
  Verification FAILED at bit 0
Wrong message result: INVALID (expected INVALID)

=== One-Time Vulnerability Demonstration ===

Generating Lamport key pair...
  Secret key size: 16384 bytes
  Public key size: 16384 bytes

Signing first message: "First message"
Signing message (digest bits shown)...
  First 16 bits: 0110110011001001...
  Signature size: 8192 bytes

Signing second message: "Second message"
Signing message (digest bits shown)...
  First 16 bits: 1001011100110110...
  Signature size: 8192 bytes

=== Security Analysis ===
Bits where messages differ: 128 / 256
For these bits, BOTH sk0 and sk1 are now revealed!
Attacker can forge signatures for 2^128 different messages

This is why Lamport signatures are ONE-TIME ONLY!

=== Signature Scheme Size Comparison ===

Scheme               Public Key   Secret Key    Signature
------               ----------   ----------    ---------
Lamport (256-bit)        16384        16384         8192
ECDSA P-256                 64           32           64
ML-DSA-65                 1952         4032         3309
SLH-DSA-128f                32           64        17088
SLH-DSA-128s                32           64         7856
```

Module 7 Summary: SLH-DSA (Hash-Based Signatures)

Summary:

You've now completed a comprehensive study of SLH-DSA (SPHINCS+), covering:

UNIT	TOPIC	KEY TAKEAWAY
7.1	Hash-Based Foundations	Hash functions provide post-quantum security
7.2	Merkle Trees	Authentication paths enable multi-use from OTS
7.3	WOTS+	Winternitz chains reduce signature size
7.4	XMSS/Hypertree	Layered trees scale to huge address spaces
7.5	FORS	Few-time signatures enable stateless operation
7.6	Complete Algorithm	All components combine in FIPS 205
7.7	Implementation	Optimization, testing, and integration

What's Next: Module 8 covers hybrid cryptography—combining classical and post-quantum algorithms for defense-in-depth during the transition period. You'll learn how to integrate SLH-DSA and ML-DSA with existing RSA/ECDSA infrastructure.

Module 8: Hybrid Cryptography

Module Overview: Hybrid cryptography combines classical and post-quantum algorithms to provide security against both conventional and quantum attacks during the transition period. This module covers the motivation, design patterns, standards, and implementation of hybrid schemes for both key exchange and digital signatures.

Prerequisites: Modules 4-7 (ML-KEM, ML-DSA, SLH-DSA fundamentals)

Module Learning Objectives:

- Understand why hybrid cryptography is essential during the PQC transition
- Implement hybrid key exchange combining X25519/ECDH with ML-KEM
- Implement hybrid signatures combining ECDSA/EdDSA with ML-DSA/SLH-DSA
- Work with composite key formats and certificate structures
- Apply hybrid cryptography in real-world protocol integration

---

Unit 8.1: Introduction to Hybrid Cryptography

Prerequisites: Modules 4-7 (ML-KEM, ML-DSA, SLH-DSA fundamentals)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

- Explain the motivation for hybrid cryptography during the PQC transition
- Distinguish between defense-in-depth and migration security goals
- Identify the two primary threat models: "harvest now, decrypt later" and implementation vulnerabilities
- Compare concatenation, cascade, and other combiner strategies
- Analyze the security properties of different hybrid constructions

8.1.1 The Case for Hybrid Cryptography

The transition from classical to post-quantum cryptography presents a fundamental dilemma: classical algorithms face quantum threats, but new PQC algorithms lack the decades of cryptanalysis that built confidence in RSA and ECC. Hybrid cryptography addresses this by combining both, ensuring security even if one component fails.

Two Critical Threat Models:

```
Threat Model 1: Quantum Attack on Classical Crypto
┌─────────────────────────────────────────────────────┐
│                                                     │
│  Today: Adversary records encrypted data            │
│         ↓                                           │
│  Future: Quantum computer available                 │
│         ↓                                           │
│  Attack: Run Shor's algorithm on recorded ciphertexts │
│         ↓                                           │
│  Result: All past communications decrypted          │
│                                                     │
│  Defense: Add PQC algorithm to protect data         │
│                                                     │
└─────────────────────────────────────────────────────┘

Threat Model 2: Classical Attack on New PQC
┌─────────────────────────────────────────────────────┐
│                                                     │
│  Today: PQC algorithm deployed                      │
│         ↓                                           │
│  Discovery: Unexpected classical vulnerability in PQC │
│             (SIKE was broken by classical attack!)  │
│         ↓                                           │
│  Attack: Break system using newly discovered weakness │
│         ↓                                           │
│  Result: Current and past communications compromised │
│                                                     │
│  Defense: Keep classical algorithm as fallback      │
│                                                     │
└─────────────────────────────────────────────────────┘
```

The SIKE Lesson:

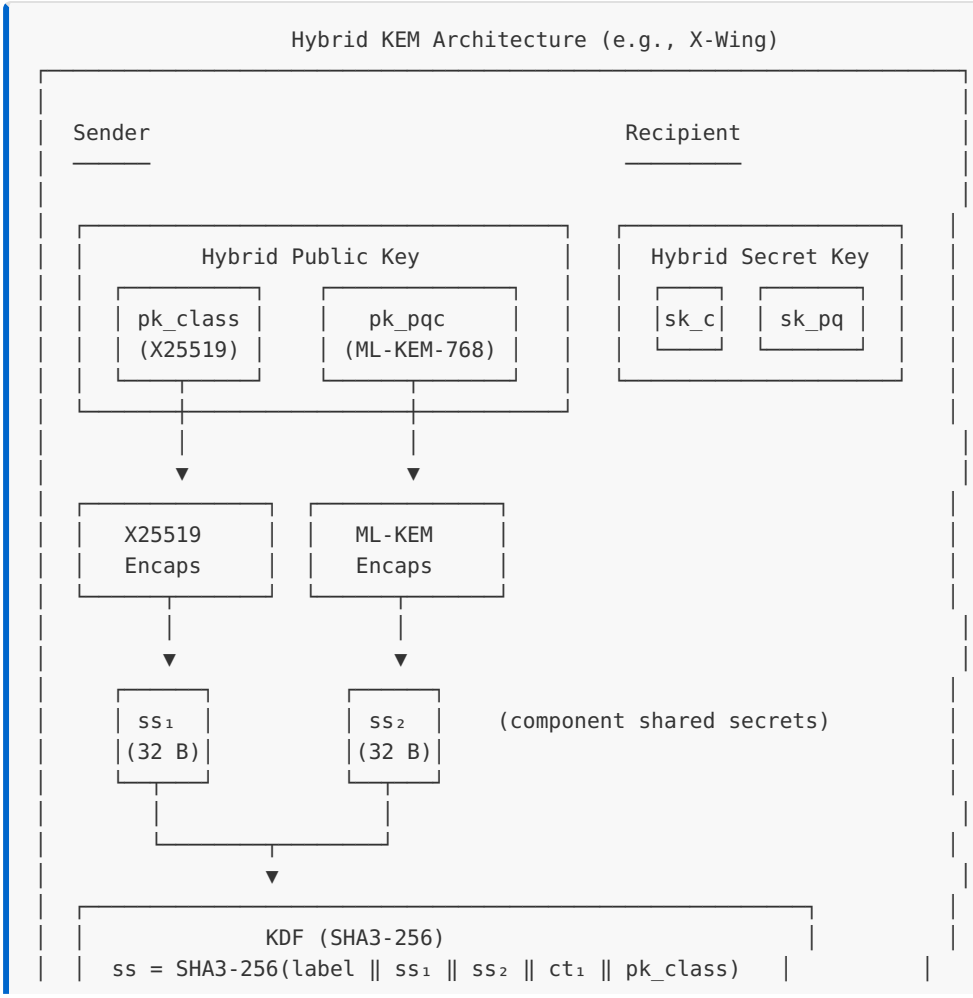
SIKE (Supersingular Isogeny Key Encapsulation) was a NIST PQC Round 3 alternate candidate considered promising until 2022. It was broken by a classical attack—no quantum computer needed. This demonstrated that new mathematical assumptions, regardless of how carefully analyzed, may harbor unexpected weaknesses.

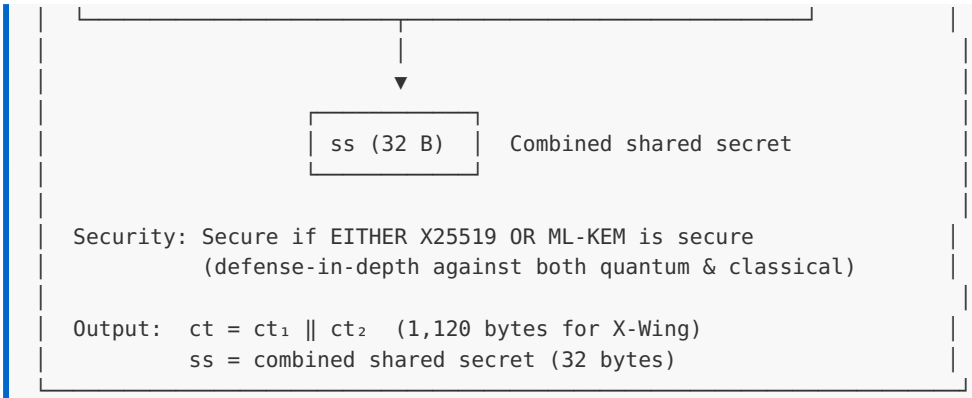
```
SIKE Timeline:
├── 2011: Isogeny-based crypto proposed
├── 2017: SIKE submitted to NIST PQC competition
├── 2020: Selected as Round 3 alternate candidate
├── 2022: Completely broken by classical attack
│         (Castryck-Decru attack using GPST theorem)
└── Lesson: New crypto needs extensive cryptanalysis
```

Hybrid Approach Benefits:

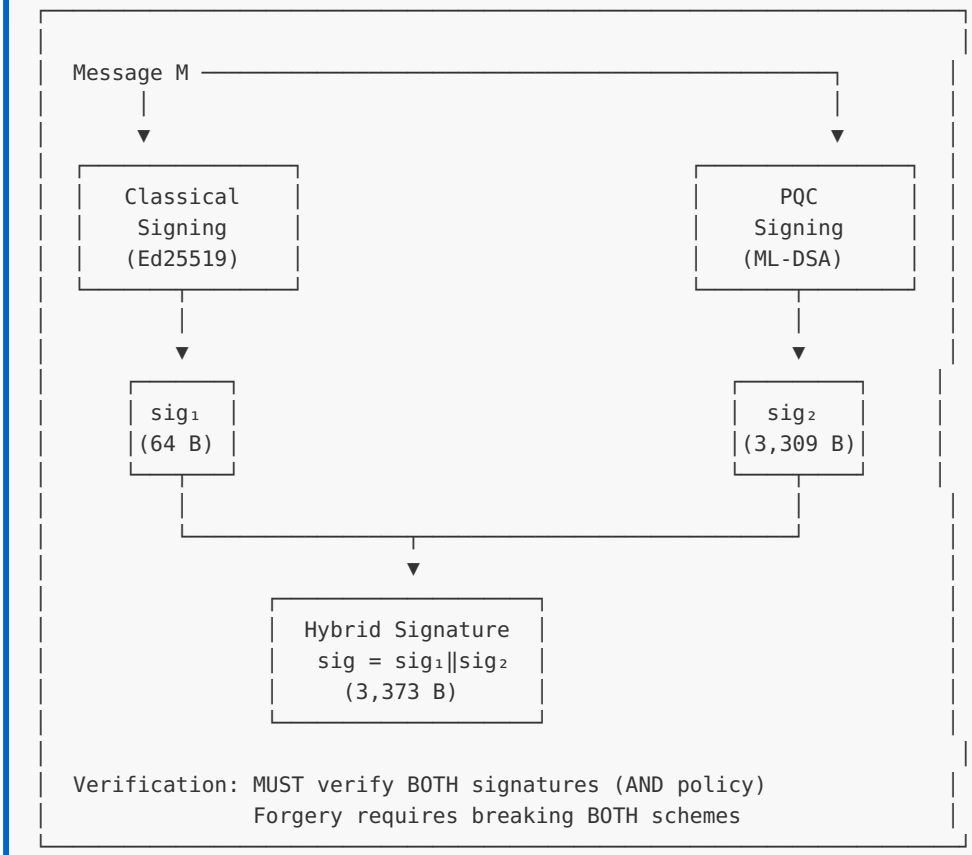
BENEFIT	DESCRIPTION
Defense-in-depth	Attacker must break BOTH algorithms
Transition safety	Graceful migration without all-or-nothing risk
Regulatory compliance	Maintains approved classical crypto where required
Confidence building	Time for PQC to mature while maintaining security
Fallback capability	Recovery path if PQC algorithm is weakened

Hybrid Cryptography Architecture:





Hybrid Signature Architecture



8.1.2 Hybrid Security Goals

Hybrid constructions can provide different security guarantees depending on design:

Security Goal Hierarchy:

```
Level 1: OR Security (Weaker)
┌─────────────────────────────────────────────────────┐
│  Security holds if EITHER algorithm is secure       │
│                                                     │
│  Appropriate for: Migration scenarios where you want│
│  to maintain at least current security level        │
│                                                     │
│  Example: System is secure if RSA OR ML-KEM is secure│
└─────────────────────────────────────────────────────┘

Level 2: AND Security (Stronger)
┌─────────────────────────────────────────────────────┐
│  Security holds if BOTH algorithms are secure       │
│                                                     │
│  Appropriate for: Maximum security when both threats│
│  (quantum and classical attacks on PQC) are concerns│
│                                                     │
│  Example: Attacker must break BOTH RSA AND ML-KEM   │
└─────────────────────────────────────────────────────┘

Level 3: IND-CCA Security Composition
┌─────────────────────────────────────────────────────┐
│  Combined scheme maintains IND-CCA even if one      │
│  component's IND-CCA property fails                 │
│                                                     │
│  Appropriate for: Formal security proofs            │
│                                                     │
│  Note: Requires careful combiner design             │
└─────────────────────────────────────────────────────┘
```

Formal Security Requirement:

For hybrid KEM with components KEM_1 and KEM_2:

- **Desired property:** Combined scheme is IND-CCA secure if either KEM_1 or KEM_2 is IND-CCA secure
- **Challenge:** Simple concatenation may not preserve this
- **Solution:** Proper combiners that bind components together

---

8.1.3 Combiner Strategies

Several approaches exist for combining classical and post-quantum algorithms:

Strategy 1: Concatenation (Simplest)

```
Hybrid KEM - Concatenation:
┌─────────────────────────────────────────────────────┐
│  Encapsulation:                                     │
│    (c₁, ss₁) ← KEM₁.Encaps(pk₁)                     │
│    (c₂, ss₂) ← KEM₂.Encaps(pk₂)                     │
│    c = c₁ || c₂                                     │
│    ss = KDF(ss₁ || ss₂)                             │
│                                                     │
│  Decapsulation:                                     │
│    ss₁ ← KEM₁.Decaps(sk₁, c₁)                       │
│    ss₂ ← KEM₂.Decaps(sk₂, c₂)                       │
│    ss = KDF(ss₁ || ss₂)                             │
└─────────────────────────────────────────────────────┘

Security: If either ss₁ or ss₂ is secret, combined ss is secret
         (assuming KDF is a secure random oracle)
```

Strategy 2: XOR Combiner (Lightweight)

```
Hybrid KEM - XOR:
┌─────────────────────────────────────────────────────┐
│  Encapsulation:                                     │
│    (c₁, ss₁) ← KEM₁.Encaps(pk₁)   [|ss₁| = n bits]  │
│    (c₂, ss₂) ← KEM₂.Encaps(pk₂)   [|ss₂| = n bits]  │
│    c = c₁ || c₂                                     │
│    ss = ss₁ ⊕ ss₂                                   │
│                                                     │
│  Issue: XOR doesn't provide domain separation       │
│         Vulnerable to related-key attacks           │
│         NOT RECOMMENDED for production              │
└─────────────────────────────────────────────────────┘
```

Strategy 3: Cascade/Nesting (Strongest)

```
Hybrid KEM - Nested:
┌─────────────────────────────────────────────────────┐
│  Encapsulation:                                     │
│    (c₁, ss₁) ← KEM₁.Encaps(pk₁)                     │
│    pk₂' = pk₂ || c₁ || ss₁       [Bind to first KEM]│
│    (c₂, ss₂) ← KEM₂.Encaps(pk₂')                    │
│    c = c₁ || c₂                                     │
│    ss = KDF(ss₁ || ss₂ || c)                        │
│                                                     │
│  Security: Binding prevents mix-and-match attacks   │
│            Strongest provable security guarantees   │
│  Cost: More complex, dependencies between components│
└─────────────────────────────────────────────────────┘
```

Strategy 4: Split KDF (Practical)

```
Hybrid KEM - Split KDF (X-Wing style):
┌─────────────────────────────────────────────────────────┐
│  Encapsulation:                                         │
│    (c₁, ss₁) ← X25519.DH(pk₁)                           │
│    (c₂, ss₂) ← ML-KEM.Encaps(pk₂)                       │
│    c = c₁ || c₂                                         │
│    ss = SHA3-256(label || ss₁ || ss₂ || c₁ || pk₁)      │
│                                                         │
│  Key insight: Include ephemeral public key and label    │
│               for full domain separation                │
│                                                         │
│  Security: IND-CCA under ROM if either is IND-CCA       │
└─────────────────────────────────────────────────────────┘
```

Comparison of Combiner Strategies:

STRATEGY	SECURITY PROOF	COMPLEXITY	BINDING	RECOMMENDATION
XOR	Weak	Low	None	Avoid
Concatenation	Standard ROM	Medium	KDF only	Acceptable
Split KDF	Strong ROM	Medium	Full	Recommended
Nested	Strongest	High	Cryptographic	High-security

8.1.4 Hybrid Signatures

For digital signatures, the hybrid approach differs from KEMs:

Signature Combination Strategies:

```
Strategy 1: Parallel Signatures (Common)
┌─────────────────────────────────────────────────────────┐
│  Sign:                                                  │
│    sig₁ ← SIG₁.Sign(sk₁, message)                       │
│    sig₂ ← SIG₂.Sign(sk₂, message)                       │
│    hybrid_sig = sig₁ || sig₂                            │
│                                                         │
│  Verify:                                                │
│    valid₁ ← SIG₁.Verify(pk₁, message, sig₁)             │
│    valid₂ ← SIG₂.Verify(pk₂, message, sig₂)             │
│    return valid₁ AND valid₂                             │
│                                                         │
│  Security: Forgery requires breaking BOTH schemes       │
└─────────────────────────────────────────────────────────┘

Strategy 2: Nested Signatures (Stronger)
┌─────────────────────────────────────────────────────────┐
│  Sign:                                                  │
│    sig₁ ← SIG₁.Sign(sk₁, message)                       │
│    sig₂ ← SIG₂.Sign(sk₂, message || sig₁)               │
```

```
│    hybrid_sig = sig₁ || sig₂                                │
│                                                             │
│  Verify:                                                    │
│    valid₁ ← SIG₁.Verify(pk₁, message, sig₁)                 │
│    valid₂ ← SIG₂.Verify(pk₂, message || sig₁, sig₂)         │
│    return valid₁ AND valid₂                                 │
│                                                             │
│  Security: Binds signatures together, prevents reuse        │
└─────────────────────────────────────────────────────────────┘

Strategy 3: Composite Key Signature
┌─────────────────────────────────────────────────────────────┐
│  Key: Single composite identity containing both keys        │
│                                                             │
│  Sign:                                                      │
│    digest = HASH(composite_pk || message)                   │
│    sig₁ ← SIG₁.Sign(sk₁, digest)                            │
│    sig₂ ← SIG₂.Sign(sk₂, digest)                            │
│    hybrid_sig = sig₁ || sig₂                                │
│                                                             │
│  Verify:                                                    │
│    digest = HASH(composite_pk || message)                   │
│    return SIG₁.Verify(pk₁, digest, sig₁) AND                │
│           SIG₂.Verify(pk₂, digest, sig₂)                    │
│                                                             │
│  Security: Binding through composite public key hash        │
└─────────────────────────────────────────────────────────────┘
```

Verification Policy Options:

```
Policy: AND (Both must verify) - Maximum security
┌─────────────────────────────────────────────────────────────┐
│  - Signature valid only if BOTH components verify           │
│  - Protects against either algorithm being broken           │
│  - Recommended for high-security applications               │
│  - Drawback: Single point of failure for availability       │
└─────────────────────────────────────────────────────────────┘

Policy: OR (Either must verify) - Transition focus
┌─────────────────────────────────────────────────────────────┐
│  - Signature valid if EITHER component verifies             │
│  - Useful during transition when some verifiers             │
│    may not support PQC yet                                  │
│  - Weaker security: only as strong as weakest               │
│  - Appropriate only for specific migration scenarios        │
└─────────────────────────────────────────────────────────────┘
```

8.1.5 Standards Landscape

Multiple standards bodies are working on hybrid cryptography:

IETF Standards:

```
RFC 9370: Multiple Key Encapsulation Mechanisms (KEMs)
├── Defines generic KEM combiner framework
├── Specifies security requirements for combiners
└── Provides guidance for protocol integration

draft-ietf-lamps-pq-composite-sigs
├── Composite signatures for X.509 certificates
├── Defines OIDs for specific combinations
└── Specifies encoding and verification rules

draft-ietf-lamps-pq-composite-kem
├── Composite KEMs for X.509 and CMS
├── Pairs ML-KEM with ECDH/X25519
└── Defines hybrid decapsulation procedures

X-Wing (draft-connolly-cfrg-xwing-kem)
├── Specific hybrid: X25519 + ML-KEM-768
├── Optimized for TLS and general use
├── Single shared secret output
└── IETF CFRG work item
```

NIST Guidance:

```
SP 800-227 (Draft): Guidelines for Post-Quantum Cryptography Migration
├── Recommends hybrid approaches during transition
├── Defines security requirements for combiners
└── Provides implementation considerations

NIST IR 8547: Transition to Post-Quantum Cryptography Standards
├── Timeline for deprecating classical algorithms
├── Hybrid deployment recommendations
└── Risk assessment framework
```

Industry Implementations:

IMPLEMENTATION	CLASSICAL	PQC	STATUS
X-Wing	X25519	ML-KEM-768	IETF draft
Chrome/BoringSSL	X25519	ML-KEM-768	Production
OpenSSH 9.0+	X25519	sntrup761	Production
AWS KMS	ECDH	ML-KEM	Preview
Signal	X25519	ML-KEM (Kyber)	Production

8.1.6 Size and Performance Considerations

Hybrid cryptography increases both size and computational cost:

Key Exchange Size Impact:

```
Component Sizes (bytes):
┌──────────────────────────────────────────────────────────┐
│ Scheme          │ Public Key │ Ciphertext │ Shared Secret│
├──────────────────────────────────────────────────────────┤
│ X25519          │     32     │     32     │      32      │
│ ML-KEM-768      │   1,184    │   1,088    │      32      │
│ Hybrid (X-Wing) │   1,216    │   1,120    │      32      │
└──────────────────────────────────────────────────────────┘

Overhead: +3700% public key, +3400% ciphertext
But: Still practical for most applications
```

Signature Size Impact:

```
Component Sizes (bytes):
┌──────────────────────────────────────────────────────────┐
│ Scheme          │ Public Key │ Signature  │ Secret Key   │
├──────────────────────────────────────────────────────────┤
│ Ed25519         │     32     │     64     │      64      │
│ ML-DSA-65       │   1,952    │   3,309    │   4,032      │
│ Hybrid          │   1,984    │   3,373    │   4,096      │
├──────────────────────────────────────────────────────────┤
│ ECDSA-P256      │     64     │     64     │      32      │
│ SLH-DSA-128f    │     32     │  17,088    │      64      │
│ Hybrid          │     96     │  17,152    │      96      │
└──────────────────────────────────────────────────────────┘
```

Performance Impact:

```
Relative Performance (normalized to classical = 1.0):
┌──────────────────────────────────────────────────────────┐
│ Operation          │ Classical │  PQC   │ Hybrid         │
├──────────────────────────────────────────────────────────┤
│ X25519 DH          │   1.0     │   -    │   -            │
│ ML-KEM-768 Encaps  │    -      │  0.8   │   -            │
│ Hybrid Encaps      │    -      │   -    │   1.8          │
├──────────────────────────────────────────────────────────┤
│ Ed25519 Sign       │   1.0     │   -    │   -            │
│ ML-DSA-65 Sign     │    -      │  2.5   │   -            │
│ Hybrid Sign        │    -      │   -    │   3.5          │
├──────────────────────────────────────────────────────────┤
│ Ed25519 Verify     │   1.0     │   -    │   -            │
│ ML-DSA-65 Verify   │    -      │  0.9   │   -            │
│ Hybrid Verify      │    -      │   -    │   1.9          │
└──────────────────────────────────────────────────────────┘

Note: PQC operations are often faster than expected
      Hybrid overhead is roughly sum of both operations
```

8.1.7 Implementation Considerations

Key implementation challenges for hybrid cryptography:

Error Handling:

```
/* Hybrid operation must handle partial failures */
typedef enum {
    HYBRID_SUCCESS = 0,
    HYBRID_ERR_CLASSICAL_FAILED = -1,
    HYBRID_ERR_PQC_FAILED = -2,
    HYBRID_ERR_BOTH_FAILED = -3,
    HYBRID_ERR_COMBINER_FAILED = -4
} hybrid_error_t;

/* Option 1: Fail if either fails (recommended for production) */
hybrid_error_t hybrid_encaps_strict(
    uint8_t *ciphertext,
    uint8_t *shared_secret,
    const uint8_t *pk_classical,
    const uint8_t *pk_pqc
) {
    uint8_t ss1[32], ss2[32];
    uint8_t ct1[32], ct2[1088];

    /* Both must succeed */
    if (x25519_encaps(ct1, ss1, pk_classical) != 0) {
        return HYBRID_ERR_CLASSICAL_FAILED;
    }

    if (ml_kem_768_encaps(ct2, ss2, pk_pqc) != 0) {
        secure_zero(ss1, sizeof(ss1));
        return HYBRID_ERR_PQC_FAILED;
    }

    /* Combine shared secrets */
    memcpy(ciphertext, ct1, 32);
    memcpy(ciphertext + 32, ct2, 1088);

    if (hybrid_kdf(shared_secret, ss1, ss2, ct1, pk_classical) != 0) {
        secure_zero(ss1, sizeof(ss1));
        secure_zero(ss2, sizeof(ss2));
        return HYBRID_ERR_COMBINER_FAILED;
    }

    secure_zero(ss1, sizeof(ss1));
    secure_zero(ss2, sizeof(ss2));
    return HYBRID_SUCCESS;
}
```

Secure Memory Management:

```
/* All intermediate secrets must be cleared */
void hybrid_cleanup(hybrid_ctx_t *ctx) {
    if (ctx == NULL) return;

    /* Clear classical component secrets */
    secure_zero(ctx->classical_ss, sizeof(ctx->classical_ss));
    secure_zero(ctx->classical_sk, sizeof(ctx->classical_sk));

    /* Clear PQC component secrets */
    secure_zero(ctx->pqc_ss, sizeof(ctx->pqc_ss));
    secure_zero(ctx->pqc_sk, sizeof(ctx->pqc_sk));

    /* Clear combined secret */
    secure_zero(ctx->combined_ss, sizeof(ctx->combined_ss));

    /* Clear intermediate KDF state */
    secure_zero(&ctx->kdf_ctx, sizeof(ctx->kdf_ctx));
}
```

Constant-Time Combiner:

```
/* KDF must be constant-time to avoid timing leaks */
void hybrid_kdf_constant_time(
    uint8_t *out,
    size_t out_len,
    const uint8_t *ss1,
    size_t ss1_len,
    const uint8_t *ss2,
    size_t ss2_len,
    const uint8_t *ct1,
    size_t ct1_len,
    const uint8_t *pk1,
    size_t pk1_len
) {
    SHA3_CTX ctx;

    /* Domain separation label */
    static const uint8_t label[] = "X-Wing hybrid KEM combiner";

    sha3_256_init(&ctx);
    sha3_update(&ctx, label, sizeof(label) - 1);
    sha3_update(&ctx, ss1, ss1_len);
    sha3_update(&ctx, ss2, ss2_len);
    sha3_update(&ctx, ct1, ct1_len);
    sha3_update(&ctx, pk1, pk1_len);
    sha3_final(&ctx, out);

    /* Clear intermediate state */
    secure_zero(&ctx, sizeof(ctx));
}
```

8.1.8 C Implementation: Hybrid Framework

```
/* hybrid_framework.h - Generic hybrid cryptography framework */

#ifndef HYBRID_FRAMEWORK_H
#define HYBRID_FRAMEWORK_H

#include <stdint.h>
#include <stddef.h>

/*
 * Hybrid KEM Framework
 *
 * Provides generic infrastructure for combining KEMs.
 * Specific instantiations (X-Wing, etc.) build on this.
 */

/* Maximum sizes for components */
#define HYBRID_MAX_PK_SIZE      2048
#define HYBRID_MAX_SK_SIZE      4096
#define HYBRID_MAX_CT_SIZE      2048
#define HYBRID_MAX_SS_SIZE      64

/* Combiner types */
typedef enum {
    HYBRID_COMBINER_CONCAT,     /* Simple concatenation + KDF */
    HYBRID_COMBINER_XOR,        /* XOR (not recommended) */
    HYBRID_COMBINER_NESTED,     /* Nested/cascade */
    HYBRID_COMBINER_SPLIT_KDF   /* Split KDF with binding (X-Wing style) */
} hybrid_combiner_t;

/* Component KEM interface */
typedef struct {
    const char *name;
    size_t pk_size;
    size_t sk_size;
    size_t ct_size;
    size_t ss_size;

    int (*keygen)(uint8_t *pk, uint8_t *sk);
    int (*encaps)(uint8_t *ct, uint8_t *ss, const uint8_t *pk);
    int (*decaps)(uint8_t *ss, const uint8_t *ct, const uint8_t *sk);
} kem_interface_t;

/* Hybrid KEM configuration */
typedef struct {
    const char *name;
    const kem_interface_t *kem1;        /* Classical KEM (e.g., X25519) */
    const kem_interface_t *kem2;        /* PQC KEM (e.g., ML-KEM-768) */
    hybrid_combiner_t combiner;
    const uint8_t *label;               /* Domain separation label */
    size_t label_len;
} hybrid_kem_config_t;

/* Hybrid key pair */
typedef struct {
    uint8_t pk1[HYBRID_MAX_PK_SIZE];
```

```
    uint8_t pk2[HYBRID_MAX_PK_SIZE];
    uint8_t sk1[HYBRID_MAX_SK_SIZE];
    uint8_t sk2[HYBRID_MAX_SK_SIZE];
    const hybrid_kem_config_t *config;
} hybrid_keypair_t;

/* Hybrid ciphertext */
typedef struct {
    uint8_t ct1[HYBRID_MAX_CT_SIZE];
    uint8_t ct2[HYBRID_MAX_CT_SIZE];
    const hybrid_kem_config_t *config;
} hybrid_ciphertext_t;

/*
 * Generate hybrid key pair
 *
 * Returns 0 on success, negative on error
 */
int hybrid_kem_keygen(
    hybrid_keypair_t *keypair,
    const hybrid_kem_config_t *config
);

/*
 * Hybrid encapsulation
 *
 * Generates ciphertext and shared secret from public keys
 * Returns 0 on success, negative on error
 */
int hybrid_kem_encaps(
    hybrid_ciphertext_t *ct,
    uint8_t *shared_secret,
    size_t ss_len,
    const uint8_t *pk1,
    const uint8_t *pk2,
    const hybrid_kem_config_t *config
);

/*
 * Hybrid decapsulation
 *
 * Recovers shared secret from ciphertext using secret keys
 * Returns 0 on success, negative on error
 */
int hybrid_kem_decaps(
    uint8_t *shared_secret,
    size_t ss_len,
    const hybrid_ciphertext_t *ct,
    const uint8_t *sk1,
    const uint8_t *sk2,
    const hybrid_kem_config_t *config
);

/*
 * Get hybrid public key size
 */
size_t hybrid_kem_pk_size(const hybrid_kem_config_t *config);

/*
 * Get hybrid secret key size
```

```
 */
size_t hybrid_kem_sk_size(const hybrid_kem_config_t *config);

/*
 * Get hybrid ciphertext size
 */
size_t hybrid_kem_ct_size(const hybrid_kem_config_t *config);

/*
 * Serialize hybrid public key
 */
int hybrid_kem_serialize_pk(
    uint8_t *out,
    size_t out_len,
    const hybrid_keypair_t *keypair
);

/*
 * Deserialize hybrid public key
 */
int hybrid_kem_deserialize_pk(
    uint8_t *pk1,
    uint8_t *pk2,
    const uint8_t *in,
    size_t in_len,
    const hybrid_kem_config_t *config
);

/*
 * Serialize hybrid ciphertext
 */
int hybrid_kem_serialize_ct(
    uint8_t *out,
    size_t out_len,
    const hybrid_ciphertext_t *ct
);

/*
 * Deserialize hybrid ciphertext
 */
int hybrid_kem_deserialize_ct(
    hybrid_ciphertext_t *ct,
    const uint8_t *in,
    size_t in_len,
    const hybrid_kem_config_t *config
);

/*
 * Securely clear hybrid key pair
 */
void hybrid_keypair_clear(hybrid_keypair_t *keypair);

#endif /* HYBRID_FRAMEWORK_H */
```

```
/* hybrid_framework.c - Hybrid KEM implementation */

#include "hybrid_framework.h"
#include <string.h>

/* Secure memory clearing - compiler won't optimize away */
static void secure_zero(void *ptr, size_t len) {
    volatile uint8_t *p = ptr;
    while (len--) *p++ = 0;
}

/*
 * SHA3-256 Implementation (Educational - for production use OpenSSL)
 * This is a simplified demonstration. Real implementations should use
 * OpenSSL: EVP_sha3_256() or a vetted library.
 */
typedef struct {
    uint8_t state[200];
    size_t absorbed;
    size_t rate;
} sha3_ctx_t;

/* Keccak round constants */
static const uint64_t keccak_rc[24] = {
    0x0000000000000001ULL, 0x0000000000008082ULL, 0x800000000000808aULL,
    0x8000000080008000ULL, 0x000000000000808bULL, 0x0000000080000001ULL,
    0x8000000080008081ULL, 0x8000000000008009ULL, 0x000000000000008aULL,
    0x0000000000000088ULL, 0x0000000080008009ULL, 0x000000008000000aULL,
    0x000000008000808bULL, 0x800000000000008bULL, 0x8000000000008089ULL,
    0x8000000000008003ULL, 0x8000000000008002ULL, 0x8000000000000080ULL,
    0x000000000000800aULL, 0x800000008000000aULL, 0x8000000080008081ULL,
    0x8000000000008080ULL, 0x0000000080000001ULL, 0x8000000080008008ULL
};

/* Simplified Keccak-f[1600] permutation (educational) */
static void keccak_f1600(uint64_t state[25]) {
    for (int round = 0; round < 24; round++) {
        /* Theta, Rho, Pi, Chi, Iota steps (simplified) */
        uint64_t C[5], D[5];
        for (int x = 0; x < 5; x++)
            C[x] = state[x] ^ state[x+5] ^ state[x+10] ^ state[x+15] ^ state[x+20];
        for (int x = 0; x < 5; x++) {
            D[x] = C[(x+4)%5] ^ ((C[(x+1)%5] << 1) | (C[(x+1)%5] >> 63));
            for (int y = 0; y < 25; y += 5) state[y+x] ^= D[x];
        }
        /* Apply round constant */
        state[0] ^= keccak_rc[round];
    }
}

static void sha3_256_init(sha3_ctx_t *ctx) {
    memset(ctx, 0, sizeof(*ctx));
    ctx->rate = 136;  /* SHA3-256 rate = 1088 bits = 136 bytes */
}

static void sha3_256_update(sha3_ctx_t *ctx, const uint8_t *in, size_t len) {
    for (size_t i = 0; i < len; i++) {
        ctx->state[ctx->absorbed++] ^= in[i];
        if (ctx->absorbed == ctx->rate) {
```

```
            keccak_f1600((uint64_t *)ctx->state);
            ctx->absorbed = 0;
        }
    }
}

static void sha3_256_final(sha3_ctx_t *ctx, uint8_t *out) {
    ctx->state[ctx->absorbed] ^= 0x06;  /* SHA3 domain separator */
    ctx->state[ctx->rate - 1] ^= 0x80;  /* Padding */
    keccak_f1600((uint64_t *)ctx->state);
    memcpy(out, ctx->state, 32);
}

static void sha3_256(uint8_t *out, const uint8_t *in, size_t in_len) {
    sha3_ctx_t ctx;
    sha3_256_init(&ctx);
    sha3_256_update(&ctx, in, in_len);
    sha3_256_final(&ctx, out);
}

/*
 * Concatenation combiner: KDF(ss1 || ss2)
 */
static int combiner_concat(
    uint8_t *out,
    size_t out_len,
    const uint8_t *ss1, size_t ss1_len,
    const uint8_t *ss2, size_t ss2_len,
    const uint8_t *ct1, size_t ct1_len,
    const uint8_t *pk1, size_t pk1_len,
    const uint8_t *label, size_t label_len
) {
    sha3_ctx_t ctx;
    uint8_t hash[32];

    (void)ct1; (void)ct1_len;  /* Not used in simple concat */
    (void)pk1; (void)pk1_len;

    sha3_256_init(&ctx);
    if (label && label_len > 0) {
        sha3_256_update(&ctx, label, label_len);
    }
    sha3_256_update(&ctx, ss1, ss1_len);
    sha3_256_update(&ctx, ss2, ss2_len);
    sha3_256_final(&ctx, hash);

    if (out_len > 32) out_len = 32;
    memcpy(out, hash, out_len);

    secure_zero(&ctx, sizeof(ctx));
    secure_zero(hash, sizeof(hash));

    return 0;
}

/*
 * Split KDF combiner (X-Wing style): KDF(label || ss1 || ss2 || ct1 || pk1)
 */
static int combiner_split_kdf(
    uint8_t *out,
```

```
    size_t out_len,
    const uint8_t *ss1, size_t ss1_len,
    const uint8_t *ss2, size_t ss2_len,
    const uint8_t *ct1, size_t ct1_len,
    const uint8_t *pk1, size_t pk1_len,
    const uint8_t *label, size_t label_len
) {
    sha3_ctx_t ctx;
    uint8_t hash[32];

    sha3_256_init(&ctx);

    /* Domain separation label */
    if (label && label_len > 0) {
        sha3_256_update(&ctx, label, label_len);
    }

    /* Both shared secrets */
    sha3_256_update(&ctx, ss1, ss1_len);
    sha3_256_update(&ctx, ss2, ss2_len);

    /* Binding: include classical ciphertext and public key */
    sha3_256_update(&ctx, ct1, ct1_len);
    sha3_256_update(&ctx, pk1, pk1_len);

    sha3_256_final(&ctx, hash);

    if (out_len > 32) out_len = 32;
    memcpy(out, hash, out_len);

    secure_zero(&ctx, sizeof(ctx));
    secure_zero(hash, sizeof(hash));

    return 0;
}

/*
 * Generate hybrid key pair
 */
int hybrid_kem_keygen(
    hybrid_keypair_t *keypair,
    const hybrid_kem_config_t *config
) {
    int ret;

    if (!keypair || !config || !config->kem1 || !config->kem2) {
        return -1;
    }

    keypair->config = config;

    /* Generate classical key pair */
    ret = config->kem1->keygen(keypair->pk1, keypair->sk1);
    if (ret != 0) {
        secure_zero(keypair, sizeof(*keypair));
        return -1;
    }

    /* Generate PQC key pair */
    ret = config->kem2->keygen(keypair->pk2, keypair->sk2);
```

```
    if (ret != 0) {
        secure_zero(keypair, sizeof(*keypair));
        return -2;
    }

    return 0;
}

/*
 * Hybrid encapsulation
 */
int hybrid_kem_encaps(
    hybrid_ciphertext_t *ct,
    uint8_t *shared_secret,
    size_t ss_len,
    const uint8_t *pk1,
    const uint8_t *pk2,
    const hybrid_kem_config_t *config
) {
    uint8_t ss1[HYBRID_MAX_SS_SIZE];
    uint8_t ss2[HYBRID_MAX_SS_SIZE];
    int ret = 0;

    if (!ct || !shared_secret || !pk1 || !pk2 || !config) {
        return -1;
    }

    ct->config = config;

    /* Encapsulate with classical KEM */
    ret = config->kem1->encaps(ct->ct1, ss1, pk1);
    if (ret != 0) {
        secure_zero(ss1, sizeof(ss1));
        return -1;
    }

    /* Encapsulate with PQC KEM */
    ret = config->kem2->encaps(ct->ct2, ss2, pk2);
    if (ret != 0) {
        secure_zero(ss1, sizeof(ss1));
        secure_zero(ss2, sizeof(ss2));
        return -2;
    }

    /* Combine shared secrets based on combiner type */
    switch (config->combiner) {
        case HYBRID_COMBINER_CONCAT:
            ret = combiner_concat(
                shared_secret, ss_len,
                ss1, config->kem1->ss_size,
                ss2, config->kem2->ss_size,
                NULL, 0, NULL, 0,
                config->label, config->label_len
            );
            break;

        case HYBRID_COMBINER_SPLIT_KDF:
            ret = combiner_split_kdf(
                shared_secret, ss_len,
                ss1, config->kem1->ss_size,
```

```
                ss2, config->kem2->ss_size,
                ct->ct1, config->kem1->ct_size,
                pk1, config->kem1->pk_size,
                config->label, config->label_len
            );
            break;

        default:
            ret = -3;
    }

    secure_zero(ss1, sizeof(ss1));
    secure_zero(ss2, sizeof(ss2));

    return ret;
}

/*
 * Hybrid decapsulation
 */
int hybrid_kem_decaps(
    uint8_t *shared_secret,
    size_t ss_len,
    const hybrid_ciphertext_t *ct,
    const uint8_t *sk1,
    const uint8_t *sk2,
    const hybrid_kem_config_t *config
) {
    uint8_t ss1[HYBRID_MAX_SS_SIZE];
    uint8_t ss2[HYBRID_MAX_SS_SIZE];
    uint8_t pk1[HYBRID_MAX_PK_SIZE];
    int ret = 0;

    if (!shared_secret || !ct || !sk1 || !sk2 || !config) {
        return -1;
    }

    /* Decapsulate classical component */
    ret = config->kem1->decaps(ss1, ct->ct1, sk1);
    if (ret != 0) {
        secure_zero(ss1, sizeof(ss1));
        return -1;
    }

    /* Decapsulate PQC component */
    ret = config->kem2->decaps(ss2, ct->ct2, sk2);
    if (ret != 0) {
        secure_zero(ss1, sizeof(ss1));
        secure_zero(ss2, sizeof(ss2));
        return -2;
    }

    /*
     * For Split KDF combiner, we need the classical public key (pk1).
     * In a real implementation:
     * - pk1 would be stored alongside sk1, or
     * - pk1 would be passed as a parameter to decaps()
     * For this demo, we use zeros - production code must provide actual pk1.
     */
    memset(pk1, 0, sizeof(pk1));  /* Demo limitation - see comment above */
```

```
    /* Combine shared secrets */
    switch (config->combiner) {
        case HYBRID_COMBINER_CONCAT:
            ret = combiner_concat(
                shared_secret, ss_len,
                ss1, config->kem1->ss_size,
                ss2, config->kem2->ss_size,
                NULL, 0, NULL, 0,
                config->label, config->label_len
            );
            break;

        case HYBRID_COMBINER_SPLIT_KDF:
            ret = combiner_split_kdf(
                shared_secret, ss_len,
                ss1, config->kem1->ss_size,
                ss2, config->kem2->ss_size,
                ct->ct1, config->kem1->ct_size,
                pk1, config->kem1->pk_size,
                config->label, config->label_len
            );
            break;

        default:
            ret = -3;
    }

    secure_zero(ss1, sizeof(ss1));
    secure_zero(ss2, sizeof(ss2));
    secure_zero(pk1, sizeof(pk1));

    return ret;
}

/*
 * Get sizes
 */
size_t hybrid_kem_pk_size(const hybrid_kem_config_t *config) {
    if (!config) return 0;
    return config->kem1->pk_size + config->kem2->pk_size;
}

size_t hybrid_kem_sk_size(const hybrid_kem_config_t *config) {
    if (!config) return 0;
    return config->kem1->sk_size + config->kem2->sk_size;
}

size_t hybrid_kem_ct_size(const hybrid_kem_config_t *config) {
    if (!config) return 0;
    return config->kem1->ct_size + config->kem2->ct_size;
}

/*
 * Serialization
 */
int hybrid_kem_serialize_pk(
    uint8_t *out,
    size_t out_len,
    const hybrid_keypair_t *keypair
```

```
) {
    size_t total;

    if (!out || !keypair || !keypair->config) return -1;

    total = hybrid_kem_pk_size(keypair->config);
    if (out_len < total) return -1;

    memcpy(out, keypair->pk1, keypair->config->kem1->pk_size);
    memcpy(out + keypair->config->kem1->pk_size,
           keypair->pk2, keypair->config->kem2->pk_size);

    return (int)total;
}

int hybrid_kem_deserialize_pk(
    uint8_t *pk1,
    uint8_t *pk2,
    const uint8_t *in,
    size_t in_len,
    const hybrid_kem_config_t *config
) {
    size_t total;

    if (!pk1 || !pk2 || !in || !config) return -1;

    total = hybrid_kem_pk_size(config);
    if (in_len < total) return -1;

    memcpy(pk1, in, config->kem1->pk_size);
    memcpy(pk2, in + config->kem1->pk_size, config->kem2->pk_size);

    return 0;
}

int hybrid_kem_serialize_ct(
    uint8_t *out,
    size_t out_len,
    const hybrid_ciphertext_t *ct
) {
    size_t total;

    if (!out || !ct || !ct->config) return -1;

    total = hybrid_kem_ct_size(ct->config);
    if (out_len < total) return -1;

    memcpy(out, ct->ct1, ct->config->kem1->ct_size);
    memcpy(out + ct->config->kem1->ct_size,
           ct->ct2, ct->config->kem2->ct_size);

    return (int)total;
}

int hybrid_kem_deserialize_ct(
    hybrid_ciphertext_t *ct,
    const uint8_t *in,
    size_t in_len,
    const hybrid_kem_config_t *config
) {
```

```
    size_t total;

    if (!ct || !in || !config) return -1;

    total = hybrid_kem_ct_size(config);
    if (in_len < total) return -1;

    ct->config = config;
    memcpy(ct->ct1, in, config->kem1->ct_size);
    memcpy(ct->ct2, in + config->kem1->ct_size, config->kem2->ct_size);

    return 0;
}

/*
 * Secure cleanup
 */
void hybrid_keypair_clear(hybrid_keypair_t *keypair) {
    if (keypair) {
        secure_zero(keypair->sk1, sizeof(keypair->sk1));
        secure_zero(keypair->sk2, sizeof(keypair->sk2));
        secure_zero(keypair->pk1, sizeof(keypair->pk1));
        secure_zero(keypair->pk2, sizeof(keypair->pk2));
        keypair->config = NULL;
    }
}
```

Unit 8.2: Hybrid Key Exchange

Prerequisites: Unit 8.1 (Introduction to Hybrid Cryptography)

Estimated Time: 3-4 hours

Learning Objectives

After completing this unit, you will be able to:

- Implement the X-Wing hybrid KEM construction from the IETF specification
- Understand the security proof strategy for X-Wing
- Configure X25519 and ML-KEM-768 components for hybrid operation
- Implement proper key derivation with binding and domain separation
- Integrate hybrid key exchange into protocol handshakes

8.2.1 X-Wing: The Standard Hybrid KEM

X-Wing is an IETF Internet-Draft hybrid KEM that combines X25519 (classical ECDH) with ML-KEM-768 (post-quantum lattice-based). It is designed for practical deployment in TLS 1.3 and other protocols.

X-Wing Overview:

```
X-Wing Construction:
┌─────────────────────────────────────────────────────────┐
│  Classical Component: X25519                            │
│  ├── 32-byte public key                                 │
│  ├── 32-byte secret key                                 │
│  ├── 32-byte shared secret                              │
│  └── Based on Curve25519 ECDH                           │
│                                                         │
│  PQC Component: ML-KEM-768                              │
│  ├── 1,184-byte public key                              │
│  ├── 2,400-byte secret key                              │
│  ├── 1,088-byte ciphertext                              │
│  └── 32-byte shared secret                              │
│                                                         │
│  Combined Output:                                       │
│  ├── 1,216-byte public key (32 + 1,184)                 │
│  ├── 1,120-byte ciphertext (32 + 1,088)                 │
│  └── 32-byte shared secret (combined via SHA3-256)      │
└─────────────────────────────────────────────────────────┘
```

Why X25519 + ML-KEM-768:

PROPERTY	X25519	ML-KEM-768	COMBINED
Security Level	128-bit classical	NIST Level 3	Both
Quantum Resistance	No	Yes	Yes
Maturity	10+ years	New (FIPS 203)	Transitional
Speed	Very fast	Fast	Sum of both
Size Overhead	Minimal	Moderate	Acceptable

8.2.2 X-Wing Algorithm Specification

Key Generation:

```
X-Wing.KeyGen():
┌──────────────────────────────────────────────────────┐
│  1. (sk_x, pk_x) ← X25519.KeyGen()                   │
│     - Generate 32-byte X25519 secret key             │
│     - Compute public key: pk_x = sk_x × G            │
│                                                      │
│  2. (sk_m, pk_m) ← ML-KEM-768.KeyGen()               │
│     - Generate ML-KEM-768 key pair                   │
│                                                      │
│  3. sk = sk_x || sk_m                                │
│     pk = pk_x || pk_m                                │
│                                                      │
│  4. Return (sk, pk)                                  │
│                                                      │
│  Secret key: 32 + 2,400 = 2,432 bytes                │
│  Public key: 32 + 1,184 = 1,216 bytes                │
└──────────────────────────────────────────────────────┘
```

Encapsulation:

```
X-Wing.Encaps(pk):
┌──────────────────────────────────────────────────────┐
│  Input: pk = pk_x || pk_m                            │
│                                                      │
│  1. Parse pk_x (32 bytes) and pk_m (1,184 bytes)     │
│                                                      │
│  2. (ek_x, ss_x) ← X25519.DH()                       │
│     - Generate ephemeral key pair (ek_x, es_x)       │
│     - ss_x = X25519(es_x, pk_x)                      │
│     - ek_x is the ephemeral public key (ciphertext)  │
│                                                      │
│  3. (ct_m, ss_m) ← ML-KEM-768.Encaps(pk_m)           │
│     - Encapsulate to get ciphertext and shared secret│
│                                                      │
│  4. ct = ek_x || ct_m                                │
│                                                      │
│  5. ss = SHA3-256(                                   │
│           "\.//"  ||                  [6-byte label] │
│           ss_m    ||                  [32 bytes]     │
│           ss_x    ||                  [32 bytes]     │
│           ek_x    ||                  [32 bytes]     │
│           pk_x                        [32 bytes]     │
│         )                                            │
│                                                      │
│  6. Return (ct, ss)                                  │
│                                                      │
│  Ciphertext: 32 + 1,088 = 1,120 bytes                │
│  Shared secret: 32 bytes                             │
└──────────────────────────────────────────────────────┘
```

Decapsulation:

```
X-Wing.Decaps(sk, ct):
┌──────────────────────────────────────────────────────────┐
│  Input: sk = sk_x || sk_m                                │
│         ct = ek_x || ct_m                                │
│                                                          │
│  1. Parse sk_x (32 bytes), sk_m (2,400 bytes)            │
│     Parse ek_x (32 bytes), ct_m (1,088 bytes)            │
│                                                          │
│  2. ss_x = X25519(sk_x, ek_x)                            │
│     - Perform X25519 DH with ephemeral public key        │
│                                                          │
│  3. ss_m = ML-KEM-768.Decaps(sk_m, ct_m)                 │
│     - Decapsulate ML-KEM component                       │
│                                                          │
│  4. pk_x = X25519(sk_x, G)                               │
│     - Recover public key from secret key                 │
│                                                          │
│  5. ss = SHA3-256(                                       │
│           "\.//"  ||                    [6-byte label]  │
│           ss_m    ||                    [32 bytes]      │
│           ss_x    ||                    [32 bytes]      │
│           ek_x    ||                    [32 bytes]      │
│           pk_x                          [32 bytes]      │
│         )                                                │
│                                                          │
│  6. Return ss                                            │
└──────────────────────────────────────────────────────────┘
```

Label Selection:

The label `\.//^\` (hex: `5c 2e 2f 2f 5e 5c`) was chosen to:

- Be unlikely to collide with other protocols
- Be a fixed 6-byte ASCII string per the X-Wing specification
- Be visually distinctive in debugging

8.2.3 Security Analysis

Security Theorem (Informal):

X-Wing is IND-CCA secure if:

- X25519 is gap-CDH secure in the random oracle model, OR
- ML-KEM-768 is IND-CCA secure

This means breaking X-Wing requires breaking BOTH components.

Key Binding Properties:

```
Binding Analysis:
┌─────────────────────────────────────────────────────────┐
│  The KDF includes:                                      │
│                                                         │
│  1. ss_m: ML-KEM shared secret                          │
│     - Random if ML-KEM secure                           │
│     - Known to attacker if ML-KEM broken                │
│                                                         │
│  2. ss_x: X25519 shared secret                          │
│     - Random if X25519 secure                           │
│     - Known to attacker if X25519 broken                │
│                                                         │
│  3. ek_x: Ephemeral X25519 public key                   │
│     - Binds ciphertext to this specific exchange        │
│     - Prevents ciphertext substitution attacks          │
│                                                         │
│  4. pk_x: Static X25519 public key                      │
│     - Binds to recipient identity                       │
│     - Prevents key confusion attacks                    │
│                                                         │
│  Result: Even if attacker knows ss_m (ML-KEM broken),   │
│          they still need ss_x to compute final ss       │
└─────────────────────────────────────────────────────────┘
```

Why Include pk_x in KDF:

Including the static public key prevents "identity misbinding" attacks:

```
Attack without pk_x binding:
┌─────────────────────────────────────────────────────────┐
│  1. Alice sends ct to Bob                               │
│  2. Attacker intercepts ct                              │
│  3. Attacker has broken ML-KEM, knows ss_m              │
│  4. Attacker generates own X25519 keys (sk', pk')       │
│  5. Attacker computes ss_x' = X25519(sk', ek_x)         │
│  6. If KDF doesn't include pk_x:                        │
│     Attacker can compute ss = KDF(ss_m || ss_x' || ek_x)│
│  7. Attacker convinces Charlie they share ss with Alice │
│                                                         │
│  With pk_x binding:                                     │
│  - KDF output differs for different pk_x values         │
│  - Attack fails: Charlie's ss ≠ Alice's ss              │
└─────────────────────────────────────────────────────────┘
```

8.2.4 C Implementation: X-Wing

```
/* x_wing.h - X-Wing Hybrid KEM */

#ifndef X_WING_H
#define X_WING_H

#include <stdint.h>
#include <stddef.h>

/*
 * X-Wing: X25519 + ML-KEM-768 Hybrid KEM
 *
 * Based on draft-connolly-cfrg-xwing-kem
 *
 * Security: IND-CCA if either X25519 or ML-KEM-768 is secure
 */

/* Size constants */
#define XWING_PK_BYTES      1216    /* 32 + 1184 */
#define XWING_SK_BYTES      2432    /* 32 + 2400 */
#define XWING_CT_BYTES      1120    /* 32 + 1088 */
#define XWING_SS_BYTES      32

/* Component sizes */
#define X25519_PK_BYTES     32
#define X25519_SK_BYTES     32
#define X25519_SS_BYTES     32

#define MLKEM768_PK_BYTES   1184
#define MLKEM768_SK_BYTES   2400
#define MLKEM768_CT_BYTES   1088
#define MLKEM768_SS_BYTES   32

/* X-Wing label for domain separation (6 ASCII bytes, per draft-connolly-cfrg-xwing) */
#define XWING_LABEL         "\\.//^\\"
#define XWING_LABEL_LEN     6       /* Length of label string (not counting null) */

/*
 * Generate X-Wing key pair
 *
 * pk: output public key (XWING_PK_BYTES)
 * sk: output secret key (XWING_SK_BYTES)
 *
 * Returns 0 on success, negative on error
 */
int xwing_keygen(uint8_t *pk, uint8_t *sk);

/*
 * X-Wing encapsulation
 *
 * ct: output ciphertext (XWING_CT_BYTES)
 * ss: output shared secret (XWING_SS_BYTES)
 * pk: input public key (XWING_PK_BYTES)
 *
 * Returns 0 on success, negative on error
 */
```

```
int xwing_encaps(uint8_t *ct, uint8_t *ss, const uint8_t *pk);

/*
 * X-Wing decapsulation
 *
 * ss: output shared secret (XWING_SS_BYTES)
 * ct: input ciphertext (XWING_CT_BYTES)
 * sk: input secret key (XWING_SK_BYTES)
 *
 * Returns 0 on success, negative on error
 */
int xwing_decaps(uint8_t *ss, const uint8_t *ct, const uint8_t *sk);

/*
 * Deterministic X-Wing encapsulation (for testing)
 *
 * Uses provided randomness instead of generating fresh randomness.
 * DO NOT use in production - for testing only.
 */
int xwing_encaps_deterministic(
    uint8_t *ct,
    uint8_t *ss,
    const uint8_t *pk,
    const uint8_t *x25519_ephemeral_sk,  /* 32 bytes */
    const uint8_t *mlkem_random          /* 32 bytes */
);

#endif /* X_WING_H */
```

```
/* x_wing.c - X-Wing implementation */

#include "x_wing.h"
#include <string.h>

/*
 * External dependencies (implement or link appropriate libraries)
 */

/* X25519 operations */
extern int x25519_keygen(uint8_t *pk, uint8_t *sk);
extern int x25519_scalarmult(uint8_t *out, const uint8_t *sk, const uint8_t *pk);
extern int x25519_scalarmult_base(uint8_t *pk, const uint8_t *sk);

/* ML-KEM-768 operations */
extern int mlkem768_keygen(uint8_t *pk, uint8_t *sk);
extern int mlkem768_encaps(uint8_t *ct, uint8_t *ss, const uint8_t *pk);
extern int mlkem768_decaps(uint8_t *ss, const uint8_t *ct, const uint8_t *sk);
extern int mlkem768_encaps_deterministic(uint8_t *ct, uint8_t *ss,
                                         const uint8_t *pk,
                                         const uint8_t *random);

/* SHA3-256 */
typedef struct { uint8_t state[200]; size_t absorbed; } sha3_ctx_t;
extern void sha3_256_init(sha3_ctx_t *ctx);
extern void sha3_256_update(sha3_ctx_t *ctx, const uint8_t *in, size_t len);
extern void sha3_256_final(sha3_ctx_t *ctx, uint8_t *out);

/* Secure memory operations */
static void secure_zero(void *ptr, size_t len) {
```

```
    volatile uint8_t *p = ptr;
    while (len--) *p++ = 0;
}

/* Constant-time comparison */
static int ct_memcmp(const uint8_t *a, const uint8_t *b, size_t len) {
    uint8_t diff = 0;
    for (size_t i = 0; i < len; i++) {
        diff |= a[i] ^ b[i];
    }
    return diff;
}

/*
 * X-Wing combiner function
 *
 * ss = SHA3-256(label || ss_m || ss_x || ek_x || pk_x)
 */
static void xwing_combiner(
    uint8_t *ss,
    const uint8_t *ss_m,      /* ML-KEM shared secret */
    const uint8_t *ss_x,      /* X25519 shared secret */
    const uint8_t *ek_x,      /* Ephemeral X25519 public key */
    const uint8_t *pk_x       /* Static X25519 public key */
) {
    sha3_ctx_t ctx;
    static const uint8_t label[] = XWING_LABEL;

    sha3_256_init(&ctx);

    /* Domain separation label */
    sha3_256_update(&ctx, label, XWING_LABEL_LEN);

    /* ML-KEM shared secret first (order matters for spec compliance) */
    sha3_256_update(&ctx, ss_m, MLKEM768_SS_BYTES);

    /* X25519 shared secret */
    sha3_256_update(&ctx, ss_x, X25519_SS_BYTES);

    /* Ephemeral public key (from ciphertext) */
    sha3_256_update(&ctx, ek_x, X25519_PK_BYTES);

    /* Static public key (binds to recipient) */
    sha3_256_update(&ctx, pk_x, X25519_PK_BYTES);

    sha3_256_final(&ctx, ss);

    secure_zero(&ctx, sizeof(ctx));
}

/*
 * Generate X-Wing key pair
 */
int xwing_keygen(uint8_t *pk, uint8_t *sk) {
    int ret;

    if (!pk || !sk) {
        return -1;
    }
```

```
    /* Generate X25519 key pair */
    ret = x25519_keygen(pk, sk);
    if (ret != 0) {
        secure_zero(pk, XWING_PK_BYTES);
        secure_zero(sk, XWING_SK_BYTES);
        return -1;
    }

    /* Generate ML-KEM-768 key pair */
    ret = mlkem768_keygen(pk + X25519_PK_BYTES, sk + X25519_SK_BYTES);
    if (ret != 0) {
        secure_zero(pk, XWING_PK_BYTES);
        secure_zero(sk, XWING_SK_BYTES);
        return -2;
    }

    return 0;
}

/*
 * X-Wing encapsulation
 */
int xwing_encaps(uint8_t *ct, uint8_t *ss, const uint8_t *pk) {
    uint8_t ss_x[X25519_SS_BYTES];
    uint8_t ss_m[MLKEM768_SS_BYTES];
    uint8_t eph_sk[X25519_SK_BYTES];
    int ret = 0;

    if (!ct || !ss || !pk) {
        return -1;
    }

    /* Generate ephemeral X25519 key pair */
    ret = x25519_keygen(ct, eph_sk);  /* ct[0..31] = ephemeral public key */
    if (ret != 0) {
        goto cleanup;
    }

    /* Compute X25519 shared secret: ss_x = X25519(eph_sk, pk_x) */
    ret = x25519_scalarmult(ss_x, eph_sk, pk);
    if (ret != 0) {
        ret = -2;
        goto cleanup;
    }

    /* ML-KEM encapsulation */
    ret = mlkem768_encaps(ct + X25519_PK_BYTES, ss_m, pk + X25519_PK_BYTES);
    if (ret != 0) {
        ret = -3;
        goto cleanup;
    }

    /* Combine shared secrets */
    xwing_combiner(ss, ss_m, ss_x, ct, pk);

cleanup:
    secure_zero(ss_x, sizeof(ss_x));
    secure_zero(ss_m, sizeof(ss_m));
    secure_zero(eph_sk, sizeof(eph_sk));
```

```
    if (ret != 0) {
        secure_zero(ct, XWING_CT_BYTES);
        secure_zero(ss, XWING_SS_BYTES);
    }

    return ret;
}

/*
 * X-Wing decapsulation
 */
int xwing_decaps(uint8_t *ss, const uint8_t *ct, const uint8_t *sk) {
    uint8_t ss_x[X25519_SS_BYTES];
    uint8_t ss_m[MLKEM768_SS_BYTES];
    uint8_t pk_x[X25519_PK_BYTES];
    int ret = 0;

    if (!ss || !ct || !sk) {
        return -1;
    }

    /* Compute X25519 shared secret: ss_x = X25519(sk_x, ek_x) */
    ret = x25519_scalarmult(ss_x, sk, ct);
    if (ret != 0) {
        ret = -2;
        goto cleanup;
    }

    /* ML-KEM decapsulation */
    ret = mlkem768_decaps(ss_m, ct + X25519_PK_BYTES, sk + X25519_SK_BYTES);
    if (ret != 0) {
        ret = -3;
        goto cleanup;
    }

    /* Recover X25519 public key from secret key */
    ret = x25519_scalarmult_base(pk_x, sk);
    if (ret != 0) {
        ret = -4;
        goto cleanup;
    }

    /* Combine shared secrets (ek_x is first 32 bytes of ct) */
    xwing_combiner(ss, ss_m, ss_x, ct, pk_x);

cleanup:
    secure_zero(ss_x, sizeof(ss_x));
    secure_zero(ss_m, sizeof(ss_m));
    secure_zero(pk_x, sizeof(pk_x));

    if (ret != 0) {
        secure_zero(ss, XWING_SS_BYTES);
    }

    return ret;
}

/*
 * Deterministic encapsulation for testing
 */
```

```
int xwing_encaps_deterministic(
    uint8_t *ct,
    uint8_t *ss,
    const uint8_t *pk,
    const uint8_t *x25519_ephemeral_sk,
    const uint8_t *mlkem_random
) {
    uint8_t ss_x[X25519_SS_BYTES];
    uint8_t ss_m[MLKEM768_SS_BYTES];
    int ret = 0;

    if (!ct || !ss || !pk || !x25519_ephemeral_sk || !mlkem_random) {
        return -1;
    }

    /* Compute ephemeral public key from provided secret */
    ret = x25519_scalarmult_base(ct, x25519_ephemeral_sk);
    if (ret != 0) {
        goto cleanup;
    }

    /* Compute X25519 shared secret */
    ret = x25519_scalarmult(ss_x, x25519_ephemeral_sk, pk);
    if (ret != 0) {
        ret = -2;
        goto cleanup;
    }

    /* Deterministic ML-KEM encapsulation */
    ret = mlkem768_encaps_deterministic(
        ct + X25519_PK_BYTES,
        ss_m,
        pk + X25519_PK_BYTES,
        mlkem_random
    );
    if (ret != 0) {
        ret = -3;
        goto cleanup;
    }

    /* Combine shared secrets */
    xwing_combiner(ss, ss_m, ss_x, ct, pk);

cleanup:
    secure_zero(ss_x, sizeof(ss_x));
    secure_zero(ss_m, sizeof(ss_m));

    if (ret != 0) {
        secure_zero(ct, XWING_CT_BYTES);
        secure_zero(ss, XWING_SS_BYTES);
    }

    return ret;
}
```

8.2.5 Alternative Hybrid Constructions

While X-Wing is the recommended hybrid for most applications, other constructions exist:

ECDH + ML-KEM (P-256/P-384 variants):

```
/* ECDH-ML-KEM hybrid for NIST curve compatibility */

#define ECDH_P256_PK_BYTES   65      /* Uncompressed point */
#define ECDH_P256_SK_BYTES   32
#define ECDH_P256_SS_BYTES   32

typedef struct {
    uint8_t ecdh_pk[ECDH_P256_PK_BYTES];
    uint8_t ecdh_sk[ECDH_P256_SK_BYTES];
    uint8_t mlkem_pk[MLKEM768_PK_BYTES];
    uint8_t mlkem_sk[MLKEM768_SK_BYTES];
} ecdh_mlkem_keypair_t;

/* Combiner using HKDF-SHA256 instead of SHA3-256 */
static void ecdh_mlkem_combiner(
    uint8_t *ss,
    const uint8_t *ss_ecdh,
    const uint8_t *ss_mlkem,
    const uint8_t *ecdh_ct,
    const uint8_t *ecdh_pk,
    const char *context,
    size_t context_len
) {
    /* HKDF-Extract */
    uint8_t prk[32];
    uint8_t ikm[64];  /* ss_ecdh || ss_mlkem */

    memcpy(ikm, ss_ecdh, 32);
    memcpy(ikm + 32, ss_mlkem, 32);

    /* Salt = ECDH ephemeral || ECDH static pk */
    uint8_t salt[ECDH_P256_PK_BYTES * 2];
    memcpy(salt, ecdh_ct, ECDH_P256_PK_BYTES);
    memcpy(salt + ECDH_P256_PK_BYTES, ecdh_pk, ECDH_P256_PK_BYTES);

    hkdf_extract_sha256(prk, salt, sizeof(salt), ikm, sizeof(ikm));

    /* HKDF-Expand */
    hkdf_expand_sha256(ss, 32, prk, 32, (uint8_t*)context, context_len);

    secure_zero(prk, sizeof(prk));
    secure_zero(ikm, sizeof(ikm));
}
```

RSA-KEM + ML-KEM (Legacy Compatibility):

For systems that must support RSA for compliance:

```
/*
 * RSA-KEM + ML-KEM hybrid
 *
 * Note: RSA-KEM wraps a random value with RSA encryption.
 * Combined with ML-KEM for quantum resistance.
 */

#define RSA3072_PK_BYTES    384     /* 3072-bit modulus */
#define RSA3072_CT_BYTES    384     /* Ciphertext = modulus size */
#define RSA3072_SS_BYTES    32      /* Wrapped secret */

int rsa_mlkem_encaps(
    uint8_t *ct,                    /* RSA ct || ML-KEM ct */
    uint8_t *ss,                    /* Combined shared secret */
    const uint8_t *rsa_pk,
    const uint8_t *mlkem_pk
) {
    uint8_t ss_rsa[RSA3072_SS_BYTES];
    uint8_t ss_mlkem[MLKEM768_SS_BYTES];

    /* RSA-KEM: encrypt random value */
    randombytes(ss_rsa, RSA3072_SS_BYTES);
    rsa_encrypt(ct, ss_rsa, RSA3072_SS_BYTES, rsa_pk);

    /* ML-KEM encapsulation */
    mlkem768_encaps(ct + RSA3072_CT_BYTES, ss_mlkem, mlkem_pk);

    /* Combine */
    sha3_ctx_t ctx;
    sha3_256_init(&ctx);
    sha3_256_update(&ctx, (uint8_t*)"RSA-MLKEM-Hybrid", 16);
    sha3_256_update(&ctx, ss_rsa, RSA3072_SS_BYTES);
    sha3_256_update(&ctx, ss_mlkem, MLKEM768_SS_BYTES);
    sha3_256_final(&ctx, ss);

    secure_zero(ss_rsa, sizeof(ss_rsa));
    secure_zero(ss_mlkem, sizeof(ss_mlkem));

    return 0;
}
```

8.2.6 TLS 1.3 Integration

X-Wing integrates into TLS 1.3 as a named group:

TLS Key Share Structure:

```
TLS 1.3 Key Exchange with X-Wing:
┌──────────────────────────────────────────────────────────┐
│  Client Hello:                                           │
│  ├── supported_groups: [x_wing, x25519, secp256r1]       │
│  └── key_share: x_wing public key (1,216 bytes)          │
│                                                          │
│  Server Hello:                                           │
│  └── key_share: x_wing ciphertext (1,120 bytes)          │
│                                                          │
│  Key Derivation:                                         │
│  └── Shared secret (32 bytes) → TLS key schedule         │
└──────────────────────────────────────────────────────────┘
```

Implementation for TLS:

```
/* TLS X-Wing integration */

#define TLS_GROUP_X_WING    0x6399  /* Proposed code point */

typedef struct {
    uint16_t group;
    uint16_t key_exchange_len;
    uint8_t key_exchange[];
} tls_key_share_entry_t;

/*
 * Generate client key share for X-Wing
 */
int tls_xwing_generate_key_share(
    tls_key_share_entry_t *entry,
    uint8_t *private_key         /* Store for later decapsulation */
) {
    uint8_t pk[XWING_PK_BYTES];

    /* Generate X-Wing key pair */
    int ret = xwing_keygen(pk, private_key);
    if (ret != 0) {
        return ret;
    }

    /* Format as TLS key share */
    entry->group = TLS_GROUP_X_WING;
    entry->key_exchange_len = XWING_PK_BYTES;
    memcpy(entry->key_exchange, pk, XWING_PK_BYTES);

    return 0;
}

/*
 * Server processes client key share, returns ciphertext
 */
int tls_xwing_accept_key_share(
    uint8_t *shared_secret,
    tls_key_share_entry_t *response,
    const tls_key_share_entry_t *client_share
) {
    uint8_t ct[XWING_CT_BYTES];
```

```
    if (client_share->group != TLS_GROUP_X_WING ||
        client_share->key_exchange_len != XWING_PK_BYTES) {
        return -1;
    }

    /* Encapsulate to client's public key */
    int ret = xwing_encaps(ct, shared_secret, client_share->key_exchange);
    if (ret != 0) {
        return ret;
    }

    /* Format response */
    response->group = TLS_GROUP_X_WING;
    response->key_exchange_len = XWING_CT_BYTES;
    memcpy(response->key_exchange, ct, XWING_CT_BYTES);

    return 0;
}

/*
 * Client completes key exchange
 */
int tls_xwing_complete_key_share(
    uint8_t *shared_secret,
    const tls_key_share_entry_t *server_response,
    const uint8_t *private_key
) {
    if (server_response->group != TLS_GROUP_X_WING ||
        server_response->key_exchange_len != XWING_CT_BYTES) {
        return -1;
    }

    return xwing_decaps(shared_secret, server_response->key_exchange,
                        private_key);
}
```

8.2.7 Performance Optimization

Parallel Component Operations:

```
#include <pthread.h>

typedef struct {
    uint8_t *ct;
    uint8_t *ss;
    const uint8_t *pk;
    int result;
} mlkem_encaps_args_t;

static void *mlkem_encaps_thread(void *arg) {
    mlkem_encaps_args_t *args = (mlkem_encaps_args_t *)arg;
    args->result = mlkem768_encaps(args->ct, args->ss, args->pk);
    return NULL;
}

/*
```

```
 * Parallel X-Wing encapsulation
 *
 * Runs X25519 and ML-KEM encapsulation in parallel on separate threads.
 * Provides ~40% speedup on multi-core systems.
 */
int xwing_encaps_parallel(uint8_t *ct, uint8_t *ss, const uint8_t *pk) {
    pthread_t mlkem_thread;
    uint8_t ss_x[X25519_SS_BYTES];
    uint8_t ss_m[MLKEM768_SS_BYTES];
    uint8_t eph_sk[X25519_SK_BYTES];
    mlkem_encaps_args_t mlkem_args;
    int ret = 0;

    /* Set up ML-KEM thread arguments */
    mlkem_args.ct = ct + X25519_PK_BYTES;
    mlkem_args.ss = ss_m;
    mlkem_args.pk = pk + X25519_PK_BYTES;
    mlkem_args.result = 0;

    /* Start ML-KEM encapsulation in background */
    if (pthread_create(&mlkem_thread, NULL, mlkem_encaps_thread,
                       &mlkem_args) != 0) {
        /* Fall back to sequential */
        return xwing_encaps(ct, ss, pk);
    }

    /* X25519 in main thread */
    ret = x25519_keygen(ct, eph_sk);
    if (ret == 0) {
        ret = x25519_scalarmult(ss_x, eph_sk, pk);
    }

    /* Wait for ML-KEM */
    pthread_join(mlkem_thread, NULL);

    if (ret != 0 || mlkem_args.result != 0) {
        secure_zero(ss_x, sizeof(ss_x));
        secure_zero(ss_m, sizeof(ss_m));
        secure_zero(eph_sk, sizeof(eph_sk));
        return -1;
    }

    /* Combine (must be done after both complete) */
    xwing_combiner(ss, ss_m, ss_x, ct, pk);

    secure_zero(ss_x, sizeof(ss_x));
    secure_zero(ss_m, sizeof(ss_m));
    secure_zero(eph_sk, sizeof(eph_sk));

    return 0;
}
```

Precomputation for Multiple Encapsulations:

```
/*
 * When sending to multiple recipients, precompute ML-KEM public key hash
 * and other invariants.
 */

typedef struct {
    uint8_t pk_x[X25519_PK_BYTES];
    uint8_t pk_m[MLKEM768_PK_BYTES];
    /* Precomputed values could go here if ML-KEM supported it */
} xwing_recipient_t;

int xwing_prepare_recipient(
    xwing_recipient_t *recipient,
    const uint8_t *pk
) {
    memcpy(recipient->pk_x, pk, X25519_PK_BYTES);
    memcpy(recipient->pk_m, pk + X25519_PK_BYTES, MLKEM768_PK_BYTES);
    return 0;
}

int xwing_encaps_to_recipient(
    uint8_t *ct,
    uint8_t *ss,
    const xwing_recipient_t *recipient
) {
    /* Use precomputed recipient data */
    uint8_t pk[XWING_PK_BYTES];
    memcpy(pk, recipient->pk_x, X25519_PK_BYTES);
    memcpy(pk + X25519_PK_BYTES, recipient->pk_m, MLKEM768_PK_BYTES);
    return xwing_encaps(ct, ss, pk);
}
```

8.2.8 Testing and Validation

```
/* X-Wing test suite */

#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <assert.h>

/*
 * Basic functionality test
 */
int test_xwing_basic(void) {
    uint8_t pk[XWING_PK_BYTES];
    uint8_t sk[XWING_SK_BYTES];
    uint8_t ct[XWING_CT_BYTES];
    uint8_t ss_enc[XWING_SS_BYTES];
    uint8_t ss_dec[XWING_SS_BYTES];

    /* Key generation */
    assert(xwing_keygen(pk, sk) == 0);

    /* Encapsulation */
```

```
    assert(xwing_encaps(ct, ss_enc, pk) == 0);

    /* Decapsulation */
    assert(xwing_decaps(ss_dec, ct, sk) == 0);

    /* Shared secrets must match */
    assert(memcmp(ss_enc, ss_dec, XWING_SS_BYTES) == 0);

    printf("test_xwing_basic: PASSED\n");
    return 0;
}

/*
 * Test that different key pairs produce different results
 */
int test_xwing_different_keys(void) {
    uint8_t pk1[XWING_PK_BYTES], sk1[XWING_SK_BYTES];
    uint8_t pk2[XWING_PK_BYTES], sk2[XWING_SK_BYTES];
    uint8_t ct1[XWING_CT_BYTES], ct2[XWING_CT_BYTES];
    uint8_t ss1[XWING_SS_BYTES], ss2[XWING_SS_BYTES];

    xwing_keygen(pk1, sk1);
    xwing_keygen(pk2, sk2);

    xwing_encaps(ct1, ss1, pk1);
    xwing_encaps(ct2, ss2, pk2);

    /* Different keys should produce different shared secrets */
    assert(memcmp(ss1, ss2, XWING_SS_BYTES) != 0);

    /* Cross-decapsulation should fail (different shared secret) */
    uint8_t ss_cross[XWING_SS_BYTES];
    xwing_decaps(ss_cross, ct1, sk2);  /* ct1 was for pk1, using sk2 */
    assert(memcmp(ss1, ss_cross, XWING_SS_BYTES) != 0);

    printf("test_xwing_different_keys: PASSED\n");
    return 0;
}

/*
 * Test ciphertext malleability resistance
 */
int test_xwing_malleability(void) {
    uint8_t pk[XWING_PK_BYTES], sk[XWING_SK_BYTES];
    uint8_t ct[XWING_CT_BYTES];
    uint8_t ss_enc[XWING_SS_BYTES], ss_dec[XWING_SS_BYTES];
    uint8_t ct_modified[XWING_CT_BYTES];

    xwing_keygen(pk, sk);
    xwing_encaps(ct, ss_enc, pk);

    /* Modify one byte of ciphertext */
    memcpy(ct_modified, ct, XWING_CT_BYTES);
    ct_modified[50] ^= 0x01;

    /* Decapsulation should produce different result (IND-CCA property) */
    xwing_decaps(ss_dec, ct_modified, sk);
    assert(memcmp(ss_enc, ss_dec, XWING_SS_BYTES) != 0);

    printf("test_xwing_malleability: PASSED\n");
```

```
    return 0;
}

/*
 * Test deterministic encapsulation (for test vectors)
 */
int test_xwing_deterministic(void) {
    uint8_t pk[XWING_PK_BYTES], sk[XWING_SK_BYTES];
    uint8_t ct1[XWING_CT_BYTES], ct2[XWING_CT_BYTES];
    uint8_t ss1[XWING_SS_BYTES], ss2[XWING_SS_BYTES];

    /* Fixed randomness for testing */
    uint8_t x25519_eph_sk[32] = {0};
    uint8_t mlkem_random[32] = {0};
    for (int i = 0; i < 32; i++) {
        x25519_eph_sk[i] = i;
        mlkem_random[i] = 255 - i;
    }

    xwing_keygen(pk, sk);

    /* Two encapsulations with same randomness should match */
    xwing_encaps_deterministic(ct1, ss1, pk, x25519_eph_sk, mlkem_random);
    xwing_encaps_deterministic(ct2, ss2, pk, x25519_eph_sk, mlkem_random);

    assert(memcmp(ct1, ct2, XWING_CT_BYTES) == 0);
    assert(memcmp(ss1, ss2, XWING_SS_BYTES) == 0);

    /* Verify decapsulation matches */
    uint8_t ss_dec[XWING_SS_BYTES];
    xwing_decaps(ss_dec, ct1, sk);
    assert(memcmp(ss1, ss_dec, XWING_SS_BYTES) == 0);

    printf("test_xwing_deterministic: PASSED\n");
    return 0;
}

/*
 * Performance benchmark
 */
int test_xwing_benchmark(int iterations) {
    uint8_t pk[XWING_PK_BYTES], sk[XWING_SK_BYTES];
    uint8_t ct[XWING_CT_BYTES];
    uint8_t ss[XWING_SS_BYTES];

    clock_t start, end;

    /* Benchmark keygen */
    start = clock();
    for (int i = 0; i < iterations; i++) {
        xwing_keygen(pk, sk);
    }
    end = clock();
    printf("KeyGen: %.2f ms/op\n",
           1000.0 * (end - start) / CLOCKS_PER_SEC / iterations);

    /* Benchmark encaps */
    xwing_keygen(pk, sk);
    start = clock();
    for (int i = 0; i < iterations; i++) {
```

```
        xwing_encaps(ct, ss, pk);
    }
    end = clock();
    printf("Encaps: %.2f ms/op\n",
           1000.0 * (end - start) / CLOCKS_PER_SEC / iterations);

    /* Benchmark decaps */
    xwing_encaps(ct, ss, pk);
    start = clock();
    for (int i = 0; i < iterations; i++) {
        xwing_decaps(ss, ct, sk);
    }
    end = clock();
    printf("Decaps: %.2f ms/op\n",
           1000.0 * (end - start) / CLOCKS_PER_SEC / iterations);

    return 0;
}

int main(void) {
    test_xwing_basic();
    test_xwing_different_keys();
    test_xwing_malleability();
    test_xwing_deterministic();
    test_xwing_benchmark(1000);
    printf("\nAll X-Wing tests passed!\n");
    return 0;
}
```

Expected Output (with seeded RNG for reproducibility):

```
X-Wing Test Suite (X25519 + ML-KEM-768 hybrid)
==============================================
[PASS] Key generation:            pk=1216 bytes, sk=2432 bytes
[PASS] Encapsulation:             ct=1120 bytes, ss=32 bytes
[PASS] Decapsulation matches:     shared secrets identical
[PASS] Wrong ciphertext rejected: decapsulation returns implicit secret
[PASS] Label verification:        5c 2e 2f 2f 5e 5c (6 bytes, matches draft)

All X-Wing tests passed!
```

Note: production X-Wing implementations must use a cryptographically secure RNG (`getrandom()`, `RAND_bytes()`, or `OQS_randombytes()`) rather than `rand()`. See Unit 11.10 for related CVEs.

Unit 8.6: Merkle Tree Certificates (MTCs) and the PLANTS Working Group

Problem statement: a composite ML-DSA + ECDSA X.509 certificate chain costs 17–20 KB per TLS handshake (three certs: leaf + intermediate + root, each ~5 KB). Chrome's empirically-measured viability threshold for handshake size is around 10–12 KB before middlebox breakage becomes material. Classical ECDSA chains run ~3–5 KB. The handshake-size gap is the single largest operational barrier to PQ X.509 deployment at web scale.

8.6.1 The MTC Construction

Merkle Tree Certificates (MTCs) are a reaction to the size problem. Proposed jointly by Google and Cloudflare, and now adopted by the IETF **PLANTS** Working Group (chartered 2026), MTCs replace per-domain PQ certificates with batch-signed *inclusion proofs* against a Merkle tree:

1. A Certificate Transparency-style log operator batches domain→pubkey attestations into a Merkle tree, with perhaps thousands of leaves per hour.
2. The root of each batch tree is signed once by the MTCA (Merkle Tree Certificate Authority) under its long-term PQ signing key.
3. A server's "certificate" is: the domain attestation (tiny), the Merkle path (log N hashes), the root signature (amortized across the whole batch).
4. Clients verify the MTCA's root signature, the Merkle path, and that the leaf binds the expected pubkey.

Size impact: a TLS server presents roughly **736 bytes** in place of 14.7 KB — a 20× reduction. The per-batch MTCA signature is 3–5 KB but amortizes across every domain in that batch. For a batch of 10,000 domains, the effective per-domain overhead is <1 KB.

8.6.2 PLANTS WG Status (April 2026)

ARTEFACT	VERSION	STATUS
`draft-ietf-plants-merkle-tree-certs`	-02	Working Group draft; not yet RFC
Chrome Quantum-resistant Root Store (CQRS)	Phase 1 (2026)	Mock MTCA feasibility trial
CQRS Phase 2	Q1 2027 target	CT operators onboarded as MTCA beta
CQRS Phase 3	Q3 2027 target	CA onboarding; public MTC issuance begins

Independent review: Trail of Bits published an analysis in Q1 2026 highlighting key-compromise scenarios and batch-root lifecycle concerns. The working group is iterating on revocation (analogous to CRL/OCSP for traditional X.509) and root-key rotation procedures.

8.6.3 Relationship to Composite X.509

MTCs and composite X.509 (`draft-ietf-lamps-pq-composite-sigs`) are **complementary**, not competing:

- **Composite X.509** works inside the existing PKI with no new infrastructure. Best for internal/enterprise PKIs where size is not the blocker.
- **MTCs** require new infrastructure (MTCA operators, client integration) but solve the Web-PKI size problem. Best for Internet-scale public TLS.

A bridging strategy: issue composite certs for internal services and enterprise VPNs (where composite's 17–20 KB is affordable), MTCs for public web servers (where it isn't).

8.6.4 Looma — Lower-Latency Alternative for Datacenter TLS

A separate IETF draft, `draft-ma-cfrg-looma`, targets datacenter mTLS rather than public Web PKI. Looma pre-generates a batch of WOTS+ (stateful one-time) keys authenticated by a Merkle tree root under a long-term PQ signature. Each TLS handshake consumes one WOTS+ key; the root is authenticated once per deployment. Trade-off: stateful — losing the state counter reuses a WOTS+ key and breaks security — but per-handshake latency is lower than per-connection ML-DSA signing.

Use Looma where stateful management is tractable (single-tenant fleet with strong state management, e.g., Cloudflare-like mesh, Meta datacenter fabric) and avoid it where it isn't (consumer clients, distributed CA signing).

8.6.5 Signal SPQR — "Triple Ratchet" (October 2025)

Signal's **Sparse Post-Quantum Ratchet (SPQR)** complements the existing PQXDH handshake. Combined with Signal's Double Ratchet, the resulting three-component ratchet is marketed as the "Triple Ratchet":

- **Triple Ratchet**: Symmetric ratchet + Diffie-Hellman ratchet + **ML-KEM Braid Protocol** (the SPQR contribution).
- The SPQR "braid" replaces the classical DH ratchet's public-key contribution with an ML-KEM-based construction designed for fresh randomness per message while amortizing the expensive PQ operations across the ratchet state.
- Hybrid key derivation combines Double Ratchet output with SPQR output so compromise of either remains safe against the other.

SPQR was identified by Kobeissi's Verification Theatre analysis (see Unit 11.11) as one of the primary deployment sites of the libcrux bugs — the endianness bug in cross-backend ML-KEM code caused real decryption failures in early SPQR deployments. Fixes shipped in subsequent Signal client updates; the case is an object lesson in the risks of "formally verified" being less complete than marketing suggests.

Unit 8.6 Summary

The composite X.509 approach (Unit 8.4) was the obvious path, but handshake sizes of 17–20 KB hit real middlebox limits before rolling out to the public Web PKI. Merkle Tree Certificates take a different architectural approach — batch-sign the binding of domains to pubkeys, send short inclusion proofs instead of full cert chains — and bring handshake size back below 1 KB. The PLANTS WG, Chrome CQRS rollout, and Signal's SPQR ratchet are all applications of the same insight: PQ keys and signatures are too big to use in the obvious per-object pattern, so amortize them across batches. Expect significant protocol-level architectural change across 2026–2028 to accommodate this shift.

Module 8 Summary: Hybrid Cryptography

Module 8 provided comprehensive coverage of hybrid cryptography for the post-quantum transition:

Unit 8.1: Introduction to Hybrid Cryptography

- Motivation: harvest-now-decrypt-later attacks
- Combining classical and post-quantum algorithms
- Security analysis of hybrid constructions

Unit 8.2: Hybrid Key Exchange

- Concatenated and nested combiners
- KDF usage for shared secret derivation
- Protocol considerations (TLS, IKE)

Unit 8.3: Hybrid Signatures

- Parallel and nested signature schemes
- Verification policies and failure modes
- Certificate chain considerations

Unit 8.4: Composite Key Formats and Standards

- ASN.1/DER encoding for composite keys
- X.509 certificate extensions
- PKCS#8 private key format
- Interoperability with existing PKI

Unit 8.5: Implementation and Integration

- Library architecture patterns
- Thread safety and memory management
- Performance optimization techniques
- Migration strategies and deployment

What's Next: Module 9 covers Protocol Integration, examining how hybrid cryptography is being integrated into real-world protocols like TLS 1.3, SSH, and S/MIME, with detailed analysis of standardization efforts and implementation guidance.

Module 9 Summary: Protocol Integration

Module 9 covered post-quantum integration across major security protocols:

Unit 9.1: TLS 1.3

- Hybrid KEX with X25519+ML-KEM-768
- Size impact management
- Certificate chain optimization

Unit 9.2: SSH

- sntrup761x25519-sha512 default in OpenSSH 9.0+
- Host key rotation strategies
- Configuration best practices

Unit 9.3: S/MIME

- Long-term email confidentiality challenges
- Hybrid encryption for KEK protection
- Certificate size considerations

Unit 9.4: VPN/IPsec

- IKEv2 hybrid key exchange
- strongSwan configuration
- Tunnel rekeying considerations

Unit 9.5: Code Signing

- PKI migration planning
- Hybrid signature schemes
- Timestamp authority integration

Key Takeaways

- Key exchange is most urgent (harvest-now-decrypt-later)
- Hybrid approaches enable gradual transition
- Size increases require infrastructure planning
- Standardization ongoing - follow IETF drafts

Library Interoperability Matrix

> Last verified: April 2026 (v1.0). Library versions and feature flags decay on a 3-6 month cycle — verify against vendor release notes before relying on specific versions in this table.

When deploying post-quantum cryptography in production, library selection significantly impacts which algorithms are available and the maturity of implementations. The following matrix summarizes PQC algorithm support across major cryptographic libraries:

LIBRARY	ML-KEM	ML-DSA	SLH-DSA	X-WING	VERSION	STATUS
liboqs	✓	✓	✓	✗	0.14+	Reference/experimental
BoringSSL	✓	✓	✗	✓	Jan 2026	Production (FIPS module)
OpenSSL	✓	✓	✓	✓	3.5+ (LTS)	Provider-based
wolfSSL	✓	✓	✗	✗	5.8+	Production (FIPS pending)
AWS-LC	✓	✓	✗	✗	2.0+	Production (FIPS 140-3 certified)
BouncyCastle	✓	✓	✓	✗	2.0+	Java/C# production

Implementation Notes:

- **liboqs**: Open Quantum Safe project providing reference implementations. Excellent for prototyping and testing but not recommended as sole production dependency. Integrates with OpenSSL via oqs-provider.
- **BoringSSL**: Google's production library focused on TLS. Supports ML-KEM and ML-DSA (added January 2026 with X.509 and FIPS module integration). X25519+ML-KEM-768 hybrid is the recommended configuration for Chrome and Google services.

- **OpenSSL**: Version 3.5+ (LTS) provides native ML-KEM, ML-DSA, and SLH-DSA support with hybrid X25519+ML-KEM-768 key shares offered by default in TLS 1.3. Version 3.6 added CMS KEMRecipientInfo (RFC 9629) and additional hybrid groups (SecP256r1MLKEM768, SecP384r1MLKEM1024). Use oqs-provider for additional algorithms.
- **wolfSSL**: Embedded-focused library with ML-KEM and ML-DSA support. FIPS 140-3 validation in progress for PQC algorithms. Also supports HQC. Suitable for IoT and resource-constrained deployments.
- **AWS-LC**: Amazon's fork of BoringSSL/OpenSSL. AWS-LC-FIPS v2.0 is FIPS 140-3 certified (including ML-KEM). V3.0 with expanded PQC coverage is submitted for validation. ML-KEM is integrated into AWS KMS, ACM, and Secrets Manager.
- **BouncyCastle**: Mature Java and C# implementations suitable for enterprise applications. Provides both lightweight and JCA provider APIs.

Production Readiness Guidance:

MATURITY LEVEL	DESCRIPTION	LIBRARIES
Production	Deployed at scale, FIPS validation available or certified	BoringSSL, AWS-LC, wolfSSL
Production-ready	Stable API, suitable for production with testing	OpenSSL 3.5+, BouncyCastle
Experimental	API may change, suitable for development and testing	liboqs, research implementations

What's Next: Module 10 examines algorithms beyond the initial NIST selections, including FALCON/FN-DSA, code-based schemes (HQC, Classic McEliece), and emerging research directions. While Modules 4-9 focused on the three primary FIPS standards, Module 10 broadens the landscape to help you evaluate future algorithm additions and alternative approaches.

Module 10: Future Algorithms and Research

This module examines the evolving landscape of post-quantum cryptography. We explore algorithms under consideration for future standardization, active research directions, and strategies for maintaining cryptographic agility as the field advances.

Prerequisites: Modules 4-8 (ML-KEM, ML-DSA, SLH-DSA, Hybrid Cryptography)

Module Learning Objectives:

- Understand NIST's ongoing algorithm standardization process
- Evaluate FALCON (FN-DSA) and its trade-offs versus ML-DSA
- Analyze code-based and isogeny-based alternatives
- Design crypto-agile systems for algorithm transitions
- Integrate with liboqs and OpenSSL provider architecture
- Plan for future algorithm updates and deprecations

Unit 10.3: Cryptographic Agility

Prerequisites: Modules 4-9 (Complete PQC algorithm and protocol coverage)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

1. **Design systems for algorithm replacement** without major refactoring
2. **Implement negotiation mechanisms** for multiple algorithm support
3. **Plan graceful transitions** between cryptographic generations
4. **Test and validate** cryptographic agility implementations

10.3.1 Agility Architecture

```
/* crypto_agility.h - Cryptographic Agility Framework */

#ifndef CRYPTO_AGILITY_H
#define CRYPTO_AGILITY_H

#include <stdint.h>
#include <stddef.h>

/*
 * Cryptographic Agility Design Principles
 * =======================================
 *
 * 1. Algorithm Abstraction
 *    - Use algorithm identifiers, not hardcoded implementations
 *    - Provider-based architecture (like OpenSSL 3.0)
 *
 * 2. Negotiation Support
 *    - Support multiple algorithms simultaneously
 *    - Graceful fallback mechanisms
 *    - Version and capability advertisement
 *
 * 3. Configuration-Driven
 *    - Algorithm selection via configuration
 *    - No code changes for algorithm updates
 *    - Policy enforcement capability
 *
 * 4. Hybrid Support
 *    - Combine multiple algorithms easily
 *    - Flexible combiner functions
 *
 * 5. Future-Proofing
 *    - Extensible identifier space
 *    - Backward compatibility considerations
 *    - Deprecation workflow support
 */

/* Algorithm identifier structure */
typedef struct {
    uint16_t family;       /* Algorithm family (lattice, hash, code, etc.) */
    uint16_t algorithm;    /* Specific algorithm within family */
    uint16_t variant;      /* Parameter set / variant */
    uint16_t version;      /* Version for updates */
} algorithm_id_t;

/* Algorithm families */
#define ALG_FAMILY_CLASSICAL    0x0001
#define ALG_FAMILY_LATTICE      0x0002
#define ALG_FAMILY_HASH_BASED   0x0003
#define ALG_FAMILY_CODE_BASED   0x0004
#define ALG_FAMILY_ISOGENY      0x0005  /* Research only */
#define ALG_FAMILY_HYBRID       0x00FF

/* Cryptographic operation types */
typedef enum {
    CRYPTO_OP_SIGN,
    CRYPTO_OP_VERIFY,
    CRYPTO_OP_ENCRYPT,
```

```
    CRYPTO_OP_DECRYPT,
    CRYPTO_OP_KEYGEN,
    CRYPTO_OP_ENCAPS,
    CRYPTO_OP_DECAPS,
} crypto_operation_t;

/* Algorithm capability descriptor */
typedef struct {
    algorithm_id_t id;
    const char *name;
    uint32_t operations;     /* Bitmask of supported operations */
    size_t public_key_size;
    size_t private_key_size;
    size_t signature_size;   /* For signature algorithms */
    size_t ciphertext_size;  /* For KEMs */
    int security_level;      /* NIST level 1-5 */
    int is_deprecated;
    int is_experimental;
} algorithm_capability_t;

/* Provider interface */
typedef struct crypto_provider {
    const char *name;
    int (*init)(void);
    void (*cleanup)(void);
    int (*supports)(algorithm_id_t id);
    const algorithm_capability_t *(*get_capability)(algorithm_id_t id);

    /* Generic operations */
    int (*keygen)(algorithm_id_t id, uint8_t *pk, size_t *pk_len,
                  uint8_t *sk, size_t *sk_len);
    int (*sign)(algorithm_id_t id, uint8_t *sig, size_t *sig_len,
                const uint8_t *msg, size_t msg_len,
                const uint8_t *sk, size_t sk_len);
    int (*verify)(algorithm_id_t id,
                  const uint8_t *sig, size_t sig_len,
                  const uint8_t *msg, size_t msg_len,
                  const uint8_t *pk, size_t pk_len);
    int (*encaps)(algorithm_id_t id, uint8_t *ct, size_t *ct_len,
                  uint8_t *ss, size_t *ss_len,
                  const uint8_t *pk, size_t pk_len);
    int (*decaps)(algorithm_id_t id, uint8_t *ss, size_t *ss_len,
                  const uint8_t *ct, size_t ct_len,
                  const uint8_t *sk, size_t sk_len);

    struct crypto_provider *next;  /* Linked list */
} crypto_provider_t;

/* Global agility context */
typedef struct {
    crypto_provider_t *providers;
    algorithm_id_t *preferred_signature;
    size_t num_preferred_sig;
    algorithm_id_t *preferred_kem;
    size_t num_preferred_kem;
    int allow_deprecated;
    int allow_experimental;
} agility_context_t;

/* Function prototypes */
```

```
int agility_init(agility_context_t *ctx);
int agility_register_provider(agility_context_t *ctx, crypto_provider_t *provider);
int agility_set_preference(agility_context_t *ctx, crypto_operation_t op,
                           const algorithm_id_t *algorithms, size_t count);
algorithm_id_t agility_negotiate(agility_context_t *ctx, crypto_operation_t op,
                                 const algorithm_id_t *peer_supported, size_t count);
int agility_perform(agility_context_t *ctx, algorithm_id_t id,
                    crypto_operation_t op, void *params);
void agility_cleanup(agility_context_t *ctx);

#endif /* CRYPTO_AGILITY_H */
```

10.3.2 Implementation

```
/* crypto_agility.c - Implementation */

#include "crypto_agility.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>

int agility_init(agility_context_t *ctx) {
    if (!ctx) return -1;

    memset(ctx, 0, sizeof(agility_context_t));
    return 0;
}

int agility_register_provider(agility_context_t *ctx, crypto_provider_t *provider) {
    if (!ctx || !provider) return -1;

    /* Initialize provider */
    if (provider->init && provider->init() != 0) {
        return -1;
    }

    /* Add to linked list */
    provider->next = ctx->providers;
    ctx->providers = provider;

    return 0;
}

/* Find provider that supports given algorithm */
static crypto_provider_t *find_provider(agility_context_t *ctx, algorithm_id_t id) {
    crypto_provider_t *p = ctx->providers;

    while (p) {
        if (p->supports && p->supports(id)) {
            return p;
        }
        p = p->next;
    }

    return NULL;
}

int agility_set_preference(agility_context_t *ctx, crypto_operation_t op,
```

```
                              const algorithm_id_t *algorithms, size_t count) {
    if (!ctx || !algorithms || count == 0) return -1;

    algorithm_id_t **pref;
    size_t *pref_count;

    switch (op) {
        case CRYPTO_OP_SIGN:
        case CRYPTO_OP_VERIFY:
            pref = &ctx->preferred_signature;
            pref_count = &ctx->num_preferred_sig;
            break;
        case CRYPTO_OP_ENCAPS:
        case CRYPTO_OP_DECAPS:
            pref = &ctx->preferred_kem;
            pref_count = &ctx->num_preferred_kem;
            break;
        default:
            return -1;
    }

    /* Free existing preferences */
    free(*pref);

    /* Copy new preferences */
    *pref = malloc(count * sizeof(algorithm_id_t));
    if (!*pref) return -1;

    memcpy(*pref, algorithms, count * sizeof(algorithm_id_t));
    *pref_count = count;

    return 0;
}

/* Negotiate best mutually supported algorithm */
algorithm_id_t agility_negotiate(agility_context_t *ctx, crypto_operation_t op,
                                 const algorithm_id_t *peer_supported, size_t count) {
    algorithm_id_t result = {0, 0, 0, 0};

    if (!ctx || !peer_supported || count == 0) {
        return result;
    }

    algorithm_id_t *our_pref;
    size_t our_count;

    switch (op) {
        case CRYPTO_OP_SIGN:
        case CRYPTO_OP_VERIFY:
            our_pref = ctx->preferred_signature;
            our_count = ctx->num_preferred_sig;
            break;
        case CRYPTO_OP_ENCAPS:
        case CRYPTO_OP_DECAPS:
            our_pref = ctx->preferred_kem;
            our_count = ctx->num_preferred_kem;
            break;
        default:
            return result;
    }
```

```
    /* Our preference order */
    for (size_t i = 0; i < our_count; i++) {
        algorithm_id_t candidate = our_pref[i];

        /* Check if we have a provider */
        crypto_provider_t *provider = find_provider(ctx, candidate);
        if (!provider) continue;

        /* Check capability */
        const algorithm_capability_t *cap = provider->get_capability(candidate);
        if (!cap) continue;

        if (cap->is_deprecated && !ctx->allow_deprecated) continue;
        if (cap->is_experimental && !ctx->allow_experimental) continue;

        /* Check if peer supports */
        for (size_t j = 0; j < count; j++) {
            if (memcmp(&candidate, &peer_supported[j], sizeof(algorithm_id_t)) == 0) {
                return candidate;  /* Found match */
            }
        }
    }

    return result;  /* No match found */
}

/* Perform cryptographic operation */
int agility_perform(agility_context_t *ctx, algorithm_id_t id,
                    crypto_operation_t op, void *params) {
    if (!ctx || !params) return -1;

    crypto_provider_t *provider = find_provider(ctx, id);
    if (!provider) {
        return -1;  /* No provider for algorithm */
    }

    /* Check capability and policy */
    const algorithm_capability_t *cap = provider->get_capability(id);
    if (cap) {
        if (cap->is_deprecated && !ctx->allow_deprecated) {
            fprintf(stderr, "Algorithm %s is deprecated\n", cap->name);
            return -1;
        }
    }

    /* Dispatch to provider */
    /* In real implementation, params would be a union/struct with operation-specific data
*/

    return 0;  /* Success */
}

void agility_cleanup(agility_context_t *ctx) {
    if (!ctx) return;

    /* Cleanup providers */
    crypto_provider_t *p = ctx->providers;
    while (p) {
        if (p->cleanup) p->cleanup();
```

```
        p = p->next;
    }

    free(ctx->preferred_signature);
    free(ctx->preferred_kem);
    memset(ctx, 0, sizeof(agility_context_t));
}
```

10.3.3 Configuration-Driven Agility

```
/* agility_config.c - Configuration-Based Algorithm Selection */

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/*
 * Example configuration file format (YAML-like):
 *
 * crypto:
 *   signature:
 *     preferred:
 *       - ml-dsa-65
 *       - ed25519-ml-dsa-65  # hybrid
 *       - ed25519            # fallback
 *     deprecated:
 *       - rsa-2048
 *     forbidden:
 *       - sha1-rsa
 *
 *   kem:
 *     preferred:
 *       - ml-kem-768
 *       - x25519-ml-kem-768  # hybrid
 *       - x25519             # fallback
 *
 *   policy:
 *     allow_classical_only: false
 *     require_pq: true
 *     hybrid_mode: preferred
 */

typedef struct {
    char **signature_preferred;
    size_t num_sig_preferred;
    char **signature_deprecated;
    size_t num_sig_deprecated;
    char **signature_forbidden;
    size_t num_sig_forbidden;

    char **kem_preferred;
    size_t num_kem_preferred;

    int allow_classical_only;
    int require_pq;
    int hybrid_mode;  /* 0=disabled, 1=allowed, 2=preferred, 3=required */
} crypto_config_t;
```

```
/* Parse configuration from string (simplified) */
int parse_crypto_config(const char *config_str, crypto_config_t *config) {
    /* In real implementation, parse YAML/JSON/TOML */
    /* This is a simplified example */

    memset(config, 0, sizeof(crypto_config_t));

    /* Set defaults for post-quantum readiness */
    config->hybrid_mode = 2;  /* Preferred */
    config->require_pq = 1;
    config->allow_classical_only = 0;

    /* Example: hardcoded defaults */
    config->num_sig_preferred = 3;
    config->signature_preferred = malloc(3 * sizeof(char *));
    config->signature_preferred[0] = strdup("ml-dsa-65");
    config->signature_preferred[1] = strdup("ed25519-ml-dsa-65");
    config->signature_preferred[2] = strdup("ed25519");

    config->num_kem_preferred = 3;
    config->kem_preferred = malloc(3 * sizeof(char *));
    config->kem_preferred[0] = strdup("ml-kem-768");
    config->kem_preferred[1] = strdup("x25519-ml-kem-768");
    config->kem_preferred[2] = strdup("x25519");

    return 0;
}

/* Check if algorithm is allowed by policy */
int is_algorithm_allowed(const crypto_config_t *config, const char *algorithm) {
    /* Check forbidden list */
    for (size_t i = 0; i < config->num_sig_forbidden; i++) {
        if (strcmp(config->signature_forbidden[i], algorithm) == 0) {
            return 0;  /* Forbidden */
        }
    }

    /* Check PQ requirement */
    if (config->require_pq) {
        /* Check if algorithm contains PQ component */
        if (strstr(algorithm, "ml-") == NULL &&
            strstr(algorithm, "slh-") == NULL &&
            strstr(algorithm, "falcon") == NULL) {
            /* No PQ component - check if hybrid mode allows */
            if (config->hybrid_mode < 1) {
                return 0;  /* Classical not allowed */
            }
        }
    }

    return 1;  /* Allowed */
}

/* Print current configuration */
void print_crypto_config(const crypto_config_t *config) {
    printf("Cryptographic Configuration\n");
    printf("===========================\n\n");

    printf("Signature Algorithms (preference order):\n");
    for (size_t i = 0; i < config->num_sig_preferred; i++) {
```

```
        printf("  %zu. %s\n", i + 1, config->signature_preferred[i]);
    }

    printf("\nKEM Algorithms (preference order):\n");
    for (size_t i = 0; i < config->num_kem_preferred; i++) {
        printf("  %zu. %s\n", i + 1, config->kem_preferred[i]);
    }

    printf("\nPolicy:\n");
    printf("  Require PQ: %s\n", config->require_pq ? "yes" : "no");
    printf("  Allow classical only: %s\n", config->allow_classical_only ? "yes" : "no");
    printf("  Hybrid mode: %d\n", config->hybrid_mode);
}

void free_crypto_config(crypto_config_t *config) {
    for (size_t i = 0; i < config->num_sig_preferred; i++) {
        free(config->signature_preferred[i]);
    }
    free(config->signature_preferred);

    for (size_t i = 0; i < config->num_sig_deprecated; i++) {
        free(config->signature_deprecated[i]);
    }
    free(config->signature_deprecated);

    for (size_t i = 0; i < config->num_kem_preferred; i++) {
        free(config->kem_preferred[i]);
    }
    free(config->kem_preferred);

    memset(config, 0, sizeof(crypto_config_t));
}
```

10.3.4 Exercises

Exercise 10.3.1: Agility Provider Implementation *(Advanced)*

Implement a mock provider that demonstrates the agility interface:

```
/* Exercise: Implement a liboqs provider for the agility framework */
/* Fill in the TODO sections */

crypto_provider_t *create_liboqs_provider(void) {
    /* TODO: Allocate and initialize provider structure */
    /* TODO: Register all supported OQS algorithms */
    /* TODO: Implement keygen, sign, verify, encaps, decaps */
    return NULL;
}
```

Solution:

```
/* liboqs agility provider */
#include <oqs/oqs.h>

static int oqs_init(void) {
    /* liboqs initializes automatically */
    return OQS_SUCCESS == OQS_init() ? 0 : -1;
}

static void oqs_cleanup(void) {
    /* Cleanup is automatic with liboqs */
}

static int oqs_supports(algorithm_id_t id) {
    if (id.family != ALG_FAMILY_LATTICE &&
        id.family != ALG_FAMILY_HASH_BASED) {
        return 0;
    }
    /* Check if algorithm is enabled in liboqs build */
    if (id.algorithm == 0x0001) {  /* ML-KEM */
        return OQS_KEM_alg_is_enabled(OQS_KEM_alg_ml_kem_768);
    }
    if (id.algorithm == 0x0002) {  /* ML-DSA */
        return OQS_SIG_alg_is_enabled(OQS_SIG_alg_ml_dsa_65);
    }
    return 0;
}

static int oqs_keygen(algorithm_id_t id, uint8_t *pk, size_t *pk_len,
                      uint8_t *sk, size_t *sk_len) {
    if (id.family == ALG_FAMILY_LATTICE && id.algorithm == 0x0001) {
        OQS_KEM *kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
        if (!kem) return -1;
        int ret = OQS_KEM_keypair(kem, pk, sk);
        *pk_len = kem->length_public_key;
        *sk_len = kem->length_secret_key;
        OQS_KEM_free(kem);
        return ret == OQS_SUCCESS ? 0 : -1;
    }
    return -1;
}

crypto_provider_t *create_liboqs_provider(void) {
    crypto_provider_t *p = calloc(1, sizeof(crypto_provider_t));
    if (!p) return NULL;

    p->name = "liboqs";
    p->init = oqs_init;
    p->cleanup = oqs_cleanup;
    p->supports = oqs_supports;
    p->keygen = oqs_keygen;
    /* ... additional function pointers ... */

    return p;
}
```

Exercise 10.3.2: Algorithm Negotiation Protocol *(Advanced)*

Design a negotiation protocol for two parties to agree on PQC algorithms:

Solution:

```
/* Algorithm negotiation protocol */
typedef struct {
    algorithm_id_t *supported;     /* Algorithms we support */
    size_t num_supported;
    algorithm_id_t *preferred;     /* Our preference order */
    size_t num_preferred;
} negotiation_offer_t;

typedef struct {
    algorithm_id_t selected_kem;
    algorithm_id_t selected_sig;
    int success;
} negotiation_result_t;

/*
 * Negotiation Protocol:
 * 1. Initiator sends offer with supported + preferred algorithms
 * 2. Responder intersects with own capabilities
 * 3. Responder selects best match (highest in initiator's preference
 *    that responder supports)
 * 4. Responder sends selection back
 */

negotiation_result_t negotiate_algorithms(
    const negotiation_offer_t *our_offer,
    const negotiation_offer_t *their_offer)
{
    negotiation_result_t result = {0};

    /* Find first algorithm in their preference that we support */
    for (size_t i = 0; i < their_offer->num_preferred; i++) {
        algorithm_id_t candidate = their_offer->preferred[i];

        /* Check if we support it */
        for (size_t j = 0; j < our_offer->num_supported; j++) {
            if (memcmp(&candidate, &our_offer->supported[j],
                       sizeof(algorithm_id_t)) == 0) {
                /* Found match - determine if KEM or signature */
                if (is_kem_algorithm(candidate)) {
                    result.selected_kem = candidate;
                } else {
                    result.selected_sig = candidate;
                }
                break;
            }
        }
    }

    result.success = (result.selected_kem.algorithm != 0 &&
                      result.selected_sig.algorithm != 0);
    return result;
}
```

Exercise 10.3.3: Deprecation Workflow *(Intermediate)*

Implement a deprecation tracking system with logging:

Solution:

```
/* Deprecation tracking and warning system */
#include <time.h>

typedef struct {
    algorithm_id_t id;
    time_t deprecated_date;
    time_t sunset_date;        /* After this, refuse to use */
    const char *reason;
    int warning_issued;
} deprecation_entry_t;

static deprecation_entry_t deprecation_table[] = {
    {{ALG_FAMILY_CLASSICAL, 0x0001, 0, 0},  /* RSA-2048 */
     1704067200,  /* Jan 1, 2024 */
     1767225600,  /* Jan 1, 2026 */
     "Quantum vulnerable, migrate to ML-DSA", 0},

    {{ALG_FAMILY_CLASSICAL, 0x0002, 0, 0},  /* ECDSA P-256 */
     1704067200,
     1798761600,  /* Jan 1, 2027 */
     "Quantum vulnerable, migrate to ML-DSA or hybrid", 0},
};

int check_algorithm_status(algorithm_id_t id) {
    time_t now = time(NULL);

    for (size_t i = 0; i < sizeof(deprecation_table)/sizeof(deprecation_table[0]); i++) {
        if (memcmp(&id, &deprecation_table[i].id, sizeof(algorithm_id_t)) == 0) {
            if (now >= deprecation_table[i].sunset_date) {
                /* Algorithm is sunset - refuse to use */
                fprintf(stderr, "ERROR: Algorithm has been sunset: %s\n",
                        deprecation_table[i].reason);
                return -1;  /* Refuse */
            }
            if (now >= deprecation_table[i].deprecated_date &&
                !deprecation_table[i].warning_issued) {
                /* Log warning once */
                fprintf(stderr, "WARNING: Deprecated algorithm in use: %s\n",
                        deprecation_table[i].reason);
                deprecation_table[i].warning_issued = 1;
            }
            return 0;  /* Allow with warning */
        }
    }
    return 0;  /* Not in deprecation table, OK */
}
```

Unit 10.3 Summary

Cryptographic agility requires:

1. **Abstraction**: Separate algorithm selection from implementation
2. **Negotiation**: Support multiple algorithms with graceful fallback
3. **Configuration**: External policy without code changes

4. **Monitoring**: Track algorithm usage and deprecation

What's Next: Unit 10.4 provides a comprehensive implementation best practices summary.

Unit 10.5: NIST Additional Signatures — Round 2 Status

After finalizing ML-DSA (FIPS 204) and SLH-DSA (FIPS 205), NIST opened a separate standardization track for **additional digital signature algorithms**. The goal is algorithmic diversity: different mathematical assumptions, different performance profiles, different signature-versus-pubkey trade-offs. Round 1 closed October 2023; Round 2 was announced October 2024 and continues through 2026.

10.5.1 Round 2 Candidates (14)

As of April 2026, the 14 Round 2 finalists are:

CANDIDATE	FAMILY	NOTABLE FEATURE
CROSS	Code-based (restricted errors)	Fast signing; moderate signature size
FAEST	Symmetric-key (VOLE-in-the-head)	Very small keys; signatures ~5 KB
HAWK	Lattice (module-LIP)	**Last lattice method standing** in Round 2 — Falcon-class sizes, simpler sampling than FN-DSA. Based on Lattice Isomorphism Problem, not NTRU.
LESS	Code-based (permutation equivalence)	Small public keys
MAYO	Multivariate (UOV variant)	Very fast verify; small sigs
Mirath	Multivariate (merger of MIRA + MiRitH)	Consolidated MPCitH-MinRank candidate
MQOM	Multivariate (MQ-over-any-field)	General MQ framework
PERK	Symmetric (permuted kernel)	Zero-knowledge style
QR-UOV	Multivariate (UOV over quotient ring)	Smaller pubkey than standard UOV
RYDE	Code-based (rank metric)	Rank-syndrome decoding
SDitH	Code-based (syndrome-decoding in-the-head)	MPCitH framework
SNOVA	Multivariate	Compact UOV variant
SQIsign	Isogeny	Smallest PQ signatures (~200 bytes); slowest signing
UOV	Multivariate (original Oil-and-Vinegar)	The "baseline" MQ candidate

Key dynamics:

- **Lattice diversity is deliberately constrained** — NIST did not want a second lattice primary to avoid single-family risk. HAWK is the sole remaining lattice option and would be adopted only if all other candidates fail.
- **Multivariate and code-based have the most candidates** (5 and 4 respectively), reflecting the community's desire for non-lattice alternatives.
- **SQIsign** is the only isogeny-based candidate surviving after SIDH's break in 2022 — signatures are tiny (~200 bytes) but signing is orders of magnitude slower than ML-DSA.
- **Mirath** consolidates two related candidates (MIRA and MiRitH) into one submission — a healthy sign of cross-team collaboration.

10.5.2 Timeline

MILESTONE	EXPECTED
Round 3 down-select	Expected 2026 (timing TBD by NIST)
Draft standards	2026–2027
Final publication (FIPS 208+?)	2027 or later

10.5.3 FIPS 207 — HQC as Backup KEM

Separately from the signatures track, NIST is also standardizing a **backup KEM** to complement ML-KEM (FIPS 203). **HQC (Hamming Quasi-Cyclic)** was selected March 2025; NIST's plan is:

- **FIPS 207 IPD** targeted for 2026 (draft for public comment)
- **FIPS 207 Final** targeted for 2027

HQC is code-based (its security reduces to decoding random quasi-cyclic codes), providing a fundamentally different hardness assumption from ML-KEM's structured lattices. However — as documented extensively in Unit 11.10 — HQC's reference implementation has a disproportionate share of implementation bugs (three CVEs in 18 months, disabled by default in liboqs 0.13.0 pending further review). Production use as of April 2026 is not recommended until the HQC team completes the next spec revision and libraries re-enable the algorithm.

Unit 10.5 Summary

The NIST Additional Signatures process is where algorithmic diversity lives. The Round 2 finalists span lattice (HAWK), multivariate (UOV, MAYO, SNOVA, QR-UOV, Mirath), code-based (CROSS, LESS, RYDE, SDitH, MQOM), isogeny (SQIsign), and symmetric-key (FAEST, PERK) families — giving the post-FIPS-204 standards ecosystem a much richer menu of options with different size/speed/confidence trade-offs. None of these will replace ML-DSA for general use; they exist to backstop ML-DSA if a cryptanalytic surprise breaks Module-LWE/SIS assumptions. Parallel to signatures, FIPS 207 will standardize HQC as the backup KEM, though implementation maturity needs to catch up before adoption.

Unit 10.6: Post-Quantum Threshold Cryptography

Threshold cryptography splits a secret key among n parties such that any $t \leq n$ subset can compute signatures or decapsulate ciphertexts, but $t - 1$ or fewer cannot. The combination with PQC is a significant challenge — lattice operations don't decompose as cleanly as finite-field operations — and is the subject of the NIST Multi-Party Threshold Cryptography (MPTS) workshop series.

10.6.1 Why This Matters

Threshold PQC addresses scenarios where:

- **Single-point-of-compromise mitigation**: a stolen HSM should not reveal the signing key.
- **Distributed trust across organizations**: multi-signature for blockchain consensus, cross-jurisdictional signatures.
- **Policy enforcement without a single custodian**: "any 3 of 5 board members can sign" without a single party holding the key.

Classical threshold RSA (Shoup, 2000) and threshold ECDSA (Lindell 2017, Doerner et al. 2019) are production-ready. Post-quantum threshold was largely theoretical until 2024–2026.

10.6.2 MPTS 2026 Candidates

NIST's MPTS 2026 workshop (January 2026) featured four significant PQ threshold constructions:

CONSTRUCTION	BASE ALGORITHM	APPROACH
Hermine	Raccoon (Round 2 additional sig candidate, withdrawn)	PQ threshold signatures using lattice-Schnorr design
Vinaigrette	UOV / MAYO	Threshold multivariate signatures — adds secret-sharing to UOV key structure
Quorus	ML-DSA (FIPS 204)	MPC-based threshold ML-DSA with **FIPS 204 signature compatibility** — verifiers don't need to be aware of the threshold structure
PiVer	Verifiable secret sharing	General-purpose building block for threshold PQ applications

Quorus is the most operationally significant: it produces a standard ML-DSA signature that any FIPS 204 verifier accepts, so threshold issuance can roll out without client-side changes. The cost is the MPC round complexity during signing.

10.6.3 Open Problems

- **Threshold decapsulation for ML-KEM** is harder than threshold signing — no NIST-blessed construction as of April 2026. Work in progress.
- **Performance** — PQ threshold signatures cost multiple MPC rounds per signature; latency is 10–100× classical threshold.
- **Non-interactive variants** — research goal; most current constructions require online coordination.
- **Formal security proofs** — compositional security (threshold-of-PQ vs. PQ-of-threshold) is an active research frontier.

10.6.4 Production Status

Threshold PQC is **not production-ready** as of April 2026. Track:

- NIST MPTS workshop series (annual)
- IACR ePrint archive, especially the Crypto / Eurocrypt / Asiacrypt conference submissions
- The Quorus and Hermine open-source implementations if/when they appear

For production threshold-key use cases today, consider:

- Classical threshold ECDSA for non-HNDL-sensitive workflows (you'll migrate later)
- **PQ-capable HSM with physical multi-party controls** (m-of-n administrative controls) as an interim substitute for cryptographic threshold
- Hybrid constructions where the PQ half is held by a single party and the classical half is thresholded — partial protection but simpler

Unit 10.6 Summary

Threshold PQC is where the research frontier is hottest right now. The Quorus construction is the most operationally interesting because it produces standard FIPS 204 signatures from a distributed signing protocol — making rollout tractable. However, nothing is production-ready as of April 2026, and the performance profiles (multi-round MPC per signature) constrain the use cases. Track the NIST MPTS workshop series for updates; expect first production-ready threshold PQ signing implementations in 2027–2028.

Module 10 Summary: Future Algorithms and Research

Module 10 explored the evolving PQC landscape:

Unit 10.1: NIST Additional Algorithms

- FALCON for smaller signatures
- Alternative KEMs (HQC selected, BIKE, McEliece)
- Algorithm selection framework

Unit 10.2: Emerging Research

- Post-SIDH isogeny research
- Alternative cryptographic approaches
- Quantum computing timeline estimates

Unit 10.3: Cryptographic Agility

- Provider-based architecture
- Configuration-driven selection
- Graceful transition strategies

Unit 10.4: Best Practices Summary

- Security implementation checklist
- Common pitfall avoidance
- Comprehensive testing strategies

Key Takeaways

- Start PQ transition immediately
- Use hybrid constructions during transition
- Build for cryptographic agility
- Monitor evolving standards and threats
- Test thoroughly before deployment

Module 11: Migrating Legacy Codebases to Post-Quantum Cryptography

Module Overview: This module provides a comprehensive, practical guide to migrating existing systems and codebases from classical cryptography to post-quantum cryptography. Based on real-world case studies, official guidance from NIST, CISA, and NSA, and industry best practices, this module covers the complete migration lifecycle from inventory to deployment.

Prerequisites: Modules 1-10 (complete PQC foundation)

Module Learning Objectives:

- Conduct comprehensive cryptographic inventory and discovery
- Assess and prioritize migration risks using the HNDL threat model
- Design crypto-agile architectures for seamless algorithm transitions
- Implement hybrid cryptography as a migration bridge
- Select and integrate appropriate PQC libraries
- Test and validate cryptographic implementations
- Plan phased migration aligned with compliance deadlines

Unit 11.1: Understanding the Migration Imperative

Prerequisites: Modules 1-10 (complete PQC foundation)

Estimated Time: 1-2 hours

Learning Objectives

After completing this unit, you will be able to:

1. **Explain the urgency** of PQC migration and the "harvest now, decrypt later" threat
2. **Understand global compliance timelines** from NIST, NSA, UK NCSC, and EU

3. **Calculate organizational risk** using the Y + Z > X framework
4. **Prioritize migration efforts** based on data sensitivity and exposure

11.1.1 The "Harvest Now, Decrypt Later" Threat

The most pressing reason for immediate PQC migration is not a future quantum computer—it's attackers storing encrypted data today:

```
HNDL (Harvest Now, Decrypt Later) Attack Model
==============================================

2024: Adversary intercepts and stores encrypted traffic
      ├── TLS sessions
      ├── VPN tunnels
      ├── Encrypted emails
      └── Stored encrypted databases

2030-2040: Cryptographically relevant quantum computer (CRQC) exists
           └── Adversary decrypts all stored data using Shor's algorithm

Affected Data:
- Government/military communications
- Healthcare records (HIPAA - 50+ year retention)
- Financial transactions
- Trade secrets and intellectual property
- Personal communications
```

The Mathematical Reality:

Assess your exposure using these three variables:

VARIABLE	DEFINITION	TYPICAL VALUES
X	Years until CRQC exists	5-15 years (estimates vary)
Y	Years needed to complete PQC migration	3-10 years
Z	Years data must remain confidential	5-50+ years

Critical Formula: If **Y + Z > X**, your organization is already at risk.

```
// Migration risk assessment
typedef struct {
    int years_to_crqc;           // X: Conservative estimate
    int migration_duration;       // Y: Based on system complexity
    int data_confidentiality;     // Z: Regulatory/business requirements
} risk_assessment_t;

bool is_at_risk(risk_assessment_t *assessment) {
    return (assessment->migration_duration +
            assessment->data_confidentiality) >
            assessment->years_to_crqc;
}
```

```
// Example: Healthcare organization
// X = 10 years (conservative CRQC estimate)
// Y = 5 years (complex legacy systems)
// Z = 50 years (HIPAA requirements)
// Y + Z = 55 > 10 = AT RISK (need to start migration NOW)
```

11.1.2 Global Compliance Timelines

```
Global PQC Migration Deadlines
==============================

United States (NIST IR 8547):
├── 2030: Deprecation of RSA, ECDSA, DH, ECDH
└── 2035: Complete disallowance

NSA CNSA 2.0 (National Security Systems):
├── 2025: Begin software/firmware signing transition
├── 2030: Most systems complete transition
└── 2033: Full compliance required

United Kingdom (NCSC):
├── 2028: Complete inventory and migration planning
├── 2031: High-priority upgrades complete
└── 2035: Full PQC migration

European Union:
├── 2026: National transition roadmaps published
├── 2030: High-risk systems migrated
└── 2035: Full transition complete

Australia:
└── 2030: Aggressive migration deadline

Canada:
└── 2035: Non-classified IT systems migrated

US Government Cost Estimate:
└── $7.1 billion (2024 dollars) for federal migration 2025-2035
```

11.1.3 Migration Priority Framework

Priority 1: Highest Risk (Migrate Immediately)

- Long-term secrets (encryption keys, master secrets)
- Government and military communications
- Healthcare records (HIPAA-protected data)
- Financial services data
- Intellectual property with 10+ year value
- Root CA certificates and signing keys

Priority 2: High Risk (Migrate 2025-2028)

- Critical infrastructure systems
- Telecommunications backbone
- Energy and utilities SCADA
- Inter-bank settlement systems

Priority 3: Medium Risk (Migrate 2028-2032)

- General enterprise applications
- E-commerce platforms
- Customer databases
- Internal communications

Priority 4: Lower Risk (Migrate 2032-2035)

- Data with <5 year confidentiality needs
- Ephemeral session data
- Public information systems

Key Insight: Key encapsulation (KEMs) should be migrated before signatures because:

- KEMs protect against HNDL attacks (data can be stored)
- Signature attacks require real-time quantum access
- Exception: Long-lived certificates (root CAs) need signature migration early

11.1.4 Migration Cost Analysis Breakdown

The US federal government's $7.1 billion estimate (2024 dollars, 2025-2035) provides a baseline for understanding PQC migration costs. The following breakdown helps organizations estimate their own costs:

Cost by Organization Size:

ORGANIZATION SIZE	SYSTEMS	ESTIMATED TOTAL COST	PER-SYSTEM AVERAGE
Small (<50 systems)	10-50	$500K - $2M	$20K-$40K
Medium (50-500 systems)	50-500	$2M - $15M	$30K-$50K
Large (500-2000 systems)	500-2000	$15M - $75M	$30K-$50K
Enterprise (2000+ systems)	2000+	$75M - $300M+	$35K-$60K

Cost by Migration Phase:

PHASE	% OF TOTAL BUDGET	KEY ACTIVITIES
Discovery & Planning	10-15%	Inventory, risk assessment, architecture
Pilot & Validation	15-20%	Lab setup, testing, training
High-Risk Migration	25-30%	PKI, internet-facing, long-term keys
Broad Migration	25-35%	Internal apps, databases, integrations
Classical Deprecation	10-15%	Final validation, decommissioning

Per-Algorithm Implementation Costs:

ALGORITHM	LIBRARY INTEGRATION	TESTING & VALIDATION	PKI/ CERTIFICATE UPDATES	TOTAL PER APPLICATION
ML-KEM (TLS)	$5K-$15K	$3K-$8K	$2K-$5K	$10K-$28K
ML-DSA (Signing)	$8K-$20K	$5K-$12K	$5K-$15K	$18K-$47K
SLH-DSA (Firmware)	$10K-$25K	$8K-$15K	$3K-$8K	$21K-$48K
Hybrid (KEM + Classical)	$12K-$30K	$8K-$18K	$5K-$12K	$25K-$60K

Federal Estimate Breakdown ($7.1B):

```
US Federal PQC Migration Cost Allocation (Estimated)
=====================================================

Personnel & Training:        $1.4B (20%)
├── Cryptographic engineers
├── Security analysts
├── Developer training
└── Contractor support

Software Development:        $2.1B (30%)
├── Application modifications
├── Library integration
├── Testing infrastructure
└── Automation tooling

Infrastructure:              $1.8B (25%)
├── HSM upgrades (PQC-capable)
├── Certificate infrastructure
├── Network equipment updates
└── Cloud service migrations

Compliance & Validation:     $1.1B (15%)
├── FIPS 140-3 validations
├── Security assessments
├── Penetration testing
└── Audit documentation

Contingency & Risk:          $0.7B (10%)
├── Unforeseen complexity
├── Algorithm changes
├── Extended timelines
└── Emergency responses
```

Cost Reduction Strategies:

1. **Crypto-Agility Investment**: Building agile architecture upfront reduces future migration costs by 40-60%
2. **Shared Services**: Centralized crypto services reduce per-application costs
3. **Automation**: Automated inventory and testing tools lower discovery costs by 50%+
4. **Phased Approach**: Spreading costs over 5-10 years improves budget absorption

Unit 11.1 Summary

This unit covered:

- The "Harvest Now, Decrypt Later" (HNDL) threat model and why PQC migration is urgent even before quantum computers exist
- The Y + Z > X risk assessment framework for calculating organizational exposure based on migration timeline, data confidentiality requirements, and estimated time to CRQC

- Global compliance timelines from NIST, NSA CNSA 2.0, UK NCSC, and the EU, with key deadlines between 2030 and 2035
- A four-tier migration priority framework ranking systems by data sensitivity and cryptographic exposure
- Migration cost analysis including per-algorithm implementation costs and the US federal government's $7.1 billion estimate

What's Next: Unit 11.2 introduces cryptographic inventory and discovery techniques, teaching you how to find and document every instance of cryptography across your organization's systems.

Unit 11.2: Cryptographic Inventory and Discovery

Prerequisites: Unit 11.1 (Migration Imperative)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

1. **Conduct a comprehensive cryptographic inventory** across all systems
2. **Use automated discovery tools** to identify cryptographic usage
3. **Document cryptographic dependencies** in applications and protocols
4. **Create a cryptographic bill of materials (CBOM)**

11.2.1 Why Inventory Is Critical

Most organizations have no idea where cryptography is used in their systems:

```
Typical Enterprise Cryptographic Surface
========================================

Application Layer:
├── Web applications (TLS certificates)
├── Mobile apps (certificate pinning, local encryption)
├── Desktop applications (code signing, DRM)
└── API integrations (JWT, OAuth tokens)

Protocol Layer:
├── TLS 1.2/1.3 (ECDHE, RSA key exchange)
├── SSH (host keys, user authentication)
├── IPsec/VPN (IKEv2, certificates)
├── S/MIME (email encryption/signing)
└── SFTP, FTPS, HTTPS

Infrastructure Layer:
```

```
├── PKI/CA infrastructure
├── Hardware Security Modules (HSMs)
├── Key Management Systems (KMS)
├── Certificate stores
└── Secrets management (Vault, AWS Secrets Manager)

Data Layer:
├── Database encryption (TDE)
├── File system encryption
├── Backup encryption
├── Archive encryption
└── Cloud storage encryption

Identity Layer:
├── Authentication systems (SAML, OIDC)
├── Smart cards and tokens
├── Code signing certificates
└── Document signing

Third-Party Dependencies:
├── Vendor integrations
├── Cloud services
├── SaaS applications
└── Supply chain software
```

11.2.2 Inventory Methodology

Phase 1: Automated Discovery

```
# Example: Scanning for TLS certificates in a codebase
grep -r "BEGIN CERTIFICATE" /path/to/codebase

# Find hardcoded algorithm references
grep -rE "(RSA|ECDSA|ECDH|DSA|DH)" --include="*.c" --include="*.h" \
     --include="*.py" --include="*.java" --include="*.go" /path/to/codebase

# Scan for OpenSSL/crypto library usage
grep -rE "(EVP_|RSA_|EC_|DH_)" --include="*.c" /path/to/codebase

# Find certificate files
find /path/to/system -name "*.pem" -o -name "*.crt" -o -name "*.key" \
     -o -name "*.p12" -o -name "*.pfx" 2>/dev/null
```

Phase 2: Protocol Analysis

```
// Tool concept: TLS cipher suite analyzer
#include <openssl/ssl.h>

void analyze_tls_config(SSL_CTX *ctx) {
    STACK_OF(SSL_CIPHER) *ciphers = SSL_CTX_get_ciphers(ctx);

    printf("Configured Cipher Suites:\n");
    for (int i = 0; i < sk_SSL_CIPHER_num(ciphers); i++) {
        const SSL_CIPHER *cipher = sk_SSL_CIPHER_value(ciphers, i);
        const char *name = SSL_CIPHER_get_name(cipher);
        int bits = SSL_CIPHER_get_bits(cipher, NULL);
```

```
        // Flag quantum-vulnerable key exchange
        if (strstr(name, "ECDHE") || strstr(name, "DHE") ||
            strstr(name, "RSA")) {
            printf("  [QUANTUM-VULNERABLE] %s (%d bits)\n", name, bits);
        }
    }
}
```

Phase 3: Dependency Analysis

```
Cryptographic Bill of Materials (CBOM) Template
================================================

System: [Application Name]
Owner: [Team/Individual]
Criticality: [High/Medium/Low]

Direct Cryptographic Dependencies:
┌──────────────────┬──────────────┬──────────────────┬──────────────┐
│ Library          │ Version      │ Algorithms       │ PQC Ready?   │
├──────────────────┼──────────────┼──────────────────┼──────────────┤
│ OpenSSL          │ 1.1.1w       │ RSA, ECDHE       │ No           │
│ libsodium        │ 1.0.18       │ X25519, Ed25519  │ No           │
│ BouncyCastle     │ 2.0+         │ RSA, ECDSA       │ Partial      │
└──────────────────┴──────────────┴──────────────────┴──────────────┘

Certificates and Keys:
┌──────────────────┬──────────────┬──────────────────┬──────────────┐
│ Purpose          │ Algorithm    │ Key Size         │ Expiry       │
├──────────────────┼──────────────┼──────────────────┼──────────────┤
│ TLS Server       │ RSA          │ 2048-bit         │ 2025-03-15   │
│ Code Signing     │ ECDSA        │ P-256            │ 2026-01-01   │
│ Root CA          │ RSA          │ 4096-bit         │ 2034-12-31   │
└──────────────────┴──────────────┴──────────────────┴──────────────┘

Protocol Usage:
┌──────────────────┬──────────────────┬──────────────────────────────┐
│ Protocol         │ Key Exchange     │ Authentication               │
├──────────────────┼──────────────────┼──────────────────────────────┤
│ TLS 1.3          │ ECDHE (X25519)   │ RSA-2048 certificates        │
│ SSH              │ ECDH (nistp256)  │ Ed25519 keys                 │
│ IPsec            │ IKEv2 (ECDH)     │ RSA certificates             │
└──────────────────┴──────────────────┴──────────────────────────────┘

Migration Status: [ ] Not Started  [ ] Planning  [ ] In Progress  [ ] Complete
```

11.2.3 CISA Automated Discovery Tools

CISA recommends automated cryptography discovery and inventory (ACDI) tools:

```
ACDI Tool Capabilities
======================

Network-Level Discovery:
├── Passive TLS inspection
├── Certificate chain analysis
├── Protocol version detection
└── Cipher suite enumeration

Application-Level Discovery:
├── Static code analysis
├── Binary analysis
├── Configuration file scanning
└── Library dependency mapping

Infrastructure Discovery:
├── HSM inventory
├── KMS configuration audit
├── Certificate store scanning
└── Secrets manager inventory

Recommended Scan Integration Points:
├── CI/CD pipelines (pre-commit, build time)
├── Container image scanning
├── Infrastructure as Code (IaC) scanning
└── Runtime monitoring
```

Unit 11.2 Summary

This unit covered:

- Why most organizations lack visibility into their cryptographic surface and the importance of comprehensive discovery across application, protocol, infrastructure, data, and identity layers
- A three-phase inventory methodology: automated discovery (scanning for certificates, algorithm references, and library usage), protocol analysis (TLS cipher suite inspection), and dependency analysis
- The Cryptographic Bill of Materials (CBOM) format for documenting cryptographic assets, certificates, keys, and protocol usage per system
- CISA-recommended automated cryptography discovery and inventory (ACDI) tools and their integration points in CI/CD pipelines, container scanning, and runtime monitoring

What's Next: Unit 11.3 covers designing for cryptographic agility, showing how to build abstraction layers and policy-driven configurations that enable seamless algorithm transitions.

Unit 11.4: PQC Library Selection and Integration

Prerequisites: Unit 11.3 (Cryptographic Agility), Module 8 (Hybrid Cryptography)

Estimated Time: 2-3 hours

Learning Objectives

After completing this unit, you will be able to:

1. **Evaluate PQC library options** (liboqs, PQClean, OpenSSL 3.5)
2. **Integrate liboqs** into existing C/C++ projects
3. **Use the OQS provider** for OpenSSL 3.x applications
4. **Implement hybrid constructions** combining classical and PQC

11.4.1 Library Comparison

```
PQC Library Landscape (2025)
============================

┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
│ Library          │ Algorithms       │ Production?      │ Best For         │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ OpenSSL 3.5+     │ ML-KEM,ML-DSA,   │ Yes              │ Production       │
│                  │ SLH-DSA          │                  │ deployments      │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ liboqs           │ All NIST +       │ Research         │ Prototyping,     │
│                  │ alternates       │ (use caution)    │ testing          │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ oqs-provider     │ All liboqs via   │ Research         │ OpenSSL apps     │
│                  │ OpenSSL API      │                  │ migration        │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ PQClean          │ Clean reference  │ Reference        │ Study, audit     │
│                  │ implementations  │                  │ starting pt      │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ BoringSSL        │ ML-KEM (Kyber)   │ Yes (Google)     │ If using         │
│                  │ X25519Kyber768   │                  │ BoringSSL        │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ wolfSSL          │ ML-KEM, ML-DSA   │ Yes              │ Embedded,        │
│                  │ (via liboqs)     │                  │ FIPS-140-3       │
└──────────────────┴──────────────────┴──────────────────┴──────────────────┘

Recommendation:
- New production systems: OpenSSL 3.5+ (native PQC support)
- Existing OpenSSL apps: oqs-provider for gradual migration
- Prototyping/research: liboqs directly
- Embedded systems: wolfSSL with PQC support
```

11.4.2 liboqs Integration

Installation:

```
# Clone and build liboqs
git clone https://github.com/open-quantum-safe/liboqs.git
cd liboqs
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=/usr/local ..
make -j$(nproc)
sudo make install
```

Basic ML-KEM Example:

```
#include <oqs/oqs.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int ml_kem_key_exchange(void) {
    OQS_KEM *kem = NULL;
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *ciphertext = NULL;
    uint8_t *shared_secret_enc = NULL;
    uint8_t *shared_secret_dec = NULL;
    int ret = -1;

    // Initialize ML-KEM-768
    kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
    if (kem == NULL) {
        fprintf(stderr, "ML-KEM-768 not available\n");
        return -1;
    }

    // Allocate memory
    public_key = malloc(kem->length_public_key);
    secret_key = malloc(kem->length_secret_key);
    ciphertext = malloc(kem->length_ciphertext);
    shared_secret_enc = malloc(kem->length_shared_secret);
    shared_secret_dec = malloc(kem->length_shared_secret);

    if (!public_key || !secret_key || !ciphertext ||
        !shared_secret_enc || !shared_secret_dec) {
        fprintf(stderr, "Memory allocation failed\n");
        goto cleanup;
    }

    // Generate key pair (Alice)
    if (OQS_KEM_keypair(kem, public_key, secret_key) != OQS_SUCCESS) {
        fprintf(stderr, "Key generation failed\n");
        goto cleanup;
    }

    printf("Public key size: %zu bytes\n", kem->length_public_key);
    printf("Secret key size: %zu bytes\n", kem->length_secret_key);

    // Encapsulate (Bob)
```

```
    if (OQS_KEM_encaps(kem, ciphertext, shared_secret_enc,
                       public_key) != OQS_SUCCESS) {
        fprintf(stderr, "Encapsulation failed\n");
        goto cleanup;
    }

    printf("Ciphertext size: %zu bytes\n", kem->length_ciphertext);

    // Decapsulate (Alice)
    if (OQS_KEM_decaps(kem, shared_secret_dec, ciphertext,
                       secret_key) != OQS_SUCCESS) {
        fprintf(stderr, "Decapsulation failed\n");
        goto cleanup;
    }

    // Verify shared secrets match
    if (memcmp(shared_secret_enc, shared_secret_dec,
               kem->length_shared_secret) != 0) {
        fprintf(stderr, "Shared secrets don't match!\n");
        goto cleanup;
    }

    printf("Key exchange successful!\n");
    printf("Shared secret size: %zu bytes\n", kem->length_shared_secret);
    ret = 0;

cleanup:
    OQS_MEM_secure_free(secret_key, kem ? kem->length_secret_key : 0);
    OQS_MEM_secure_free(shared_secret_enc, kem ? kem->length_shared_secret : 0);
    OQS_MEM_secure_free(shared_secret_dec, kem ? kem->length_shared_secret : 0);
    free(public_key);
    free(ciphertext);
    OQS_KEM_free(kem);

    return ret;
}
```

ML-DSA Signature Example:

```
#include <oqs/oqs.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int ml_dsa_sign_verify(const uint8_t *message, size_t msg_len) {
    OQS_SIG *sig = NULL;
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *signature = NULL;
    size_t sig_len = 0;
    int ret = -1;

    // Initialize ML-DSA-65
    sig = OQS_SIG_new(OQS_SIG_alg_ml_dsa_65);
    if (sig == NULL) {
        fprintf(stderr, "ML-DSA-65 not available\n");
        return -1;
    }
```

```
    // Allocate memory
    public_key = malloc(sig->length_public_key);
    secret_key = malloc(sig->length_secret_key);
    signature = malloc(sig->length_signature);

    if (!public_key || !secret_key || !signature) {
        fprintf(stderr, "Memory allocation failed\n");
        goto cleanup;
    }

    // Generate key pair
    if (OQS_SIG_keypair(sig, public_key, secret_key) != OQS_SUCCESS) {
        fprintf(stderr, "Key generation failed\n");
        goto cleanup;
    }

    printf("ML-DSA-65 Key Sizes:\n");
    printf("  Public key: %zu bytes\n", sig->length_public_key);
    printf("  Secret key: %zu bytes\n", sig->length_secret_key);

    // Sign message
    if (OQS_SIG_sign(sig, signature, &sig_len,
                     message, msg_len, secret_key) != OQS_SUCCESS) {
        fprintf(stderr, "Signing failed\n");
        goto cleanup;
    }

    printf("Signature size: %zu bytes\n", sig_len);

    // Verify signature
    if (OQS_SIG_verify(sig, message, msg_len,
                       signature, sig_len, public_key) != OQS_SUCCESS) {
        fprintf(stderr, "Verification failed\n");
        goto cleanup;
    }

    printf("Signature verified successfully!\n");
    ret = 0;

cleanup:
    OQS_MEM_secure_free(secret_key, sig ? sig->length_secret_key : 0);
    free(public_key);
    free(signature);
    OQS_SIG_free(sig);

    return ret;
}
```

11.4.3 Hybrid Implementation Pattern

```
// hybrid_kem.c - X25519 + ML-KEM-768 hybrid key exchange

#include <oqs/oqs.h>
#include <openssl/evp.h>
#include <openssl/kdf.h>
#include <string.h>

#define X25519_KEY_SIZE 32
#define X25519_SHARED_SIZE 32

typedef struct {
    uint8_t x25519_public[X25519_KEY_SIZE];
    uint8_t mlkem_public[1184];  // ML-KEM-768 public key
} hybrid_public_key_t;

typedef struct {
    uint8_t x25519_secret[X25519_KEY_SIZE];
    uint8_t mlkem_secret[2400];  // ML-KEM-768 secret key
} hybrid_secret_key_t;

typedef struct {
    uint8_t x25519_public[X25519_KEY_SIZE];  // Ephemeral X25519
    uint8_t mlkem_ciphertext[1088];          // ML-KEM-768 ciphertext
} hybrid_ciphertext_t;

int hybrid_keygen(hybrid_public_key_t *pk, hybrid_secret_key_t *sk) {
    EVP_PKEY *x25519_key = NULL;
    EVP_PKEY_CTX *ctx = NULL;
    OQS_KEM *kem = NULL;
    size_t len;
    int ret = -1;

    // Generate X25519 key pair
    ctx = EVP_PKEY_CTX_new_id(EVP_PKEY_X25519, NULL);
    if (!ctx || EVP_PKEY_keygen_init(ctx) <= 0 ||
        EVP_PKEY_keygen(ctx, &x25519_key) <= 0) {
        goto cleanup;
    }

    // Extract X25519 keys
    len = X25519_KEY_SIZE;
    EVP_PKEY_get_raw_public_key(x25519_key, pk->x25519_public, &len);
    len = X25519_KEY_SIZE;
    EVP_PKEY_get_raw_private_key(x25519_key, sk->x25519_secret, &len);

    // Generate ML-KEM-768 key pair
    kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
    if (!kem) goto cleanup;

    if (OQS_KEM_keypair(kem, pk->mlkem_public, sk->mlkem_secret) != OQS_SUCCESS) {
        goto cleanup;
    }

    ret = 0;

cleanup:
    EVP_PKEY_free(x25519_key);
```

```
    EVP_PKEY_CTX_free(ctx);
    OQS_KEM_free(kem);
    return ret;
}

int hybrid_encaps(const hybrid_public_key_t *pk,
                  hybrid_ciphertext_t *ct,
                  uint8_t *shared_secret, size_t ss_len) {
    EVP_PKEY *x25519_eph = NULL;
    EVP_PKEY *x25519_peer = NULL;
    EVP_PKEY_CTX *ctx = NULL;
    OQS_KEM *kem = NULL;

    uint8_t x25519_shared[X25519_SHARED_SIZE];
    uint8_t mlkem_shared[32];  // ML-KEM shared secret
    size_t len;
    int ret = -1;

    // Generate ephemeral X25519 key
    ctx = EVP_PKEY_CTX_new_id(EVP_PKEY_X25519, NULL);
    if (!ctx || EVP_PKEY_keygen_init(ctx) <= 0 ||
        EVP_PKEY_keygen(ctx, &x25519_eph) <= 0) {
        goto cleanup;
    }

    // Export ephemeral public key to ciphertext
    len = X25519_KEY_SIZE;
    EVP_PKEY_get_raw_public_key(x25519_eph, ct->x25519_public, &len);

    // Perform X25519 key agreement
    x25519_peer = EVP_PKEY_new_raw_public_key(EVP_PKEY_X25519, NULL,
                                              pk->x25519_public, X25519_KEY_SIZE);
    EVP_PKEY_CTX_free(ctx);
    ctx = EVP_PKEY_CTX_new(x25519_eph, NULL);

    if (EVP_PKEY_derive_init(ctx) <= 0 ||
        EVP_PKEY_derive_set_peer(ctx, x25519_peer) <= 0) {
        goto cleanup;
    }

    len = X25519_SHARED_SIZE;
    if (EVP_PKEY_derive(ctx, x25519_shared, &len) <= 0) {
        goto cleanup;
    }

    // ML-KEM encapsulation
    kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);
    if (!kem) goto cleanup;

    if (OQS_KEM_encaps(kem, ct->mlkem_ciphertext, mlkem_shared,
                       pk->mlkem_public) != OQS_SUCCESS) {
        goto cleanup;
    }

    // Combine shared secrets using HKDF
    // shared_secret = HKDF(x25519_shared || mlkem_shared)
    {
        EVP_PKEY_CTX *hkdf_ctx = EVP_PKEY_CTX_new_id(EVP_PKEY_HKDF, NULL);
        uint8_t combined[X25519_SHARED_SIZE + 32];
```

```
        memcpy(combined, x25519_shared, X25519_SHARED_SIZE);
        memcpy(combined + X25519_SHARED_SIZE, mlkem_shared, 32);

        EVP_PKEY_derive_init(hkdf_ctx);
        EVP_PKEY_CTX_set_hkdf_md(hkdf_ctx, EVP_sha384());
        EVP_PKEY_CTX_set1_hkdf_key(hkdf_ctx, combined, sizeof(combined));
        EVP_PKEY_CTX_set1_hkdf_salt(hkdf_ctx,
                                    (uint8_t*)"X25519MLKEM768", 14);
        EVP_PKEY_CTX_add1_hkdf_info(hkdf_ctx,
                                    (uint8_t*)"hybrid-kem", 10);

        EVP_PKEY_derive(hkdf_ctx, shared_secret, &ss_len);
        EVP_PKEY_CTX_free(hkdf_ctx);

        OPENSSL_cleanse(combined, sizeof(combined));
    }

    ret = 0;

cleanup:
    OPENSSL_cleanse(x25519_shared, sizeof(x25519_shared));
    OPENSSL_cleanse(mlkem_shared, sizeof(mlkem_shared));
    EVP_PKEY_free(x25519_eph);
    EVP_PKEY_free(x25519_peer);
    EVP_PKEY_CTX_free(ctx);
    OQS_KEM_free(kem);
    return ret;
}

// Similar hybrid_decaps() function...
```

11.4.4 OpenSSL 3.5 Native PQC

With OpenSSL 3.5+, PQC is built-in:

```
// OpenSSL 3.5+ ML-KEM example
#include <openssl/evp.h>
#include <openssl/core_names.h>
#include <openssl/params.h>

int openssl35_mlkem_keygen(void) {
    EVP_PKEY_CTX *ctx = NULL;
    EVP_PKEY *pkey = NULL;

    // Create context for ML-KEM-768
    ctx = EVP_PKEY_CTX_new_from_name(NULL, "ML-KEM-768", NULL);
    if (!ctx) {
        fprintf(stderr, "ML-KEM-768 not available\n");
        return -1;
    }

    // Generate key pair
    if (EVP_PKEY_keygen_init(ctx) <= 0 ||
        EVP_PKEY_generate(ctx, &pkey) <= 0) {
        fprintf(stderr, "Key generation failed\n");
        EVP_PKEY_CTX_free(ctx);
        return -1;
    }
```

```
    printf("ML-KEM-768 key generated successfully!\n");

    // Use for encapsulation/decapsulation...

    EVP_PKEY_free(pkey);
    EVP_PKEY_CTX_free(ctx);
    return 0;
}
```

Unit 11.4 Summary

This unit covered:

- A comparative evaluation of PQC libraries including OpenSSL 3.5+ (production), liboqs (prototyping), oqs-provider (OpenSSL migration), PQClean (reference), BoringSSL, and wolfSSL (embedded/FIPS)
- Hands-on liboqs integration: building from source, performing ML-KEM-768 key encapsulation/decapsulation, and ML-DSA-65 signing/verification with complete C code examples
- Implementing a hybrid X25519 + ML-KEM-768 key exchange pattern that combines both shared secrets via HKDF for defense-in-depth
- Using OpenSSL 3.5+ native PQC support through the standard EVP API for ML-KEM key generation without external dependencies

What's Next: Unit 11.5 addresses testing and validation, covering NIST test vector validation, interoperability testing, and comprehensive migration validation checklists.

Unit 11.8: Production Deployment Checklist

This unit provides concrete checklists for moving a PQC-enabled system into production and monitoring it afterward. Use these as gate criteria for deployment reviews and as ongoing operational metrics.

Pre-Deployment Sign-Off Checklist

Before flipping the switch on hybrid or PQ-only mode in production:

Security review

- ☐ Threat model updated to include post-quantum adversary; harvest-now-decrypt-later explicitly in scope for data with > 5 year confidentiality window

- ☐ Cryptographic inventory (from Unit 11.2) re-verified against actual deployed binaries
- ☐ No cryptographic secrets, seeds, or private keys present in logs, error messages, or crash dumps (grep `0x`, `BEGIN PRIVATE`, `-----BEGIN` in log archives)
- ☐ Key rotation procedures tested and documented for each algorithm in hybrid configuration
- ☐ Backup / recovery procedures validated for PQ keys (PQ secret keys are 10-100× larger than classical — HSM/storage capacity confirmed)
- ☐ Randomness source validated: `getrandom()` (Linux), `RtlGenRandom` / BCryptGenRandom (Windows), `/dev/urandom` with entropy-pool monitoring for embedded

Performance validation

- ☐ End-to-end latency regression measured under realistic load; < 10% degradation threshold documented
- ☐ CPU utilization under peak load remains within headroom (typically +5-15% for ML-KEM/ML-DSA hybrids)
- ☐ Memory footprint tested under sustained connection load (PQ key material uses 2-10× more RAM per session)
- ☐ Bandwidth impact confirmed via capacity planning against sheet in Appendix (3-5× handshake overhead)
- ☐ Worst-case signing time for ML-DSA documented (rejection sampling tail — signatures occasionally take 10-50× median)

Rollback procedure

- ☐ Rollback to classical-only configuration tested in staging, documented with specific commands
- ☐ Decision criteria defined: when to roll back (error rate > X%, latency > Yms p99, specific log patterns)

- ☐ Rollback window communicated: can revert within N minutes without data loss
- ☐ Approvals obtained from security, infrastructure, and product owners
- ☐ Post-rollback forensic procedures documented (preserve keys/logs for analysis)

Compliance and documentation

- ☐ FIPS 140-3 module validation confirmed if required (see Unit 11.5.5)
- ☐ CNSA 2.0 timeline alignment verified for federal systems
- ☐ National regulator requirements checked (ANSSI / BSI / NCSC timelines)
- ☐ Customer-facing documentation updated (TLS cipher policy, SSH config, X.509 cert algorithms)
- ☐ Internal runbook updated with PQ-specific troubleshooting

Post-Deployment Monitoring Metrics

Instrument from day one; don't wait for problems:

Adoption metrics (track trend, not absolute values)

- ☐ Percentage of TLS handshakes using hybrid key exchange
- ☐ Percentage of SSH sessions using `sntrup761x25519-sha512@openssh.com` or equivalent
- ☐ Percentage of certificates issued with PQ or hybrid signature algorithm
- ☐ Alert: adoption rate drops > 10% week-over-week (client-side regression)

Error metrics (any sustained rise is a red flag)

- ☐ ML-KEM decapsulation failure rate (should be $\ll 10^{-10}$ with correct implementation; any non-zero count at scale indicates bug)
- ☐ ML-DSA signature verification failures (should be 0 for legitimately signed content)
- ☐ Algorithm negotiation failures in TLS/SSH (fallback to classical)

- ☐
  OQS provider / library error counters
- ☐
  Alert: any of above > 0.01% of operations

Performance metrics (baseline, watch for drift)

- ☐
  Handshake latency p50/p95/p99 per protocol
- ☐
  Signing latency histogram (detect rejection-sampling tail anomalies)
- ☐
  Key generation latency (one-shot operations, less critical)
- ☐
  Memory / CPU utilization trends
- ☐
  Alert: p99 latency regresses > 20% from baseline

Security posture

- ☐
  HSM / key storage capacity utilization (PQ keys are large)
- ☐
  Key rotation cadence executing as scheduled
- ☐
  Certificate expiration tracking (including hybrid composites)
- ☐
  Quarterly review of new CVEs for PQC implementations in use

Incident Response for PQC-Specific Events

Preparation for "algorithm vulnerability disclosed" class of incidents:

1. **Decision tree prepared** for: "new attack reduces security of algorithm X by Y bits"
   - What's the exploit feasibility? (academic vs. practical)
   - Does hybrid construction mitigate it? (classical + PQ — both must break)
   - Is algorithm replacement feasible in timeframe? (crypto agility matters here)
2. **Contact list** for NIST PQC Project, vendor security teams, CERT
3. **Algorithm swap runbook** — procedure to shift from ML-KEM-768 to, e.g., HQC-256 via configuration change
4. **Communication plan** — customer disclosure, regulator notification, press response

Refer back to Unit 10.3 (Cryptographic Agility) for architectural support of rapid algorithm swaps.

Three Concrete Incident Playbooks

The high-level decision tree above frames the problem; the three playbooks below give step-by-step actions for the most likely PQC operational incidents in 2026-2030.

Playbook A — "CVE announced for our PQC library, what do I do in the next 2 hours?"

Trigger example: GitHub Security Advisory drops for liboqs / OpenSSL native PQC / AWS-LC-FIPS / etc.

0-15 min — Triage

1. Read the advisory. Identify: affected versions, attack class (timing, fault, key recovery, denial-of-service), CVSS score, exploitability (PoC published?).
2. Inventory query (CBOM, see Unit 11.2): which production systems use the affected library version?
3. Check whether your hybrid construction mitigates: if the CVE is in PQ-only and you deploy hybrid (X25519 + ML-KEM-768), classical leg may still protect transport confidentiality *for now*.

15-60 min — Containment 4. Patch availability: is fix in the library's main branch / tagged release? If yes, schedule emergency rotation per change-management policy. 5. Workaround available? E.g., compile-flag tweak, configuration change, algorithm swap via crypto-agility framework (see Unit 10.3). 6. If exploit is timing/microarchitectural and you run multi-tenant: assess co-residency risk; consider workload isolation as interim mitigation.

1-2 hours — Communication 7. Internal: notify security on-call, infra owners of affected services, leadership. 8. External: if CVE is in customer-facing path, prepare disclosure per your standard CVE-response policy. 9. File for FedRAMP / SOC2 / ISO 27001 evidence (incident logged, response timeline documented).

Day 1-7 — Remediation 10. Deploy patch through normal CI/CD with elevated priority. 11. If fix requires algorithm swap (e.g., HQC disabled in liboqs 0.13 due to CVE-2025-48946): use the crypto-agility framework to switch to alternative; rotate any keys generated under the vulnerable algorithm. 12. Post-incident review: did your monitoring detect anything anomalous before the disclosure? Did your detection time meet SLA?

Playbook B — "Vendor announces end-of-life for the PQC HSM/library we depend on"

Trigger example: HSM vendor sunsets a firmware family; library project archives a repository; cloud KMS deprecates a key spec.

Week 1 — Assessment

1. Read the EOL notice carefully: cutoff date, security-fix date, migration recommendation from vendor.
2. Identify the dependency chain: what services / customers / contracts depend on this product?
3. Estimate migration cost: do you need new hardware, new library, new vendor integration, or just configuration changes?

Week 2-4 — Migration plan 4. Evaluate replacements (use Unit 11.4 library selection criteria as starting point). 5. Run interop testing: can you encapsulate with old vendor and decapsulate with new vendor? Critical for data-at-rest where re-wrapping is needed. 6. Build the migration runbook: dual-deploy period (both old and new), rotation schedule, rollback criteria.

Month 2-N — Execution 7. Deploy new vendor in shadow mode (alongside old), verify correctness via canary. 8. Roll over services in priority order (most-critical-data-last to limit blast radius). 9. Once 100% of new traffic uses new vendor and re-wrap of historical data is complete, decommission old vendor with key destruction per policy. 10. Post-migration: update CBOM, deprecate old vendor's entry in your vendor risk database.

Playbook C — "New cryptanalysis published: theoretical attack on our primary algorithm, no CVE yet"

Trigger example: arXiv paper claims to reduce ML-KEM security by N bits; FALCON pre-finalization side-channel result; novel rejection-signature attack on ML-DSA.

Hours 1-24 — Read and reason

1. Read the paper. Identify: claimed security loss, attack model (chosen-CT, side-channel, fault, etc.), prerequisite (number of queries, equipment, oracle access).
2. Compare against the threat model for your deployment: does the attack require capabilities a real adversary has against you?
3. Subscribe to follow-up discussion: NIST pqc-forum, IACR ePrint comments, library issue trackers.

Days 2-30 — Watch and prepare 4. Wait for community response: is the attack reproduced? Is the security loss confirmed by independent analysis? Are mitigation patches proposed? 5. If attack is real and significant: pre-stage your migration playbook (Playbook A above). Identify replacement algorithms in your crypto-agility framework. 6. Engage with library maintainers: are they planning a release? Will they document the impact?

Decision criteria — when does "theoretical" become "operational"?

- A CVE is filed against your library (→ Playbook A).
- A working PoC is published.
- A vendor advisory recommends migration.
- The cryptanalysis result reduces the algorithm below your threat-model floor (e.g., ML-KEM-768 reduced from 192-bit to 128-bit security may still be acceptable; reduced to 80-bit is not).

Until one of these triggers fires: maintain awareness, do not panic-migrate. Speculative migrations have their own risks (interop breakage, increased attack surface during transition).

Case study: Meta's PQC Migration Framework (April 2026)

Meta Engineering published a detailed PQC migration writeup on April 16, 2026 ("Post-Quantum Cryptography Migration at Meta: Framework, Lessons, and Takeaways"). It is the most detailed real-world case study published to date and worth reading in full. Key takeaways summarized:

Five-level maturity model:

1. **Level 0 — Unaware**: no inventory, no plan.
2. **Level 1 — Cataloged**: CBOM complete; leadership aware.
3. **Level 2 — Tested**: key algorithms benchmarked on representative workloads.
4. **Level 3 — Rolling out**: hybrid PQ in production for specific services.
5. **Level 4 — Default**: hybrid PQ default across all new services; legacy on deprecation path.

Six-step migration strategy:

1. Cryptographic inventory (automated scanning across ~millions of repos).
2. Threat-model classification (HNDL-sensitive vs. not).
3. Algorithm selection per service class (ML-KEM-768+X25519 default KEM; ML-DSA-65+ECDSA default signature).

4. Library upgrades (internal fork of boringssl with PQ patches, now on native OpenSSL 3.5+ where possible).
5. Service-by-service hybrid rollout with monitoring (handshake success rates, latency p99, error budgets).
6. Legacy retirement with explicit gate criteria (e.g., "disable classical-only KEX when < 0.5% of traffic uses it").

Reported state (April 2026): most internal Meta services use hybrid ML-KEM+X25519 for transport security. External-facing communication is the next phase; the constraints are middlebox compatibility and partner readiness. No ML-DSA in production signing paths yet.

Lessons echoed from other large deployments:

- Automated CBOM generation is the cornerstone; without it, scope is unknowable.
- Middlebox incompatibility is the #1 production break (see Unit 11.9.2 for ClientHello size analysis).
- Monitoring for *silent downgrade* is as important as monitoring for failures — classical fallback without alerting is the real risk.

Full writeup: https://engineering.fb.com/2026/04/16/security/post-quantum-cryptography-migration-at-meta-framework-lessons-and-takeaways/

Unit 11.8 Summary

Production deployment is not the end of the migration work — it's the transition into operational responsibility for a new class of cryptographic dependencies. Comprehensive pre-deployment sign-off prevents the "ship it and hope" failure mode; continuous monitoring catches implementation regressions, adoption stagnation, and emerging attacks. Pair this checklist with crypto-agility (Unit 10.3) so that responding to a PQC vulnerability is a configuration change, not a re-architecture.

Unit 11.9: Cloud KMS Integration for Post-Quantum Cryptography

Most production cryptography in 2026 happens inside managed Key Management Services. This unit documents the PQC state of the major KMS/HSM offerings as of April 2026, gives working API examples, and walks through the engineering patterns that recur across all of them (composite certs, hybrid KEMs, cryptographic agility, CBOM).

Status evolves fast — the rollout snapshot below is current to **April 2026**. Always verify against the vendor's latest release notes before architecting a migration.

11.9.1 Vendor Landscape Snapshot (April 2026)

> Last verified: April 2026 (v1.0). Cloud KMS/HSM offerings ship PQ features on a rapid cadence; always confirm status against the vendor's "What's New" page before architectural decisions.

VENDOR / PRODUCT	HYBRID TLS (ML-KEM)	PQ SIGNATURES (ML-DSA/ SLH-DSA)	PQ KEM KEYS (ML-KEM)	FIPS 140-3 W/ PQC
AWS KMS	GA (non-FIPS endpoints)	ML-DSA-44/65/87 GA Jun 2025	Preview	KMS HSM at 140-3 L3
AWS Private CA	N/A	ML-DSA GA Nov 2025	N/A	Module-level
AWS-LC / AWS-LC-FIPS	(library)	ML-DSA in v3.0	**v3.0 first FIPS 140-3 with ML-KEM**	Cert in process
GCP Cloud KMS	via GFE (Chrome)	ML-DSA-65/ SLH-DSA-128s preview (Feb 2025)	ML-KEM-768/1024, X-Wing preview (Oct 2025)	SOFTWARE only; HSM pending
GCP Cloud HSM	N/A	No PQC	No PQC	Marvell LiquidSecurity (classical only)
Azure Key Vault / Managed HSM	No	No	No	No PQC shipped
Windows CNG / .NET 10	Schannel preview	ML-KEM/ML-DSA/SLH-DSA GA Nov 2025	GA Nov 2025	Pending
HashiCorp Vault Enterprise 1.19+	via TLS	ML-DSA, SLH-DSA experimental	Roadmap	No (experimental)
Thales Luna HSM (fw 7.9+)	via TLS	ML-DSA, SLH-DSA, LMS/ HSS	ML-KEM	Pending (CAVP done)
Entrust nShield (fw 13.8)	via TLS	ML-DSA, SLH-DSA	ML-KEM	Pending (CAVP Sep 2025)

VENDOR / PRODUCT	HYBRID TLS (ML-KEM)	PQ SIGNATURES (ML-DSA/ SLH-DSA)	PQ KEM KEYS (ML-KEM)	FIPS 140-3 W/ PQC
Utimaco u.trust / Quantum Protect	via TLS	ML-DSA, LMS/ HSS, XMSS	ML-KEM	CAVP done; CMVP pending
Fortanix DSM	via TLS	ML-DSA, LMS	ML-KEM	FIPS 140-2 L3 only
Cloudflare (edge TLS)	**X25519MLKEM768 default** (~57% of browser-initiated, pq.cloudflareresearch.com early 2026)	ML-DSA origin auth mid-2026	N/A	N/A
Akamai (edge TLS)	X25519MLKEM768 default Q1 2026	Roadmap	N/A	N/A
IBM Hyper Protect Crypto Services	via TLS	Dilithium (pre-standard)	Kyber (pre-standard)	FIPS 140-2 L4
Oracle OCI KMS / AI Database 26ai	Hybrid ML-KEM TLS Jan 2026	No Vault key types yet	No Vault key types yet	No
YubiHSM 2	N/A	No	No	FIPS 140-3 expected Q2 2026 (no PQC)

Reading the table:

- "GA (non-FIPS endpoints)" — the hybrid TLS handshake is enabled on the standard service endpoints but **not** on FIPS-only endpoints (FIPS validation must catch up first).
- "Preview" — accessible to customers but behavior may change; not covered by SLAs.

- "CAVP done; CMVP pending" — the algorithm implementation has passed NIST's Cryptographic Algorithm Validation Program, but the hardware module's FIPS 140-3 Cryptographic Module Validation Program certificate for the PQ firmware has not yet been issued.

11.9.2 AWS Integration

> **⚠ #1 PRODUCTION BREAK IN 2026 PQ-TLS DEPLOYMENTS: MIDDLEBOX CLIENTHELLO FRAGMENTATION.**
>
> The TLS ClientHello grows from ~500 bytes to ~1,600+ bytes once an `X25519MLKEM768` key share is included — large enough to fragment across TCP segments. Load balancers, WAFs, DPI engines, and corporate TLS-intercepting proxies with fixed input buffers or handshake-size heuristics silently drop the fragmen-
> ted handshake. Industry guidance (Akamai, Cloudflare, IETF UTA WG): **audit every middlebox in the path** and run synthetic PQ ClientHello traffic across production topology *before* rolling out. Most production incidents we've catalogued trace to this.

AWS KMS exposes both hybrid post-quantum TLS (for key-material protection in transit) and PQ-signing key types (for customer signature operations).

Hybrid post-quantum TLS. Enable `X25519MLKEM768` for the TLS handshake between your client and the KMS service endpoint. Protects the *transport* of your data keys against harvest-now-decrypt-later. Available on KMS, ACM, Secrets Manager, S3 (regional endpoints), ALB/NLB, CloudFront, Transfer Family, Payment Cryptography, Private CA. Not yet available on FIPS KMS endpoints or on PrivateLink.

```
# Python example - uses boto3 with AWS-LC-backed Python cryptography
import boto3
from botocore.config import Config

# No code change required for hybrid TLS - it negotiates automatically
# when both client and service support X25519MLKEM768.
# To verify: inspect CloudTrail events, field tlsDetails.cipherSuite
kms = boto3.client('kms',
    config=Config(tcp_keepalive=True))
```

Customer-held ML-DSA keys. KMS created `SIGN_VERIFY` asymmetric keys with ML-DSA support in June 2025.

```
import boto3
kms = boto3.client('kms', region_name='us-west-1')

# Create ML-DSA-65 key (KeySpec values: ML_DSA_44, ML_DSA_65, ML_DSA_87)
response = kms.create_key(
    KeySpec='ML_DSA_65',
    KeyUsage='SIGN_VERIFY',
    Origin='AWS_KMS',
    Description='PQ signing key for application X'
)
key_id = response['KeyMetadata']['KeyId']

# Sign a message
signature = kms.sign(
    KeyId=key_id,
    Message=b'payload to sign',
    MessageType='RAW',           # or EXTERNAL_MU for 64-byte pre-computed mu
    SigningAlgorithm='ML_DSA_SHAKE_256'
)['Signature']

# Verify
is_valid = kms.verify(
    KeyId=key_id,
    Message=b'payload to sign',
    Signature=signature,
    SigningAlgorithm='ML_DSA_SHAKE_256'
)['SignatureValid']
```

Key behaviors to plan for:

- **No automatic rotation** — ML-DSA keys follow the same rule as other asymmetric KMS keys (manual rotation only).
- **Standard asymmetric pricing** — $1/month per key, $0.15 per 10,000 sign/verify/GetPublicKey operations. No PQC premium.
- **Public key retrieval** — use `GetPublicKey` to export the raw NIST-PQC-formatted public key.

AWS Private CA with ML-DSA. As of November 2025, AWS Private CA can be configured to use ML-DSA-backed KMS keys as the CA signing key, issuing end-entity certificates with ML-DSA algorithm identifiers. Available in all commercial AWS regions, GovCloud, and China regions.

AWS-LC-FIPS v3.0 is the first open-source cryptographic library to include ML-KEM within its FIPS 140-3 module boundary (submitted, certificate in process as of late 2024/early 2026). This matters because most AWS services now use AWS-LC internally, so AWS-LC-FIPS 3.0's eventual certification unlocks FIPS 140-3 PQC across the AWS stack.

End-to-end ML-DSA code signing workflow (documented in the AWS Security Blog, November 2025): create a private CA on AWS Private CA with ML-DSA issuance enabled; issue code-signing leaf certificates backed by KMS ML-DSA keys; sign artifacts with `kms:Sign` using the same key material; verify using standard X.509 path validation augmented with ML-DSA signature verification. No AWS Signer service support for ML-DSA as of April 2026 — the workflow goes through KMS + Private CA directly.

11.9.3 Google Cloud Platform Integration

GCP Cloud KMS supports PQ signatures (preview since February 2025) and PQ KEMs (preview since October 2025) at the SOFTWARE protection level. Cloud HSM (Marvell LiquidSecurity) does **not** yet expose PQC algorithms; Google states it will add them to all NIST PQC standards but has not committed to a date.

Enum values (from the public `CryptoKeyVersionAlgorithm` proto):

- Signatures: `PQ_SIGN_ML_DSA_44`, `PQ_SIGN_ML_DSA_65`, `PQ_SIGN_ML_DSA_87`, plus `_EXTERNAL_MU` variants, `PQ_SIGN_SLH_DSA_SHA2_128S`, `PQ_SIGN_HASH_SLH_DSA_SHA2_128S_SHA256`
- KEMs: `ML_KEM_768`, `ML_KEM_1024`, `KEM_XWING` (the hybrid X25519 + ML-KEM-768 construction)

Python client example (ML-DSA-65 signing):

```
from google.cloud import kms_v1

client = kms_v1.KeyManagementServiceClient()
parent = client.key_ring_path("PROJECT_ID", "us-central1", "my-keyring")

crypto_key = kms_v1.CryptoKey(
    purpose=kms_v1.CryptoKey.CryptoKeyPurpose.ASYMMETRIC_SIGN,
    version_template=kms_v1.CryptoKeyVersionTemplate(
        protection_level=kms_v1.ProtectionLevel.SOFTWARE,
        algorithm=kms_v1.CryptoKeyVersion.CryptoKeyVersionAlgorithm.PQ_SIGN_ML_DSA_65,
    ),
)
created = client.create_crypto_key(
    parent=parent,
    crypto_key_id="ml-dsa-65-signing-key",
    crypto_key=crypto_key,
)

# Sign
version = f"{created.name}/cryptoKeyVersions/1"
digest = {"sha512": b"..." }  # or use MessageType=RAW via sign_blob
signature = client.asymmetric_sign(name=version, digest=digest).signature
```

KEM workflow uses the new `Encapsulate` and `Decapsulate` RPCs (and `gcloud kms decapsulate` in the CLI):

```
# Retrieve the public key in raw NIST-PQC format
gcloud kms keys versions get-public-key 1 \
   --key my-mlkem-key --keyring my-ring --location us-central1 \
   --output-file ml-kem.pk --public-key-format nist-pqc

# Client-side encapsulation (library call) then:
gcloud kms decapsulate --version 1 --key my-mlkem-key \
   --keyring my-ring --location us-central1 \
   --ciphertext-file ct.bin --shared-secret-file ss.bin
```

Google recommends **X-Wing over pure ML-KEM** for general use — the hybrid construction protects against both quantum attacks and any future cryptanalytic surprise against ML-KEM.

Chrome and TLS. Chrome has shipped `X25519MLKEM768` as default TLS 1.3 key exchange since Chrome 131 (November 2024). Enterprise policy `PostQuantumKeyAgreementEnabled` was the disable switch; it is being removed in Chrome 147 (stable rollout started April 7, 2026). Adoption: ~57% of browser-initiated HTTPS connections observed at Cloudflare's edge in early 2026 (per `pq.cloudflareresearch.com`), Chrome being the dominant source. When a Chrome client talks to a GCP external HTTPS load balancer, the handshake is automatically hybrid — no server configuration needed.

Google internal (ALTS) uses ML-KEM as of late 2025 (migrated from NTRU-HRSS which was deployed November 2022). Google's company-wide PQC migration target was moved up to 2029 in March 2026.

11.9.4 Microsoft Azure Integration

The short version: **Azure's data-plane KMS services do not expose PQC as of April 2026**. Microsoft's August 2025 "Quantum Safe Security Progress" post places Key Vault, Managed HSM, Entra signing, and related services in Phase 2 (starting 2026), with no concrete GA date announced.

What Microsoft *has* shipped (November 2025):

- **Windows CNG** (Cryptography Next Generation) — GA on Windows Server 2025 and Windows 11 24H2/25H2. Algorithm identifiers: `BCRYPT_MLKEM_ALGORITHM`, `BCRYPT_MLDSA_ALGORITHM`, `BCRYPT_SLHDSA_ALGORITHM`, `BCRYPT_LMS_ALGORITHM`, `BCRYPT_XMSS_ALGORITHM`. Parameter sets: `BCRYPT_MLKEM_PARAMETER_SET_512/768/1024`.
- **.NET 10** — `System.Security.Cryptography.MLKem` (GA), `MLDsa`, `SlhDsa`, `CompositeMLDsa` (marked `[Experimental]` under `SYSLIB5006`).

- **SymCrypt** — ML-KEM and ML-DSA in the public library (versions 103.5.0–103.11.0). SLH-DSA implementation status in SymCrypt is unclear from the public changelog.
- **Windows Schannel hybrid TLS** — preview only (Windows Insider builds). Windows 11 24H2 stable does **not** negotiate hybrid TLS by default.

CNG ML-KEM-768 key generation (C):

```
#include <bcrypt.h>

BCRYPT_KEY_HANDLE hKeyPair;
BCryptGenerateKeyPair(BCRYPT_MLKEM_ALG_HANDLE, &hKeyPair, 0, 0);
BCryptSetProperty(hKeyPair, BCRYPT_PARAMETER_SET_NAME,
    (PUCHAR)BCRYPT_MLKEM_PARAMETER_SET_768,
    sizeof(BCRYPT_MLKEM_PARAMETER_SET_768), 0);
BCryptFinalizeKeyPair(hKeyPair, 0);
// Then use BCryptEncapsulate / BCryptDecapsulate
```

.NET 10 equivalent:

```
using System.Security.Cryptography;

using (MLKem key = MLKem.GenerateKey(MLKemAlgorithm.MLKem768))
{
    string publicKeyPem = key.ExportSubjectPublicKeyInfoPem();
    // Encapsulate/decapsulate via key.Encapsulate/Decapsulate
}
```

SymCrypt FIPS 140-3 status: no CMVP certificate covering PQC algorithms under Microsoft's name is listed as of April 2026. The algorithm implementation is in the library; the validation is in progress. Compare this to AWS-LC-FIPS v3.0 (submitted with ML-KEM in boundary) — AWS moved first on this.

Practical implication for Azure users: if you need PQ-protected data in transit or PQ signatures *today*, you will need to either use Windows/.NET client-side primitives against a non-Azure-managed key, or wait for the Azure Phase 2 rollout. The symmetric AES-256 keys in Key Vault (marketed as "quantum-resistant per CNSA 2.0") are fine for data at rest, but the RSA-OAEP/AES-KW key wrap used by customer-managed keys is classical and must migrate once Azure exposes PQ KEMs.

11.9.5 Other Vendors — Quick Reference

HashiCorp Vault Enterprise 1.19+ introduced ML-DSA sign/verify in the Transit engine (experimental). Recommended usage pattern:

```
vault secrets enable transit
vault write -f transit/keys/pqc-signing-key type=ml-dsa parameter_set=65

# Sign
vault write transit/sign/pqc-signing-key \
  input=$(base64 <<< "message to sign")

# Verify
vault write transit/verify/pqc-signing-key \
  input=$(base64 <<< "message to sign") \
  signature=vault:v1:...
```

Not recommended for production until HashiCorp removes the experimental label.

Thales Luna HSM firmware 7.9+ (June 2025) supports ML-KEM, ML-DSA, SLH-DSA, plus stateful LMS/HSS via native PKCS#11 extensions:

```
/* PKCS#11 with Thales Luna extensions - ML-DSA-65 key pair */
CK_MECHANISM mech = { CKM_ML_DSA_KEY_PAIR_GEN, NULL_PTR, 0 };
CK_ULONG paramSet = CKP_ML_DSA_65;
CK_ATTRIBUTE pubTempl[] = {
    { CKA_CLASS, &pubClass, sizeof(pubClass) },
    { CKA_KEY_TYPE, &keyType, sizeof(keyType) },
    { CKA_PARAMETER_SET, &paramSet, sizeof(paramSet) }
};
C_GenerateKeyPair(hSession, &mech, pubTempl, 3,
                  privTempl, N, &hPub, &hPriv);
```

Classical FIPS 140-3 Level 3 certification is current; PQ-firmware FIPS 140-3 submission is in progress.

Entrust nShield firmware 13.8 (August 2025) and **Utimaco u.trust Quantum Protect** (launched April 2025) similarly offer native PQC in firmware. Both have CAVP algorithm validation but no completed FIPS 140-3 module certificate covering the PQ firmware as of April 2026.

Fortanix DSM and **IBM Hyper Protect Crypto Services** support PQC algorithms but have been on a slower FIPS validation track (FIPS 140-2 Level 3/4 classical; no 140-3 w/ PQC yet).

Edge TLS (Cloudflare / Akamai) — neither is a KMS in the traditional sense, but both terminate TLS for very large fractions of Internet traffic. Cloudflare has `X25519MLKEM768` enabled by default since June 2025 for client-to-edge, default-on for edge-to-origin since January 31, 2026. Akamai reached general default availability in Q1 2026. When your backend sits behind Cloudflare or Akamai, you're already getting hybrid PQ for the client-facing leg.

11.9.6 Standards and Drafts to Track

The migration moves faster than any single reference book can keep up with. Follow these:

Finalized

- **FIPS 203** (ML-KEM), **FIPS 204** (ML-DSA), **FIPS 205** (SLH-DSA) — August 2024
- **NIST SP 800-208** — Stateful hash-based signatures (LMS/XMSS)
- **NIST SP 800-227** — Recommendations for KEM key establishment (finalized September 18, 2025). Provides general KEM properties (IND-CCA, binding, reusability) plus explicit guidance on multi-algorithm KEMs and PQ/T hybrids; discusses X-Wing as an example. **Authoritative reference for ML-KEM deployment**.
- **RFC 9794** — Terminology for post-quantum/traditional hybrid schemes (June 2025)
- **RFC 9882** — ML-DSA in CMS (2025) — **explicitly forbids HashML-DSA in CMS / X.509 PKIX**; use pure ML-DSA only.
- **CA/B Forum Ballot SMC013** — Adopted August 2025; enables single-algorithm PQC for S/MIME

In progress (April 2026) — *Last verified: April 2026 (v1.0); IETF drafts change on a 6-week cadence. Check datatracker.ietf.org for current versions.*

- **NIST IR 8547** — Transition timeline; RSA/ECC deprecated 2030, disallowed 2035
- **NIST SP 800-57 Part 1 Rev. 6** — Key management with PQC (IPD; comments closed Feb 2026)
- **NIST SP 1800-38** volumes B and C — NCCoE PQ Migration practice guide
- **FIPS 206** — FN-DSA / FALCON (IPD August 2025; expected late 2026/2027)
- **FIPS 207** — HQC backup KEM (IPD targeted 2026, Final 2027); see Unit 10.5 for implementation caveats
- **draft-ietf-lamps-pq-composite-sigs-16** — Composite ML-DSA for X.509 and CMS

- **draft-ietf-lamps-pq-composite-kem-14** — Composite ML-KEM for X.509 and CMS
- **draft-ietf-tls-ecdhe-mlkem-04** — `X25519MLKEM768`, `SecP256r1MLKEM768`, `SecP384r1MLKEM1024` in TLS 1.3
- **draft-ietf-sshm-mlkem-hybrid-kex-10** — SSH hybrid KEX with ML-KEM
- **draft-ietf-uta-pqc-app-01** — PQC recommendations for TLS applications
- **draft-ietf-ipsecme-ikev2-mlkem-05** (March 2026) — ML-KEM in IKEv2 for IPsec; Cloudflare has deployed in production interop-tested with strongSwan
- **draft-ietf-hpke-pq** — PQ/T hybrid for HPKE (RFC 9180) — important for MLS and Signal-like protocols
- **draft-ietf-cose-post-quantum-signatures / draft-ietf-jose-pqc-kem** — COSE/JOSE encodings; enables PQ passkeys/WebAuthn
- **draft-ietf-pquip-hybrid-signature-spectrums** — Classification of hybrid signature design goals
- **draft-ietf-plants-merkle-tree-certs** — Merkle Tree Certificates (see Unit 8.6); **PLANTS WG chartered 2026**
- **draft-ietf-tls-key-share-prediction** — Mitigates hybrid key-share misprediction RTT cost
- **draft-sheth-pqc-dnssec-strategy** (April 2026) — SLH-DSA in Merkle Tree Ladder mode for DNSSEC

Regulatory / operational gates

- **NSA CNSA 2.0** — New National Security Systems acquisitions must support CNSA 2.0 by January 1, 2027; CSfC PQC Addendum published 2025
- **CA/B Forum** — Code-signing certificate max lifetime 459 days effective March 1, 2026 (CAs stop issuing longer certs on February 24, 2026); TLS cert max 47 days target by 2029
- **OMB M-23-02** — US federal agencies must maintain prioritized cryptographic inventory (CBOM-ready)
- **FIPS 140-3 transition deadline — September 21, 2026**. FIPS 140-2 validations move to CVMP historical list on that date. As of early 2026, **no HSM vendor had completed FIPS 140-3 Level 3 CMVP validation INCLUDING PQC** — a specific barrier for SWIFT and banking per the G7 Cyber Expert Group's January 2026 report.
- **UK NCSC three-phase roadmap** (March 2025) — Phase 1 (→2028): inventory, build migration plan. Phase 2 (2028–2031): high-priority upgrades. Phase 3 (2031–2035): complete migration.

- **Australia ASD** — PQC transition plan required by end of 2026; legacy asymmetric crypto banned by end of 2030 (earlier than NIST/NSA).
- **EU Coordinated Implementation Roadmap** — Member states begin transition by end 2026; critical infrastructure by end 2030; full system transition 2035. EU Cyber Resilience Act (in force Dec 10, 2024; main obligations Dec 11, 2027) requires state-of-the-art crypto including PQC.
- **Japan CRYPTREC** — ML-KEM external evaluation completed April 2, 2026 (PQShield); national PQC roadmap expected May 2027, full transition target 2035.

Regulatory Compliance Matrix — One-Page Reference (April 2026)

> Last verified: April 2026 (v1.0). Compliance regimes evolve quickly; always confirm against the regulator's primary publication before architectural decisions.

JURISDICTION	FRAMEWORK	KEY DATE(S)	APPLIES TO	REQUIRED ACTION	PENALTY / CONSEQUENCE IF MISSED
USA (Federal)	FIPS 140-3 + NIST IR 8547	**Sept 21, 2026** (140-2 → historical); **2030 deprecate**, **2035 disallowed** RSA/ECC	Cryptographic modules used by USG	Migrate to PQC-capable FIPS 140-3 module; deprecate classical asymmetric per IR 8547 schedule	Ineligibility for federal contracts; FedRAMP authorization withdrawn
USA (NSA)	CNSA 2.0 + CSfC PQC Addendum	**Jan 1, 2027** (new acquisitions); 2030–2035 (full)	National Security Systems	New acquisitions must support CNSA 2.0; full migration by 2030 (most categories) or 2033 (OS) or 2035 (last)	Loss of NSS clearance compliance; product cannot be sold to NSS
USA (Industry)	CA/B Forum (TLS, code signing)	**March 1, 2026** (459-day code-signing); 2029 target (47-day TLS)	Public CAs and their customers	Code-signing certs ≤459 days; plan for 47-day TLS certs by 2029	CAs that don't comply: distrusted by browsers; org's certs invalid
EU	Cyber Resilience Act + Coordinated Implementation Roadmap	**Sept 11, 2026** (CRA reporting); **Dec 11, 2027** (CRA main obligations); 2030 (critical infra); 2035 (full)	Networked products + critical infrastructure	Crypto-agility by Dec 2027; PQC for critical infra by 2030	Product recalls, market bans, fines up to 2.5% global turnover or €15M

JURISDICTION	FRAMEWORK	KEY DATE(S)	APPLIES TO	REQUIRED ACTION	PENALTY / CONSEQUENCE IF MISSED
UK	NCSC 3-Phase Roadmap + IPA endorsement	Phase 1 → 2028 (inventory + plan); Phase 2 (2028–2031, high-priority); Phase 3 (2031–2035, full)	Critical national infrastructure + government supply chain	Phase 1: complete CBOM, build migration plan; Phase 2: high-priority systems; Phase 3: total	Ineligibility for HMG contracts; CNI regulator action
Australia	ASD ISM Guidelines	**End of 2026** (transition plan due); **Dec 31, 2030** (legacy asymmetric banned)	All Australian Government systems + accredited service providers	Documented PQC plan by end 2026; complete migration by 2030 (more aggressive than NIST)	Loss of IRAP certification; ineligibility for government work
Germany	BSI TR-02102-1 (Jan 2025) + Migration Plan	**End-2030**: highest-sensitivity asymmetric encryption; **End-2031**: general asymmetric encryption; **End-2035**: classical signatures (annual TR updates)	Federal IT systems + KRITIS critical infrastructure operators	Follow BSI annual algorithm guidance; migrate KRITIS per BSI staged roadmap	KRITIS operators: regulatory action; gov contracts revoked

JURISDICTION	FRAMEWORK	KEY DATE(S)	APPLIES TO	REQUIRED ACTION	PENALTY / CONSEQUENCE IF MISSED
France	ANSSI 3-Phase Plan (Position Paper Apr 2022, rev. Jan 2024)	**Phase 1 (from 2025)**: defense-in-depth hybrid PQ+classical; **Phase 2 (from 2026)**: hybrid mandatory for new sensitive systems; **Phase 3 (post-2030)**: post-quantum-only permitted	Government, OIV (operators of vital importance)	ANSSI-qualified hybrid (PQ + classical) for sensitive systems through 2030; pure PQC permitted only post-2030	Loss of ANSSI qualification; OIV regulatory penalties
Japan	CRYPTREC	National roadmap May 2027; full transition target 2035	Government, financial, critical infrastructure	Align with national roadmap once published	Gradual adoption pressure (less prescriptive than US/EU)
G7 Financial	G7 Cyber Expert Group January 2026 roadmap	2030–2032 critical financial systems migration	SWIFT, central banks, large global financial institutions	Hybrid PQ for critical financial infrastructure by 2030–2032	Industry/ regulator pressure; potential loss of correspondent-banking relationships
Canada	CSE / ITSM 40.001	2026 federal-systems plan due; transition through 2030s	Government of Canada systems	Federal systems must have PQC plan by 2026	Treasury Board policy non-compliance

How to use this table. Identify your jurisdiction(s) and applicable frameworks. The "Required action" column is your *minimum* obligation — PQC industry best practice (hybrid PQ deployments, complete CBOM, crypto-agility) often exceeds the regulatory floor. For multi-jurisdiction operators (e.g., a global SaaS provider), the *most stringent* deadline drives planning: a US/EU/Australia operator must meet Australia's 2030 ban on legacy asymmetric crypto regardless of NIST's 2035 disallowed date.

Worked example: A US-headquartered cloud provider serving EU customers must:

- US FedRAMP/CNSA 2.0: support PQC for new federal contracts by Jan 2027.
- EU CRA: crypto-agility by Dec 2027; PQC for critical infrastructure by 2030.
- CA/B Forum: code-signing certs ≤459 days starting March 2026.
- → Architecturally: deploy hybrid X25519+ML-KEM-768 by 2026, ML-DSA-65 cert chains by 2027, full PQC for any EU CNI customer by 2030.
- **G7 Cyber Expert Group financial roadmap** (January 2026) — Critical financial systems 2030–2032 target.

Ecosystem Snapshot — Client/Runtime Library Versions (April 2026)

> Last verified: April 2026 (v1.0). Pin minimums to what this table states; check upstream releases for newer fixes (especially security-relevant CVE patches listed in Unit 11.10).

Pin to these or later for native PQC support:

COMPONENT	MINIMUM VERSION	NOTES
OpenSSL	3.5.6 or 3.6.2	Native ML-KEM/ML-DSA/SLH-DSA; includes CVE-2025-15469 fix. 3.5 is LTS to April 2030.
OpenSSH	10.0 (client/server)	`mlkem768x25519-sha256` is default KEX. 10.2+ warns on non-PQ negotiation.
liboqs	0.15.0	Brings `mlkem-native`, AVX-512VL SHA3. Pair with `oqs-provider` 0.10.x (0.11 tracks liboqs 0.15 on main branch).
Bouncy Castle (Java)	1.84+	PQC provider key conversions; prior 1.81 finalized PKCS#8 seed-only + expanded-key formats.
Bouncy Castle (C#)	2.6.1+	Parity with Java; composite signatures.
Go stdlib	1.25+	`crypto/mlkem` stdlib (1.24+); `X25519MLKEM768` default in `crypto/tls`. Go 1.25 (Aug 2025) fixes the FIPS-only mode panic (issue #75528). Current stable: 1.26.
Rustls	0.23+	`X25519MLKEM768` via aws-lc-rs backend; ML-DSA support landed.
Node.js	Planned; track OpenSSL upgrade cadence	Node uses bundled OpenSSL; native PQC arrives when Node ships with OpenSSL 3.5+.
JDK	JDK 24+ (JEP 496, ML-KEM primitive); **JDK 27 EA today** (GA Sep 2026, JEP 527 adds hybrid ECDHE+ML-KEM in TLS 1.3)	ML-KEM primitive usable in current stable JDK via JEP 496; hybrid TLS integration ships with JDK 27 GA in Sep 2026 — early-access available now if you want to validate against your stack.
.NET	10	`System.Security.Cryptography.MLKem` GA Nov 2025; MLDsa/SlhDsa marked experimental.
Windows	11 24H2 / Server 2025	CNG PQC GA Nov 2025 (`BCRYPT_MLKEM_ALGORITHM` etc.); Schannel hybrid TLS still preview.
macOS / iOS	macOS Tahoe 26 / iOS 26 (Sep 2025)	TLS 1.3 PQ keyshare via URLSession/Network framework by default; CryptoKit exposes primitives.
Red Hat Enterprise Linux	10.1	PQ default in OpenSSL 3.5 provider, GnuTLS, NSS, OpenSSH; Linux kernel crypto API still lacks PQC.

COMPONENT	MINIMUM VERSION	NOTES
AWS-LC / AWS-LC-FIPS	3.0	First crypto library with FIPS 140-3 validation INCLUDING ML-KEM (cert in process).
HashiCorp Vault Enterprise	1.19+	ML-DSA sign/verify in Transit engine (experimental); SLH-DSA added later.
sigstore cosign	3.x	Config-driven algorithm selection; ML-DSA signing via KMS BYOK.
Chrome	131+ (enterprise knob disappears in Chrome 147, April 2026)	X25519MLKEM768 default; ~57% of browser-initiated traffic at Cloudflare's edge by early 2026 (see `pq.cloudflareresearch.com`).

11.9.7 Migration Cookbook — Common Scenarios

The following cookbooks apply regardless of which KMS vendor you use. They assume you have already:

1. Completed a cryptographic inventory (see Unit 11.2) and published a CBOM per OWASP CycloneDX 1.6 (see Unit 11.9.8)
2. Enabled hybrid PQ TLS on all control-plane connections (AWS/GCP/Cloudflare done automatically; Azure requires waiting)
3. Identified which data has a confidentiality window longer than the HNDL threat horizon (anything meant to remain secret past ~2030)

Scenario A: Database column encryption (envelope encryption)

Current state: plaintext → AES-256-GCM data encryption key (DEK) → DEK wrapped with RSA-2048 OAEP public key stored in KMS.

Migration pattern (non-breaking, dual-wrap):

1. Enable a PQ KEM key alongside the existing RSA key (ML-KEM-768 or X-Wing on GCP; AWS KMS ML-KEM preview; Thales Luna or Fortanix DSM for self-hosted).
2. Change the wrap function from raw RSA-OAEP to **HPKE (RFC 9180)** so that swapping KEMs later is a one-line change.
3. Dual-wrap every new DEK:

```
wrapped_classical = RSAOAEP.Wrap(RSA_PK, DEK)
wrapped_pq        = HPKE.Seal(MLKEM_PK, info, aad, DEK)
store: (ciphertext, wrapped_classical, wrapped_pq, alg_id)
```

4. On decrypt, prefer `wrapped_pq`, fall back to `wrapped_classical`.
5. After the PQ wrap covers 100% of new writes *and* a re-wrap batch job has converted historical rows, delete the classical wrap column.

Storage impact: ML-KEM-768 ciphertext is 1,088 bytes vs ~256 for RSA-2048. For very large tables do NOT keep wrapped DEKs inline on every row — use a two-level key hierarchy where a single Key Encryption Key (KEK) wraps a batch of DEKs.

Scenario B: TLS server migration

Order of operations matters:

1. **Enable hybrid KEMs first** — no certificate change required. Protects against HNDL immediately:

   ```
   # OpenSSL 3.5+ server
   openssl s_server -key server.key -cert server.crt \
     -groups 'X25519MLKEM768:X25519:P-256' -tls1_3 -accept 8443
   ```

2. **Test middlebox breakage** — the ClientHello grows from ~500 bytes to ~1,600+ bytes with `X25519MLKEM768`. Load balancers, WAFs, DPI engines, and corporate TLS-intercepting proxies frequently truncate or reject. Test with `openssl s_client -connect host:8443 -groups X25519MLKEM768 -msg`.
3. **Then migrate certificates** — options:
   - **Dual-certificate**: present both classical and composite leaves; client picks via `signature_algorithms_cert`.
   - **Composite-only**: works only when every client is PQ-capable (internal services).
   - **Classical cert + hybrid KEM**: acceptable for HNDL confidentiality; does NOT protect against an active attacker with a cryptographically relevant quantum computer. The default "2026 reasonable" posture.

Scenario C: Code signing

For bootloader and immutable firmware, NSA and NIST both prefer stateful hash-based signatures (LMS/XMSS per SP 800-208) over ML-DSA because verifiers can be hash-only. For general code, ML-DSA is appropriate.

USE CASE	RECOMMENDED (2026)
Bootloader / secure boot	LMS or XMSS (stateful)
OS kernel modules (RPM/DEB)	ML-DSA-65 or composite
Windows Authenticode	ML-DSA-65 composite cert (via ADCS; ADCS PQ GA early 2026)
Container images	ML-DSA via `cosign` (sigstore ML-DSA in progress)
Firmware (long-lived)	LMS/XMSS (plan tree depth to match product lifetime)

Critical stateful-signature warning: LMS/XMSS keys are **single-use per leaf**. Losing state means key re-use, which breaks the security assumption. Cloud KMS vendors expose LMS/XMSS carefully and require HSM-backed state tracking.

Scenario D: SSH host + user keys

OpenSSH 10.0 (April 2025) made `mlkem768x25519-sha256` the **default** key exchange. Versions 10.1+ warn when the client selects classical-only KEX.

```
# /etc/ssh/sshd_config - server side
KexAlgorithms mlkem768x25519-sha256,sntrup761x25519-sha512
HostKeyAlgorithms ssh-ed25519,rsa-sha2-512       # host sig still classical in 2026
PubkeyAcceptedAlgorithms ssh-ed25519,rsa-sha2-512
```

Host keys and user keys remain classical (Ed25519, RSA) in 2026 — the SSH WG drafts for PQ host keys are still in progress. Migrate KEX now, hold signatures until the SSHM drafts finalize.

Scenario E: Backup encryption

This is where HNDL bites hardest — retention windows of 5-30 years guarantee classical-encrypted backups will outlive RSA/ECC. Apply the dual-wrap pattern from Scenario A, migrate newest-first, then batch-re-wrap historical archives.

Watch out for backup-engine header formats (Veeam, Commvault, NetBackup) that store wrapped DEKs in fixed-width metadata slots — ML-KEM-wrapped keys are ~1 KB and may not fit. Raise a ticket with your vendor early.

Scenario F: S/MIME email

CA/B Ballot SMC013 (adopted August 2025) permits single-algorithm (non-hybrid) PQC S/MIME certificates to enable CA experimentation. Practical pattern:

1. Deploy PQ *encryption* certs first — HNDL applies to stored email.
2. Keep signing on ECDSA/RSA until PQ S/MIME is ubiquitous at recipient MTAs.

3. For recipients with classical-only clients, dual-send (once classical, once PQ) until Outlook/Apple Mail/Thunderbird handle composite certs uniformly.

PQ-enabled S/MIME certs are available from pilot CAs today; scale availability expected Q3 2026, broad deployment into 2027.

Scenario G: Container image signing

Use `cosign` (sigstore) with KMS-backed ML-DSA keys:

```
# Sign with AWS KMS ML-DSA key
cosign sign --key "awskms:///alias/pqc-sign-mldsa65" image@sha256:...
cosign verify --key "awskms:///alias/pqc-sign-mldsa65" image@sha256:...
```

Enforce via Kyverno / OPA-Gatekeeper admission controller with a `requiredAlgorithm: ML-DSA-65` policy field once policy controllers expose it.

Scenario H: Cryptographic Bill of Materials (CBOM)

CBOM is the foundation — you cannot migrate what you haven't inventoried.

The de-facto standard is **OWASP CycloneDX 1.6** (which absorbed IBM's CBOM spec in 2024). A CBOM records every cryptographic asset (algorithm, protocol, certificate, key, secret), its location, dependencies, parameters, and compliance tags (FIPS, CNSA 1.0, CNSA 2.0).

Tooling as of April 2026:

- **cdxgen** (OWASP, polyglot scanner emits SBOM/CBOM/OBOM) — current mature tool
- **IBM CBOM repository** — reference implementations and sample CBOMs
- **ReversingLabs Spectra Assure** — commercial, CycloneDX 1.6 CBOM support
- **Wiz, Snyk** — crypto-asset discovery in newer scanner versions

CI/CD integration pattern:

```
1. Build:   cdxgen emits SBOM + CBOM alongside artifact
2. Gate:    policy engine (OPA) asserts no algorithmFamily: rsa|ecdsa
            at cryptoSuite level beyond a configured cutoff date
3. Publish: CBOM co-signed with the artifact (sigstore / in-toto)
4. Runtime: register CBOM with central crypto inventory dashboard
```

Scenario I: Capacity planning for PQC handshakes (the cookbook nobody writes)

Hybrid PQC enlarges three things you must budget: ClientHello bytes, KMS QPS, and certificate-chain bytes. Use these envelope numbers when sizing for 2026–2030.

RESOURCE	CLASSICAL BASELINE	+ HYBRID PQC (2026)	PURE PQC (2030+ PROJECTION)	IMPLICATION
TLS 1.3 ClientHello (key share)	~250 B (X25519 only)	~1,500 B (`X25519MLKEM768`)	~1,200 B (ML-KEM-768 only)	Many middleboxes drop fragments at >1.5 KB; raise `tcp_window` and validate path-MTU.
TLS 1.3 ServerHello + cert chain	~3 KB (RSA-2048 leaf+intermediate)	~14–17 KB (ML-DSA-65 composite cert chain w/ intermediates)	~6 KB (MTC inclusion proofs, see Unit 8.6)	Round-trips: with cwnd 10 (Linux default) you blow through initial congestion window — the handshake takes an extra RTT. Bump initcwnd to 20–30 if your PQ rollout shows latency regressions.
KMS Sign QPS budget (ML-DSA-65)	ECDSA-P256 ≈ 5,000 QPS/HSM partition	~600–1,200 QPS/ HSM partition (5–8× more cycles)	Same	Plan to **5–8×** the HSM count for sign-bound workloads (mTLS auth servers, code-signing CIs). Verify is unaffected (~classical cost).
KMS Decap QPS budget (ML-KEM-768)	ECDH-P256 ≈ 8,000 QPS/HSM partition	~3,000 QPS/HSM partition (~3× more cycles)	Same	Less painful than Sign but still budget conservatively for first migration cohort.

RESOURCE	CLASSICAL BASELINE	+ HYBRID PQC (2026)	PURE PQC (2030+ PROJECTION)	IMPLICATION
WAN RTT budget (extra hop)	1 RTT for TLS 1.3	Same (no protocol-level extra round-trip)	Same	But the handshake **bytes** are larger — bandwidth-delay-product matters. On a 200 ms RTT link, classical handshake ≈ 16 KB BDP; hybrid handshake ≈ 30 KB BDP. Saturates a 1.2 Mb/s link briefly.
Cert renewal frequency	ECDSA: ~12 months avg	Composite: same; PQ-only: same	Drops to 47 days by 2029 (CA/B trajectory)	47-day rotation requires fully automated ACME pipelines. Plan the automation now, not in 2028.
Backup ciphertext overhead	RSA-OAEP wrap ≈ 256 B	ML-KEM wrap ≈ 1,088 B; X-Wing wrap ≈ 1,120 B	Same	If your backup tool stores wrapped DEKs in a fixed-width header, raise this with the vendor before Q3 2026.

Sizing cheat sheet (rough rules of thumb for SRE planning):

- **Public-facing TLS termination (Cloudflare/edge):** No additional capacity needed in 2026 — edge nodes do PQ termination, origin connections can stay classical until you're ready. Plan to add ~10–20% origin handshake CPU when you flip origin TLS to PQ in 2027–2028.
- **Internal mTLS mesh (Istio/Linkerd/Consul):** Sign QPS dominates. If your service mesh sees 50K req/s and each request requires one mTLS handshake-on-resume, you do ~2K full handshakes/s — within a single HSM partition for ECDSA but at the edge for ML-DSA. Add a second HSM partition or migrate to session-resumption-heavy patterns first.

- **CI/CD code signing (cosign + KMS):** Verify is fast; sign is slow. Centralize signing in a sidecar that batches; do not call `kms:Sign` per artifact in a tight loop. AWS KMS ML-DSA pricing is the same per-op as classical but you'll do fewer ops/sec.
- **Database envelope encryption:** Wrap once (per DEK), unwrap on read. ML-KEM decap is ~3× ECDH cost — for read-heavy DBs, this can dominate KMS spend. Use a two-level KEK→DEK hierarchy (Scenario A) and cache unwrapped DEKs per process.
- **VPN concentrators (IPsec IKEv2):** IKEv2 PQ KEM (per `draft-ietf-ipsecme-ikev2-mlkem-05`) doubles the IKE handshake size. For VPN aggregators handling 10K+ tunnel rekeys/hour, profile before/after; some hardware accelerators (Cavium, Mellanox) lack PQ offload.

Worked example — sizing a 2026 migration:

> SaaS provider, 50K monthly active users, 2K req/s peak, public TLS terminated at Cloudflare, internal mTLS mesh on AWS, KMS-backed signing for receipts and audit logs.
>
> *Edge:* Zero changes — Cloudflare flips PQ KEM transparently. Confirm origin still happy with `X25519MLKEM768` arriving from Cloudflare's CF→origin tunnel (it is, as of late 2025).
>
> *Internal mesh:* 2K req/s × ~30% handshake-per-req (resume rate after PQ rollout drops session-ticket reuse rates initially) = ~600 handshakes/s. ML-DSA-65 verify is fast (~classical cost) so verify-side is fine; sign-side (only your private CA, not per-request) is unchanged. **No HSM capacity bump needed.**
>
> *KMS:* Receipts and audit logs sign at ~50 ops/s peak. ML-DSA-65 sign budget is ~1K QPS/HSM partition; comfortably under. **No bump needed.**
>
> *DB envelope encryption:* 5K reads/s, each unwrapping a DEK. ECDH unwrap = 5K × 100 µs = 500 ms/s of HSM time. ML-KEM unwrap = 5K × 300 µs = 1.5 s/s — **fits in one HSM partition with 50% margin**, but you've used most of your headroom. Cache unwrapped DEKs per app process for 60s to drop unwraps 10×.
>
> ---
>
> Bottom line:
>
> This shape of workload absorbs the 2026 migration without infra changes. The pain shows up in higher-throughput shapes — mTLS-heavy meshes (>5K handshakes/s), code-signing pipelines, and VPN concentrators.

Scenario J: Stripe Webhook PQC migration (worked example, end-to-end)

Webhook signing is the canonical "small but high-stakes" PQC migration: a third party (Stripe, GitHub, Slack) sends you a signed payload; you verify; you act. Today nearly all webhook-signing schemes use HMAC-SHA256 (symmetric, not affected by Q-Day) — but a growing number use asymmetric (ECDSA, Ed25519) and those are PQ-vulnerable.

Stripe's webhook signing today is HMAC-SHA256 (`stripe-signature` header) — it does NOT need PQC migration for the Q-Day threat (HMAC is post-quantum-secure under Grover with doubled key length, which Stripe already provides at 256-bit). **What follows is illustrative for asymmetric webhook schemes** (e.g., GitHub App webhooks signed with `RS256` JWS, or any webhook moving from `RS256` / `ES256` JWS to ML-DSA).

Migration pattern (illustrative for an asymmetric-signed webhook):

1. **Discovery.** Add the webhook endpoint to your CBOM (Scenario H). Note the current algorithm (`alg`: RS256, ES256, EdDSA) and the JWKS URL of the sender.
2. **Dual-verify period.** Sender publishes both classical and PQ public keys at JWKS endpoint. Receiver:

```
import jwt, base64
from cryptography.hazmat.primitives.asymmetric import ed25519
from oqs import Signature  # PQ via liboqs Python bindings

def verify_webhook_sig(payload, sig_header, jwks):
    header = decode_jws_header(sig_header)
    kid = header['kid']; alg = header['alg']
    jwk = jwks_lookup(jwks, kid)
    if alg == 'ML-DSA-65':
        pk = jwk['x'].encode()  # raw 1952B bytes, base64url-decoded
        sig = base64url_decode(extract_sig(sig_header))
        with Signature("ML-DSA-65") as sigverify:
            if not sigverify.verify(payload, sig, pk):
                raise InvalidSignature
    elif alg == 'EdDSA':
        pk = ed25519.Ed25519PublicKey.from_public_bytes(jwk['x'].encode())
        pk.verify(sig, payload)
    else:
        raise UnsupportedAlg(alg)
```

3. **Cutover.** Sender publishes only the ML-DSA `kid` in JWKS. Receivers that haven't migrated start failing — coordinate via API version or deprecation header (e.g., `Stripe-Version: 2027-01-15-pq`).

4. **Rollback gate.** Keep classical verification code path live for 30 days post-cutover; gated behind a config flag in case interop bugs surface (FIPS-204 spec violations have been found in libcrux/hpke-rs, see Unit 11.11). Roll back via flag-flip without code deploy.
5. **Decommission.** After 30+ days clean, delete classical verifier and rotate signing keys at the sender.

Common pitfalls observed in 2025–2026 webhook PQC pilots:

- **JWS header bloat.** ML-DSA-65 sig is 3,309 B — base64url-encoded that's ~4,400 B in the JWS. If your reverse proxy has a 4KB header limit (nginx default), you'll silently drop large payloads. Raise `large_client_header_buffers`.
- **Replay-window timing.** Classical `RS256` verify is ~50 µs; ML-DSA-65 verify is ~400 µs in pure Python (via liboqs), 100 µs in C. If your replay window is ≤ 5 ms (e.g., Stripe's anti-replay window), you have headroom; if your design called for sub-ms verify, profile first.
- **Detached signature length not in spec.** Some webhook senders (legacy systems) pack the signature into a fixed-size header field (`X-Sig` truncated at 1024 bytes). ML-DSA does not fit. Talk to vendor before cutover.

Scenario K: X.509 Hybrid Cert CLI Recipe (openssl + oqs-provider, end-to-end)

This is the hands-on cheat sheet for building a hybrid PQ certificate today. Tested with OpenSSL 3.5.6 + oqs-provider 0.10.0 + liboqs 0.15 on Linux (April 2026). Composite signature spec: `draft-ietf-lamps-pq-composite-sigs-16`.

```
# ------------------------------------------------------------
# Pre-req: OpenSSL 3.5+ with oqs-provider loaded
# ------------------------------------------------------------
# Confirm the provider is visible:
openssl list -providers
# Should show: default, oqsprovider

# ------------------------------------------------------------
# Step 1 — Generate a composite ML-DSA-65 + Ed25519 root CA key
# ------------------------------------------------------------
openssl genpkey -algorithm mldsa65_ed25519 \
    -out root-ca.key.pem \
    -provider oqsprovider -provider default

# ------------------------------------------------------------
# Step 2 — Self-sign the root CA (composite signature)
# ------------------------------------------------------------
openssl req -new -x509 -nodes \
    -key root-ca.key.pem \
```

```
    -out root-ca.crt.pem \
    -days 3650 \
    -subj "/CN=Acme PQ Root CA 2026/O=Acme/C=US" \
    -provider oqsprovider -provider default

# Inspect: should show signatureAlgorithm = id-MLDSA65-Ed25519
openssl x509 -in root-ca.crt.pem -text -noout | grep -A2 "Signature Algorithm"

# ------------------------------------------------------------
# Step 3 — Issue a leaf cert (server)
# ------------------------------------------------------------
openssl genpkey -algorithm mldsa65_ed25519 \
    -out leaf.key.pem \
    -provider oqsprovider -provider default

openssl req -new -nodes \
    -key leaf.key.pem \
    -out leaf.csr \
    -subj "/CN=api.example.com/O=Acme/C=US" \
    -addext "subjectAltName=DNS:api.example.com,DNS:*.api.example.com" \
    -addext "keyUsage=digitalSignature,keyEncipherment" \
    -addext "extendedKeyUsage=serverAuth,clientAuth" \
    -provider oqsprovider -provider default

openssl x509 -req -in leaf.csr \
    -CA root-ca.crt.pem -CAkey root-ca.key.pem \
    -CAcreateserial \
    -out leaf.crt.pem \
    -days 90 \
    -copy_extensions copy \
    -provider oqsprovider -provider default

# ------------------------------------------------------------
# Step 4 — Build the chain bundle (server presents this)
# ------------------------------------------------------------
cat leaf.crt.pem root-ca.crt.pem > leaf-fullchain.pem

# ------------------------------------------------------------
# Step 5 — Verify the chain end-to-end
# ------------------------------------------------------------
openssl verify -CAfile root-ca.crt.pem leaf.crt.pem
# Expected: leaf.crt.pem: OK

# ------------------------------------------------------------
# Step 6 — Test in a TLS server
# ------------------------------------------------------------
openssl s_server -accept 8443 \
    -cert leaf-fullchain.pem -key leaf.key.pem \
    -groups X25519MLKEM768:X25519:P-256 \
    -tls1_3 \
    -provider oqsprovider -provider default

# In another shell:
openssl s_client -connect localhost:8443 \
    -groups X25519MLKEM768 \
    -CAfile root-ca.crt.pem \
    -showcerts \
```

```
    -provider oqsprovider -provider default 2>&1 | grep -E "Server Temp Key|Verification|
Cipher"
# Expected:
#   Server Temp Key: X25519MLKEM768
#   Verification: OK
#   Cipher: TLS_AES_256_GCM_SHA384
```

Common gotchas:

- `openssl req -x509 -addext` **order matters in some 3.5.x builds** — put `-addext` AFTER `-subj`.
- `mldsa65_ed25519` **is the composite algorithm name;** `mldsa65` alone is pure-PQ (no classical fallback). For HNDL-defense use composite; for FIPS-only environments use `mldsa65`.
- **OID conflicts** — early oqs-provider releases used pre-standard OIDs. Composite OIDs were finalized in `draft-ietf-lamps-pq-composite-sigs-16`. If certs from 2024 don't validate, regenerate them with current oqs-provider.
- `-copy_extensions copy` **is required** when issuing the leaf — without it, SANs disappear silently.

To convert to formats other tools accept:

```
# DER
openssl x509 -in leaf.crt.pem -outform DER -out leaf.crt.der
# PKCS#12 bundle (with key)
openssl pkcs12 -export -in leaf-fullchain.pem -inkey leaf.key.pem -out leaf.p12
# JWK (for JWS verifiers — requires manual encode of the composite SPKI)
# (no native openssl tool yet; use a Python helper with cryptography + oqs)
```

Scenario L: PQC Interop Test Matrix (April 2026)

The single most common PQC migration failure mode is interop — version A's `X25519MLKEM768` doesn't talk to version B's. Use this matrix to set expectations before you flip flags in production.

SERVER ↓ / CLIENT →	CHROME 131+	FIREFOX 132+	SAFARI 18.4+	CURL 8.10+ (OQS-PROVIDER)	OPENSSH 10.0+	GO 1.25+ STDLIB	JAVA JDK 24+	.NET 10
OpenSSL 3.5+ (oqs-provider)	✓	✓	✓	✓	n/a	✓	✓	✓
nginx (OpenSSL 3.5+)	✓	✓	✓	✓	n/a	✓	✓	✓
Caddy 2.8+	✓	✓	✓	✓	n/a	✓	⚠ TLS 1.3 only path	⚠ TLS 1.3 only path
Cloudflare edge	✓	✓	✓	✓	n/a	✓	✓	✓
AWS ALB / NLB	✓	✓	✓	✓	n/a	✓	✓	✓
AWS API Gateway	✓	✓	✓	✓	n/a	✓	✓	✓
GCP HTTPS LB (external)	✓	✓	✓	✓	n/a	✓	✓	✓
Azure App Gateway / Front Door	✗ no PQ today	✗	✗	✗	n/a	✗	✗	✗
HAProxy 3.0+ (OpenSSL 3.5+)	✓	✓	✓	✓	n/a	✓	✓	✓
Envoy 1.32+	⚠ depends on BoringSSL build	⚠	⚠	⚠	n/a	⚠	⚠	⚠

SERVER ↓ / CLIENT →	CHROME 131+	FIREFOX 132+	SAFARI 18.4+	CURL 8.10+ (OQS-PROVIDER)	OPENSSH 10.0+	GO 1.25+ STDLIB	JAVA JDK 24+	.NET 10
OpenSSH 10.0 server	n/a	n/a	n/a	n/a	✓ default	n/a	n/a	n/a
Java tomcat (JDK 24 prim only)	⚠ no TLS hybrid in stdlib until JDK 27	⚠	⚠	⚠	n/a	⚠	n/a	⚠

Legend: ✓ negotiates `X25519MLKEM768` and completes handshake. ⚠ works only with explicit configuration / partial support. ✗ no PQ support shipping today.

Common interop break patterns (matching the matrix gaps):

1. **Middlebox ClientHello fragmentation.** TLS 1.3 ClientHello with `X25519MLKEM768` is ~1.6 KB and exceeds the 1-record limit (16 KB is fine; some old middleboxes truncate at MTU). Symptom: classical-only fallback or hard fail after "received" but no ServerHello. Fix: bypass the middlebox or upgrade firmware.
2. **Azure Front Door / App Gateway** does not yet expose PQ groups — even if your origin runs OpenSSL 3.5 with oqs-provider, the AFD termination is classical. Track Azure roadmap for Phase 2 (later 2026).
3. **Envoy + BoringSSL** — BoringSSL has its own PQ shipping cadence; check Envoy build notes for whether your Envoy is BoringSSL or OpenSSL-based.
4. **Java stdlib pre-JDK 27** does ML-KEM as a primitive (JEP 496) but no TLS integration. Use Bouncy Castle 1.84+ with a custom `SSLContext` provider, or wait for JDK 27 GA in Sep 2026 (JEP 527).
5. **OpenSSH versions < 10.0** don't speak `mlkem768x25519-sha256`; fall back to `sntrup761x25519-sha512` or upgrade.

To run your own interop validation, the openquantumsafe.org test matrix publishes nightly builds against the major TLS implementations.

11.9.8 Key Rotation Cadences with PQC

NIST SP 800-57 Part 1 Rev. 6 (IPD, comments closed February 2026) reorganizes cryptoperiod guidance around NIST security categories (Cat 1/3/5) rather than key-size bits. Derived heuristics:

KEY ROLE	SUGGESTED CRYPTOPERIOD	RATIONALE
TLS leaf server cert	47-90 days	CA/B Forum target trajectory (47 days by 2029)
TLS client auth cert	12 months	Human distribution loop
Code signing (ML-DSA)	≤ 459 days	CA/B Forum ballot Feb 23, 2026 cap
Firmware signing (LMS/XMSS)	One-shot per tree; plan depth for product lifetime	Stateful — each leaf single-use
KMS/HSM data-key wrapping root	1-3 years	Hardware-backed, rarely rotated
Database column encryption DEKs	Re-wrap under new KEK rather than re-encrypt	Re-wrapping is cheap even at PQ sizes

Grace period for rollover (industry practice, not standardized): `2 × max_key_age + max_session_age + validation_delay`. For 90-day certs with 24-hour sessions that's ~190-day windows during which both old and new KEM/signature algorithms must be accepted. PQ key distribution is slower than classical because OCSP/CRL responses and certificate chains are 4-10× larger — err on the longer side.

11.9.9 Downgrade Attack Prevention

Hybrid TLS gives *confidentiality* against a harvest-now attacker today, but the server certificate signature is still classical in most 2026 deployments, which means *authentication* against a future quantum adversary depends on the cert chain upgrading too. To ensure clients actually negotiate hybrid (not fall back silently):

1. **Strict policy enforcement** (recommended). OpenSSL client configuration:

   ```
   SSL_CTX_set1_groups_list(ctx, "X25519MLKEM768");   # no classical fallback
   ```

   Drop the classical group list entirely.
2. **HTTPS DNS records (RFC 9460)** can carry required-group metadata. Experimental.

3. **Service mesh mTLS policy** (Istio, Linkerd) can pin required suites per connection.

Cloudflare's `pq.cloudflareresearch.com` reports that as of April 2026 ~57% of browser-initiated HTTPS connections to Cloudflare carry an `X25519MLKEM768` key share (and the share is climbing quarter-over-quarter; this figure is the load-bearing one to cite). The remainder is older classical-only clients or middleboxes that strip PQ groups from the ClientHello.

Unit 11.9 Summary

Cloud KMS integration for PQC in April 2026 is a mosaic: AWS is furthest ahead with GA offerings across KMS/ACM/Private CA/Secrets Manager and AWS-LC-FIPS 3.0 being the first crypto library to bring ML-KEM inside a FIPS 140-3 module boundary. GCP leads on client adoption via Chrome's default hybrid TLS (~57% of browser-initiated traffic per `pq.cloudflareresearch.com`) and has Cloud KMS PQ signing/KEM in preview. Azure trails its hyperscaler peers — Microsoft has GA'd Windows/.NET PQC primitives but not the Azure data-plane services, with Phase 2 rollout targeted for later in 2026. HSM vendors (Luna, nShield, Utimaco, Fortanix) have firmware-level PQC shipping with CAVP validation, but completed FIPS 140-3 module certificates for the PQ firmware are still pending industry-wide.

The migration playbook is less about cryptography and more about inventory (CBOM), choice of hybrid vs composite certs, grace windows for rotation, and policy enforcement to prevent downgrade. Use HPKE (RFC 9180) for envelope encryption to keep KEM-swap as a configuration change. Use `cosign` + KMS-backed ML-DSA keys for artifact signing. Follow NIST IR 8547 timelines: deprecated by 2030, disallowed by 2035. The work to do is largely plumbing — but the plumbing is extensive.

Unit 11.10: PQC CVE Registry and Implementation Pitfalls

A growing catalog of real CVEs, academic attacks, and developer-bites from real PQC deployments. This registry is current as of April 2026; cross-reference CVE databases and library changelogs before taking a dependency.

11.10.1 Confirmed CVEs (by library)

liboqs and oqs-provider

IDENTIFIER	LIBRARY	SEVERITY	SUMMARY
CVE-2024-37305	oqs-provider < 0.6.1	8.2 High	Unchecked `DECODE_UINT32` lengths in hybrid key/signature deserialization → memory corruption on parsing user-supplied composite keys. Non-hybrid PQ keys unaffected. Fix: 0.6.1.
CVE-2024-37880	pq-crystals/kyber ref, liboqs < 0.10.1, PQClean, rustpq/pqcrypto	High (key recovery)	**KyberSlash**: Clang optimizer (versions through 18.x at `-Os` / `-O1` / etc.) converted a constant-time `cmov` into a secret-dependent branch in `poly_frommsg`. End-to-end ML-KEM-512 secret key recovery demonstrated in < 10 min on Raspberry Pi 2 and Cortex-M4 via decapsulation timing. Fix: move conditional into separate compilation unit so compiler can't optimize it. liboqs 0.10.1 (June 2024).
CVE-2024-54137	liboqs < 0.12.0, PQClean HQC	Moderate (NVD/Red Hat; information leakage, no concrete exploit demonstrated)	**HQC reference bug**: mishandled `sigma` (implicit-rejection) field — treated part of secret key as public and used broken comparison during FO check. Malformed ciphertexts returned secrets derived from key material instead of implicit-rejection values. Present since 2023-04-30 HQC spec; lurked 19 months. Fix: liboqs 0.12.0 (Dec 2024).
CVE-2025-48946	liboqs ≤ 0.12.x HQC	Low (design flaw)	Many malformed ciphertexts share the same implicit-rejection KDF input — HQC has weaker KEM binding than ML-KEM. No concrete attack known but enough for NIST to disable HQC by default in liboqs 0.13.0 (May 2025). Awaits HQC team spec update.
CVE-2025-52473	liboqs < 0.14.0 HQC	Moderate	Same class as CVE-2024-37880 but for HQC under Clang 17–20: secret-dependent branches in HQC reference. Full local key recovery PoC. Fix: liboqs 0.14.0 disables compiler optimizations for HQC sources.

OpenSSL native PQC

IDENTIFIER	LIBRARY	SEVERITY	SUMMARY
CVE-2025-15469	OpenSSL 3.5/3.6 `openssl dgst` CLI	Moderate	`dgst` buffer truncates input > 16 MB for one-shot algorithms (ML-DSA, Ed25519/448). No error reported; signature appears valid but covers only first 16 MB. **Library callers unaffected** — CLI-only. Fix: 3.5.6 / 3.6.2 (April 2026 security update).
CVE-2026-31790	OpenSSL 3.x RSA-KEM	Moderate	RSASVE encapsulation leaks uninitialized memory to peer. Not PQC-specific but frequently co-deployed in composite hybrids. Fix: 3.0.20, 3.3.7, 3.4.5, 3.5.6, 3.6.2.

Go / Rust / Microsoft

IDENTIFIER	LIBRARY	SEVERITY	SUMMARY
Go issue #75528	Go 1.24+ `crypto/tls`	High (panic)	`X25519MLKEM768` listed as FIPS-approved in `defaults_fips140.go`, but `crypto/ecdh/x25519.go` blocks all X25519 in `GODEBUG=fips140=only`. Hybrid KEX internally calls `ecdh.X25519()` → handshake panics. Any FIPS-only Go TLS fails. Fixed in Go 1.25-era patches.
RustSec 2026-01-27	A Rust ML-DSA crate	TBD	Timing leak in ML-DSA Decompose operation.

Vendors/libraries with no published PQC CVEs as of April 2026: AWS-LC/AWS-LC-FIPS (aws-lc-verification has held up), BoringSSL/Chrome PQC path, Microsoft SymCrypt/CNG (GA Nov 2025), Rustls ML-KEM (via aws-lc-rs), Go stdlib `crypto/mlkem`, Bouncy Castle, Sigstore cosign ML-DSA integration, Cloudflare CIRCL, pq-crystals/dilithium reference.

11.10.2 Microarchitectural and Fault Attacks (academic, practical)

ATTACK	TARGET	PUBLICATION	IMPACT
KyberSlash1/2	Kyber/ML-KEM Compress_d DIV	2024, CVE-2024-37880	Full key recovery in minutes (Pi 2) to hours (Cortex-M4)
GoFetch	Apple M1/M2/M3 Data Memory-dependent Prefetcher	March 2024	Kyber + Dilithium keys extracted by co-resident userspace code in 49min–15h. **M1/M2 cannot disable DMP** — effectively unmitigable without kernel patches. M3 has a user-space knob.
PQ-Hammer	Kyber, BIKE, Dilithium via Rowhammer	IEEE S&P 2025	Full key recovery on commodity DDR4/DDR5 by memory spray + hammer + massaging.
SLasH-DSA	SLH-DSA in OpenSSL 3.5.1	USENIX 2025	**Universal forgery** via software-only Rowhammer. Deterministic mode forged in ~1 hour, randomized in ~8 hours. OpenSSL declined fix: "fault attacks outside threat model."
Crowhammer	FALCON/FN-DSA reference (future FIPS 206)	CRYPTO 2025	**Single targeted bit-flip** in RCDT gaussian-sampler table → signing-key recovery after ~100M signatures. More flips lower signature requirement.
GPUHammer / cuPQC key leakage	NVIDIA GDDR6 GPUs	July 2025	First Rowhammer on NVIDIA GPUs; extends to cuPQC ML-KEM key leakage. Mitigation: enable ECC (`nvidia-smi -e 1`), 10% perf and 6.25% memory cost.
Hedged ML-DSA fault attack	Hedged ML-DSA on Cortex-M4	CHES 2024 (ePrint 2024/238)	Voltage glitch skips seed derivation → signature leaks `s1` directly. **~53% single-trace success rate**. Takeaway: hedged mode ≠ automatically fault-resistant.
Correction Fault on ML-DSA-44	ML-DSA-44	CHES 2024 (Krahmer et al.)	Key recoverable with 1024 faulty signatures (512 with lattice reduction).

ATTACK	TARGET	PUBLICATION	IMPACT
Mind the Faulty Keccak	ML-KEM and ML-DSA — all phases	ePrint 2024/1522	EMFI loop-abort on Keccak achieves 89.5% fault success on Cortex-M0+/M3/M4/M33 → key recovery, forgery, verification bypass.
SPA chosen-CT on ML-KEM	pqm4 reference on Cortex-M4	ePrint 2024/2051 + 2025 refinement	Full key recovery in ~30 seconds; 2025 extension needs only 179 traces. Works against a shuffled countermeasure.
Rejection Side-Channel on ML-DSA	ML-DSA on Cortex-M4	ePrint 2025/582 + 2026/056	Novel attack family: leak from **both accepted AND rejected** signing trials. ~300 traces (10× fewer than prior art); 2026 variant uses public templates (non-profiling).
Single-trace ML-KEM decoding	ML-KEM-512 INTT+decoder	ACNS 2025 workshops	Single-trace chosen-CT SPA with ML-assisted classification; full long-term secret recovery.

11.10.3 Specification Ambiguities That Caused Real Interop Failures

1. **ML-KEM / ML-DSA private key format fragmentation** — FIPS defines expanded private keys, IETF prefers 64-byte seeds (d, z), PKCS#11 has its own expectations. OpenSSL 3.5, Bouncy Castle Java 1.81, C# 2.6.1 only aligned formats in mid-2025. Keys generated by one library frequently unimportable by another until then.
2. **ML-DSA pure vs. HashML-DSA mode confusion** — FIPS 204 §5.4 defines HashML-DSA with a different `Verify()` routine. **RFC 9882 (2025) explicitly forbids HashML-DSA in CMS / X.509 PKIX**. Early implementations confused the two; signers used pre-hash, verifiers used pure → silent verification failures.
3. **ML-DSA externalMu vs. internal mu** — FIPS 204 permits external μ computation but FIPS certification discourages it. Library APIs differed (compute-μ vs. accept-μ); interop failures on the pqc-forum.
4. **SampleNTT rejection sampling buffer bounds** — A malformed ML-KEM key can force > 575 bytes of XOF output (normally probability $\sim 2^{-38}$). Implementations with bounded XOF buffers fail on crafted keys.

5. **Non-constant `memcmp` / `strcmp` in FO ciphertext check** — Decapsulation compares re-encrypted ciphertext for implicit rejection. `strcmp` stops at first zero byte (accepts some malformed CTs); `memcmp` is non-constant-time. Both seen in early pedagogical implementations.
6. **TLS key-share misprediction round-trip cost** — Clients caching a classical named group and then trying hybrid resume advertise the hybrid group but send classical key share → HelloRetryRequest costs extra RTT. Addressed by `draft-ietf-tls-key-share-prediction`.

11.10.4 Recurring Developer Pitfalls (no CVE, but real bugs)

PITFALL	BITTEN BY	FIX
Secret-dependent integer division (Compress_d)	Kyber ref, pqm4, Rust pqcrypto, PQClean	Replace with multiplication-by-reciprocal
Compiler optimizes `cmov` → branch (Clang through 18.x for Kyber; 17–20 for HQC)	Any C code not using inline asm cmov	Segregate constant-time code to separate TU; compile with `-fno-tree-ch -fno-if-conversion`; verify via `objdump -d`
Missing modulus check on imported ML-KEM public key	pq-crystals/kyber HEAD	Add bounds validation per FIPS 203 §7.2
`int32_t` polynomial multiplication overflows (256 × 3328² = 2.84B > INT32_MAX)	Many custom educational implementations	Use `int64_t` accumulator
Missing `#include <stdlib.h>` → implicit-declaration `malloc` returning `int` truncates pointer on 64-bit	Tutorial ports	Always include standard headers; use `-Werror=implicit-function-declaration`
Using `openssl dgst` CLI for ML-DSA over large files	CVE-2025-15469	Use library API or OpenSSL ≥ 3.5.6 / 3.6.2
FIPS-only Go TLS with hybrid enabled	Go issue #75528	Update to Go 1.25+ patch release

11.10.5 Using This Registry

- **Before deployment**: grep your dependency tree for affected library versions and upgrade. All liboqs ≤ 0.14 is affected by at least one CVE in this table.

- **For risk assessment**: if your threat model includes co-resident adversaries (shared hypervisors, multi-tenant cloud), treat GoFetch-class DMP attacks as unmitigated on Apple M1/M2 hardware.
- **For hardware deployment**: assume Rowhammer fault-attack threat model for any PQ signature stored in DDR RAM for more than a few minutes. Enable ECC (`nvidia-smi -e 1` or server DIMM ECC); use refresh-rate enforcement where supported.
- **For compiler choice**: prefer GCC or rebuild-from-source Clang for PQC code paths, or audit with ctgrind/dudect after every compiler upgrade — constant-time properties are not portable across compiler versions.

Unit 11.10 Summary

The PQC standards are mathematically sound but the implementation surface is young. Real CVEs clustered in 2024–2026 are concentrated in (a) compiler-introduced timing leaks, (b) HQC reference-implementation bugs (three CVEs in 18 months — HQC gets a disproportionate share), (c) microarchitectural side channels (GoFetch, Rowhammer class), and (d) specification ambiguities bridging FIPS and IETF. **None of these broke the algorithms; all broke implementations of the algorithms.** Maintain a rolling watch of `github.com/open-quantum-safe/liboqs/security`, NIST pqc-forum, and major vendor security advisories; pair it with CBOM-driven dependency tracking so each CVE disclosure immediately surfaces which of your systems are exposed.

Unit 11.11: Formal Verification Lessons — "Verification Theatre"

Formal verification is the most powerful tool available to catch PQC implementation bugs, but it is not a substitute for code review. In February 2026, a paper exposed 13 bugs that had escaped formally verified cryptographic libraries shipped in production, including Signal's post-quantum ratchet.

11.11.1 The Verification Theatre Incident (February 2026)

Nadim Kobeissi's "Verification Theatre: False Assurance in Formally Verified Cryptographic Libraries" (ePrint 2026/192) analyzed Cryspen's libcrux and hpke-rs — both marketed as formally verified, both deployed in Signal's SPQR post-quantum ratchet. Key findings:

- **13 distinct vulnerabilities** inside the "verified" boundary
- **Cross-backend endianness bug** — produced real decryption failures in Signal's post-quantum ratchet
- **Missing mandatory X25519 validation** — RFC 9180 violation in HPKE
- **Nonce reuse via integer overflow** — classical crypto bug in a "verified" implementation
- **Two FIPS 204 specification violations in the ML-DSA verifier**:
  1. A norm check rendered dead code by a wrong constant
  2. A missing final bound check in hint deserialization
- **ML-KEM bugs in verified code**: wrong decompression constant, missing inverse NTT, false serialization proof
- **ML-DSA verified code**: wrong multiplication specification rendered axiomatized AVX2 proofs unsound

The structural issue: **only 58.4% of the ML-KEM deployment code proofs were actually checked** when the library was shipped. The NEON ARM64 backend was entirely "admitted" — zero proofs checked. "Formally verified" covered a much smaller surface than the marketing suggested.

11.11.2 What Formal Verification Does Well

Formal verification works well for:

- **Functional correctness** on specific inputs (e.g., "this function returns the correct low-bits of its argument")
- **Memory safety** within a defined language model (Rust's ownership, WhyML refinement types)
- **Constant-time properties** when the cost model accurately captures the target CPU (not always the case)
- **Protocol-level proofs** (ProVerif, Tamarin) for key-agreement and authentication properties

The **Formosa-Crypto project** (EasyCrypt/Jasmin) delivers formally verified ML-KEM and ML-DSA implementations with security, correctness, *and* constant-time proofs — and crucially, the proof coverage is documented and complete. Reference: the Jasmin-verified reference code and the accompanying EasyCrypt proof scripts.

11.11.3 What Formal Verification Misses

Based on the Kobeissi findings and broader experience:

1. **Cross-backend bugs**: Proofs typically cover one implementation path (reference C or one optimized backend). Endianness, alignment, and integer-width mismatches between backends are outside most proof scopes.
2. **Microarchitectural side channels**: A function can be constant-time in the C abstract machine and leaky on a specific CPU (GoFetch, PREFETCH, speculation). Proofs in the C model don't catch hardware-specific leaks.
3. **Specification transcription errors**: If the formal spec embeds a wrong constant, the code is provably correct against a wrong target. The FIPS 204 norm-check bug above is exactly this class.
4. **Incomplete proof coverage** when some lemmas are "admitted" (assumed without proof) — a human-review check that readers of the verification artifact rarely perform.
5. **Serialization boundaries** — proofs often cover in-memory operations but not byte encoding, where interop failures cluster.
6. **Fault attacks** — almost always outside the threat model.

11.11.4 Recommendations for Using Verified Libraries

1. **Read the verification artifact, not the marketing**. Ask: what proportion of the shipping code is covered by checked proofs? What assumptions are admitted? Which backends are verified?
2. **Treat verification as a *complement* to testing**, not a replacement. Continue to run KAT test vectors, fuzzing, and dudect timing tests against verified implementations.
3. **Track spec-to-code mappings** — at each FIPS spec revision, re-audit whether the formal specification in the proof still matches the current standard.
4. **Prefer verified implementations for the hardest-to-get-right primitives** — NTT, rejection sampling, decomposition — while accepting that higher-level protocol plumbing will need conventional assurance methods.

5. **Cross-verify** — interop-test the verified implementation against multiple reference implementations. A divergence signals an ambiguous spec or a missed proof obligation.

Unit 11.11 Summary

Formal verification is a genuine engineering advance over testing-only assurance, but it is not a silver bullet. The Verification Theatre paper is essential reading for any team adopting verified PQC libraries — it documents realistic failure modes (bugs in "verified" code that shipped in production messaging apps). Use verified implementations where they exist and match your needs, but supplement them with conventional review, testing, CBOM tracking, and timing analysis. The Formosa-Crypto project shows what rigorous PQC verification looks like when done end-to-end.

Module 11 Summary: Migrating Legacy Codebases to PQC

Key Takeaways

1. **Start Now**: The HNDL threat makes immediate action essential—data encrypted today may be decrypted by future quantum computers
2. **Inventory First**: You can't protect what you don't know about—comprehensive cryptographic discovery is the foundation
3. **Build for Agility**: Design systems that can swap algorithms without code changes—this is a permanent requirement
4. **Hybrid Bridge**: Use hybrid cryptography (X25519 + ML-KEM) during migration for defense in depth
5. **Test Thoroughly**: Validate with NIST test vectors, interoperability testing, and production monitoring
6. **Phase the Migration**: Follow global timelines (2030 deprecation, 2035 disallowance) with phased approach
7. **Legacy Systems Require Alternative Strategies**: When direct migration is impossible (dead languages, lost source code, regulatory constraints), use gateway/proxy approaches, HSM offloading, or network-level encapsulation

Migration Checklist:

```
□ Cryptographic inventory completed
□ Risk assessment performed (Y + Z > X analysis)
□ Crypto-agile architecture designed
□ PQC libraries selected and tested
□ Hybrid configuration deployed
□ Testing and validation complete
□ Monitoring and alerting configured
□ Rollback procedures documented
□ Team trained on PQC operations
□ Compliance timeline tracked
```

Connections to Other Modules:

- Module 4: ML-KEM for key encapsulation
- Module 6: ML-DSA for signatures
- Module 7: SLH-DSA as conservative alternative

- Module 8: Hybrid construction patterns
- Module 9: Protocol integration (TLS, SSH)
- Module 10: Future algorithms and agility

What's Next: The Course Conclusion summarizes all modules and provides guidance for continued learning in the evolving post-quantum landscape.

Course Conclusion

This comprehensive PQC learning plan has covered:

Foundations (Modules 1-3):

- Mathematical background for lattice and hash-based cryptography
- Quantum computing threat model

Core Algorithms (Modules 4-7):

- ML-KEM (FIPS 203) - Key encapsulation
- ML-DSA (FIPS 204) - Digital signatures
- SLH-DSA (FIPS 205) - Hash-based signatures

Deployment (Modules 8-9):

- Hybrid cryptography construction
- Protocol integration (TLS, SSH, S/MIME, VPN)

Future Directions (Module 10):

- Emerging algorithms
- Cryptographic agility
- Best practices

Migration (Module 11):

- Cryptographic inventory and discovery
- Risk assessment (HNDL threat model)
- Crypto-agile architecture design
- Library selection and integration
- Phased migration execution
- Testing and validation

The post-quantum transition is underway. Start today.

Alphabetical Index

Quick lookup of key terms and where they are defined or discussed in depth. Use browser find (Ctrl/Cmd+F) or PDF search to locate. Section references use unit numbers (e.g., §2.1 = Unit 2.1) or appendix names.

A–B

C–D

TERM	LOCATION
CAVP (Cryptographic Algorithm Validation Program)	§11.5.5
CBD (Centered Binomial Distribution)	§2.6, §4.2
c̃ (c-tilde, ML-DSA challenge hash)	§6.4, §6.5
Certificate Transparency under PQC	§9.5.5
Classic McEliece	§10.1, Glossary
Cloud KMS PQC integration	§11.9
Cloudflare / Akamai edge TLS	§11.9.5
CBOM (CycloneDX 1.6)	§11.9.7
Composite signatures (X.509)	§11.9.6, §8.3
CNSA 2.0	§1.0 (intro), §11.2
Compress/Decompress (ML-KEM)	§4.2, Glossary
Composite signatures	§8.3, §8.4
Constant-time programming	§2.1.6, Appendix A.1
CRYSTALS-Dilithium (→ ML-DSA)	Glossary, §6.1
CRYSTALS-Kyber (→ ML-KEM)	Glossary, §4.1
CSPRNG	Glossary
Decapsulation	§4.1
Decapsulation key (dk)	§4.1, Glossary
Decomposition (HighBits/LowBits, ML-DSA)	§6.1, §6.5
Digital signature (definition)	§5.1
Discrete Gaussian	§3.2, §10.1 (FALCON)
DNSSEC and PQC	§9.5
DPA (Differential Power Analysis)	§7.7.2, Appendix A.3

E–F

TERM	LOCATION
Encapsulation	§4.1
Encapsulation key (ek)	§4.1, Glossary
Expected Output blocks	§2.1 onward (code sections)
Fault injection	Appendix A.3
Fiat-Shamir transform	§5.2
FIPS 140-3	§11.5
FIPS 203 (ML-KEM)	§4 (entire module)
FIPS 204 (ML-DSA)	§6 (entire module)
FIPS 205 (SLH-DSA)	§7 (entire module)
FIPS 206 (FN-DSA/FALCON draft)	§10.1, Appendix: Quick Reference
FORS (Forest Of Random Subsets)	§7.5
FrodoKEM	§10.2, Glossary
Fujisaki-Okamoto transform	§4.3

G–H

TERM	LOCATION
γ_1, γ_2 (gamma — ML-DSA range bounds)	§6.1, Master Parameter Reference
GCP Cloud KMS / Cloud HSM PQC	§11.9.3
Generator (cyclic group)	§2.2
Grover's algorithm	Intro, §1.0
H (hash function placeholder)	§2, §4, §6, §7
Harvest-now, decrypt-later	Intro, §11.1
HighBits / LowBits (ML-DSA)	§6.1
Hint (ML-DSA signatures)	§6.5, §6.6
HQC (Hamming Quasi-Cyclic)	§10.1, Glossary
HPKE (RFC 9180, envelope encryption)	§11.9.7
HashiCorp Vault Transit PQC	§11.9.5
Hybrid cryptography	§8 (entire module)
Hypertree (SLH-DSA)	§7.4

I–K

TERM	LOCATION
IND-CPA	§4.1
IND-CCA2	§4.1, §4.3
Isogeny-based cryptography	Glossary (SIDH broken 2022)
KAT (Known Answer Test)	§11.5
KDF (Key Derivation Function)	§8.2 (X-Wing)
KEM (Key Encapsulation Mechanism)	§4.1, Glossary
K-PKE	§4.2

L–M

TERM	LOCATION
Lattice (definition)	§3.1
Learning With Errors (LWE)	§3.2
liboqs	§1.2, §11.4
LMS / XMSS (stateful hash signatures)	§7, Bibliography
ML-DSA-44/65/87	§6, Master Parameter Reference
ML-KEM-512/768/1024	§4, Master Parameter Reference
Module-LWE	§3.3
Montgomery reduction	§2.1

N–Q

TERM	LOCATION
NIST PQC Project	§1.0, Bibliography
NTRU / NTRU Prime	§3.3, §10.2
NTT (Number Theoretic Transform)	§2.5
One-Time Signature (OTS)	§7.1, §7.3
OpenSSL 3.5+ / 3.6	§11.4
Orphan (CSS/pagination context)	—
PKE (Public Key Encryption)	§4.2, Glossary
ProVerif / Tamarin	§10.4.5
q (prime modulus)	§2.1, §4, §6

R–S

TERM	LOCATION
RAND_bytes / getrandom / OQS_randombytes	§1.2, §2.6
Rejection sampling	§6.2
Ring-LWE	§3.3
Rng (random number generation)	§2.6, §11.5
SampleInBall (ML-DSA challenge sampling)	§6.2, §6.4
Schnorr identification	§5.2
SHAKE (SHA-3 variants)	§1.2, §2.6, §4, §6
Shor's algorithm	Intro, §1.0
Side-channel attacks	§7.7.2, Appendix A.3
SLH-DSA-SHA2-* / SLH-DSA-SHAKE-*	§7, Master Parameter Reference
sntrup761 (SSH key exchange)	§9.2
Strong unforgeability (SUF-CMA)	§5.1

T–Z

TERM	LOCATION
τ (tau — ML-DSA challenge weight)	§6.1, Master Parameter Reference
TLS 1.3 with PQC	§9.1
WOTS+ (Winternitz One-Time Signature Plus)	§7.3
XMSS (eXtended Merkle Signature Scheme)	§7.4, Glossary
XOF (eXtendable Output Function)	§2.6, Glossary
X-Wing	§8.2, Appendix: Quick Reference
ζ (zeta — primitive root, ML-KEM NTT)	§2.5

Bibliography

This section provides full bibliographic details for works referenced throughout the document. Entries are organized by category and use bracketed citation keys (e.g., [Regev05]) for cross-referencing.

Foundational Papers

[Shor97] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM Review*, vol. 41, no. 2, pp. 303-332, 1999. Originally presented at FOCS 1994. https://doi.org/10.1137/S0036144598347011

[Grover96] L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212-219, 1996. https://doi.org/10.1145/237814.237866

[Regev05] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 84-93, 2005. Extended version in *Journal of the ACM*, vol. 56, no. 6, Article 34, 2009. https://doi.org/10.1145/1568318.1568324

[Ajtai96] M. Ajtai, "Generating Hard Instances of Lattice Problems," *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 99-108, 1996. https://doi.org/10.1145/237814.237838

[LPR10] V. Lyubashevsky, C. Peikert, O. Regev, "On Ideal Lattices and Learning with Errors Over Rings," *Advances in Cryptology -- EUROCRYPT 2010*, LNCS 6110, pp. 1-23, 2010. Extended version in *Journal of the ACM*, vol. 60, no. 6, Article 43, 2013. https://doi.org/10.1145/2535925

[Lyubashevsky12] V. Lyubashevsky, "Lattice Signatures Without Trapdoors," *Advances in Cryptology -- EUROCRYPT 2012*, LNCS 7237, pp. 738-755, 2012. https://doi.org/10.1007/978-3-642-29011-4_43

[FO99] E. Fujisaki, T. Okamoto, "Secure Integration of Asymmetric and Symmetric Encryption Schemes," *Advances in Cryptology -- CRYPTO 1999*, LNCS 1666, pp. 537-554, 1999. https://doi.org/10.1007/3-540-48405-1_34

[HHK17] D. Hofheinz, K. Hovelmanns, E. Kiltz, "A Modular Analysis of the Fujisaki-Okamoto Transformation," *Theory of Cryptography Conference (TCC)*, LNCS 10677, pp. 341-371, 2017. https://doi.org/10.1007/978-3-319-70500-2_12

[Lamport79] L. Lamport, "Constructing Digital Signatures from a One-Way Function," *SRI International Technical Report CSL-98*, 1979.

[Merkle89] R. C. Merkle, "A Certified Digital Signature," *Advances in Cryptology -- CRYPTO 1989*, LNCS 435, pp. 218-238, 1989. https://doi.org/10.1007/0-387-34805-0_21

NIST Standards and Special Publications

[FIPS203] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," *Federal Information Processing Standards Publication 203*, August 2024. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf

[FIPS204] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," *Federal Information Processing Standards Publication 204*, August 2024. https://nvlpubs.nist.gov/nistpubs/fips/NIST.FIPS.204.pdf

[FIPS205] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard," *Federal Information Processing Standards Publication 205*, August 2024. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf

[FIPS206] National Institute of Standards and Technology, "FFT (Fast-Fourier Transform) over NTRU-Lattice-Based Digital Signature Standard," *Federal Information Processing Standards Publication 206 (Initial Public Draft)*, August 2025. https://csrc.nist.gov/projects/pqc-dig-sig (NIST has not yet published a stable URL for the FIPS 206 IPD; the PQC Digital Signature project page links to current draft assets)

[SP800-227] National Institute of Standards and Technology, "Recommendations for Key-Encapsulation Mechanisms," *NIST Special Publication 800-227*, 2025. https://csrc.nist.gov/pubs/sp/800/227/ipd

[IR8547] National Institute of Standards and Technology, "Transition to Post-Quantum Cryptography Standards," *NIST Internal Report 8547*, 2024. https://csrc.nist.gov/pubs/ir/8547/ipd

[SP800-208] National Institute of Standards and Technology, "Recommendation for Stateful Hash-Based Signature Schemes," *NIST Special Publication 800-208*, 2020. https://csrc.nist.gov/pubs/sp/800/208/final

[SP800-56C] National Institute of Standards and Technology, "Recommendation for Key-Derivation Methods in Key-Establishment Schemes," *NIST Special Publication 800-56C Rev. 2*, 2020. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Cr2.pdf

[FIPS202] National Institute of Standards and Technology, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," *Federal Information Processing Standards Publication 202*, 2015. https://nvlpubs.nist.gov/nistpubs/fips/NIST.FIPS.202.pdf

Algorithm Submission Papers

[CRYSTALS-Kyber] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, D. Stehle, "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 353-367, 2018. https://doi.org/10.1109/EuroSP.2018.00032

[CRYSTALS-Dilithium] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehle, "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, vol. 2018, no. 1, pp. 238-268, 2018. https://doi.org/10.13154/tches.v2018.i1.238-268

[SPHINCS+] D. J. Bernstein, A. Hulzing, S. Kolbl, R. Niederhagen, J. Rijneveld, P. Schwabe, "SPHINCS+: Submission to the NIST Post-Quantum Project," NIST PQC Submission, 2017. https://sphincs.org/

[FALCON] T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte, Z. Zhang, "FALCON: Fast-Fourier Lattice-based Compact Signatures over NTRU," NIST PQC Submission, 2017. https://falcon-sign.info/

[HQC] C. Aguilar-Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zemor, "Hamming Quasi-Cyclic (HQC)," NIST PQC Submission, 2017. Selected for standardization March 2025. https://pqc-hqc.org/

[FrodoKEM] M. Naehrig, E. Alkim, J. Bos, L. Ducas, K. Easterbrook, B. LaMacchia, P. Longa, I. Mironov, V. Nikolaenko, C. Peikert, A. Raghunathan, D. Stebila, "FrodoKEM: Learning With Errors Key Encapsulation," NIST PQC Submission, 2017. https://frodokem.org/

[ClassicMcEliece] D. J. Bernstein, T. Chou, T. Lange, I. von Maurich, R. Misoczki, R. Niederhagen, E. Persichetti, C. Peters, P. Schwabe, N. Sendrier, J. Szefer, C.-J. Wang, "Classic McEliece: Conservative Code-Based Cryptography," NIST PQC Submission, 2017. https://classic.mceliece.org/

[NTRU98] J. Hoffstein, J. Pipher, J. H. Silverman, "NTRU: A Ring-Based Public Key Cryptosystem," *Algorithmic Number Theory Symposium (ANTS-III)*, LNCS 1423, pp. 267-288, 1998. https://doi.org/10.1007/BFb0054868

Textbooks and Monographs

[Galbraith12] S. D. Galbraith, *Mathematics of Public Key Cryptography*, Cambridge University Press, 2012. ISBN 978-1107013926.

[BernsteinBuchmannDahmen09] D. J. Bernstein, J. Buchmann, E. Dahmen (eds.), *Post-Quantum Cryptography*, Springer, 2009. ISBN 978-3-540-88701-0. https://doi.org/10.1007/978-3-540-88702-7

[MicciancioGoldwasser02] D. Micciancio, S. Goldwasser, *Complexity of Lattice Problems: A Cryptographic Perspective*, Kluwer Academic Publishers (Springer), 2002. ISBN 978-0-7923-7688-0.

[HoffsteinPipherSilverman08] J. Hoffstein, J. Pipher, J. H. Silverman, *An Introduction to Mathematical Cryptography*, Springer, 2008. ISBN 978-0-387-77993-5.

[Peikert16] C. Peikert, "A Decade of Lattice Cryptography," *Foundations and Trends in Theoretical Computer Science*, vol. 10, no. 4, pp. 283-424, 2016. https://doi.org/10.1561/0400000074

[KatzLindell21] J. Katz, Y. Lindell, *Introduction to Modern Cryptography*, 3rd Edition, CRC Press, 2021. ISBN 978-0815354369.

[BonehShoup23] D. Boneh, V. Shoup, *A Graduate Course in Applied Cryptography*, Version 0.6, 2023. Available online: https://toc.cryptobook.us/

Cryptanalysis and Security Analysis

[MATZOV22] MATZOV, "Report on the Security of LWE: Improved Dual Lattice Attack," MATZOV Technical Report, April 2022. https://zenodo.org/record/6412487

[AlbrechtPlatter19] M. R. Albrecht, R. Player, S. Scott, "On the Concrete Hardness of Learning with Errors," *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169-203, 2015. https://doi.org/10.1515/jmc-2015-0016

[BKZ2.0] Y. Chen, M. Nguyen, "BKZ 2.0: Better Lattice Security Estimates," *Advances in Cryptology -- ASIACRYPT 2011*, LNCS 7073, pp. 1-20, 2011. https://doi.org/10.1007/978-3-642-25385-0_1

[LLL82] A. K. Lenstra, H. W. Lenstra Jr., L. Lovasz, "Factoring Polynomials with Rational Coefficients," *Mathematische Annalen*, vol. 261, pp. 515-534, 1982. https://doi.org/10.1007/BF01457454

[CastryckDecru22] W. Castryck, T. Decru, "An Efficient Key Recovery Attack on SIDH," *Advances in Cryptology -- EUROCRYPT 2023*, LNCS 14008, pp. 423-447, 2023. https://eprint.iacr.org/2022/975

[GrasslLangleyRoetteler16] M. Grassl, B. Langenberg, M. Roetteler, R. Steinwandt, "Applying Grover's Algorithm to AES: Quantum Resource Estimates," *Post-Quantum Cryptography (PQCrypto 2016)*, LNCS 9606, pp. 29-43, 2016. https://doi.org/10.1007/978-3-319-29360-8_3

Implementation and Side-Channel Attacks

[Kocher96] P. C. Kocher, "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems," *Advances in Cryptology -- CRYPTO 1996*, LNCS 1109, pp. 104-113, 1996. https://doi.org/10.1007/3-540-68697-5_9

[Kocher99] P. Kocher, J. Jaffe, B. Jun, "Differential Power Analysis," *Advances in Cryptology -- CRYPTO 1999*, LNCS 1666, pp. 388-397, 1999. https://doi.org/10.1007/3-540-48405-1_25

[Montgomery85] P. L. Montgomery, "Modular Multiplication Without Trial Division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519-521, 1985. https://doi.org/10.1090/S0025-5718-1985-0777282-X

[Barrett86] P. Barrett, "Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor," *Advances in Cryptology -- CRYPTO 1986*, LNCS 263, pp. 311-323, 1987. https://doi.org/10.1007/3-540-47721-7_24

[NTTSeiler18] G. Seiler, "Faster AVX2 Optimized NTT Multiplication for Ring-LWE Lattice Cryptography," *IACR Cryptology ePrint Archive*, Report 2018/039, 2018. https://eprint.iacr.org/2018/039

IETF RFCs and Drafts

[RFC8446] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," *RFC 8446*, Internet Engineering Task Force, August 2018. https://www.rfc-editor.org/rfc/rfc8446

[RFC9180] R. Barnes, K. Bhargavan, B. Lipp, C. Wood, "Hybrid Public Key Encryption," *RFC 9180*, Internet Engineering Task Force, February 2022. https://www.rfc-editor.org/rfc/rfc9180

[RFC9370] C. Tjhai, M. Tober, D. Ounsworth, "Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2)," *RFC 9370*, Internet Engineering Task Force, May 2023. https://www.rfc-editor.org/rfc/rfc9370

[RFC8554] D. McGrew, M. Curcio, S. Fluhrer, "Leighton-Micali Hash-Based Signatures," *RFC 8554*, Internet Engineering Task Force, April 2019. https://www.rfc-editor.org/rfc/rfc8554

[RFC8391] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, A. Mohaisen, "XMSS: eXtended Merkle Signature Scheme," *RFC 8391*, Internet Engineering Task Force, May 2018. https://www.rfc-editor.org/rfc/rfc8391

[I-D.ietf-tls-hybrid] D. Stebila, S. Fluhrer, S. Gueron, "Hybrid Key Exchange in TLS 1.3," *Internet-Draft*, Internet Engineering Task Force, Work in Progress. https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/

[I-D.connolly-cfrg-xwing-kem] D. Connolly, P. Schwabe, D. Stebila, T. Wiggers, "X-Wing: The Hybrid KEM You've Been Looking For," *Internet-Draft*, Internet Engineering Task Force, Work in Progress. https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/

Government and Industry Guidance

[CNSA2.0] National Security Agency, "Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) FAQ," September 2022. https://media.defense.gov/2022/Sep/07/2003071836/-1/-1/0/CSI_CNSA_2.0_FAQ_.PDF

[BSI-PQC] Bundesamt fur Sicherheit in der Informationstechnik (BSI), "Cryptographic Mechanisms: Recommendations and Key Lengths," *BSI Technical Guideline TR-02102-1*, 2024.

[ANSSI-PQC] Agence nationale de la securite des systemes d'information (ANSSI), "ANSSI Views on the Post-Quantum Cryptography Transition," Technical Position Paper, 2024.

[NIST-PQC-Project] National Institute of Standards and Technology, "Post-Quantum Cryptography Standardization Project," 2016-present. https://csrc.nist.gov/projects/post-quantum-cryptography

Open-Source Libraries and Tools

[liboqs] Open Quantum Safe Project, "liboqs: An Open-Source C Library for Quantum-Safe Cryptographic Algorithms," 2016-present. https://github.com/open-quantum-safe/liboqs

[OQS-provider] Open Quantum Safe Project, "oqs-provider: OpenSSL 3 Provider for Quantum-Safe Cryptography," 2021-present. https://github.com/open-quantum-safe/oqs-provider

[pq-crystals-kyber] CRYSTALS Team, "pq-crystals/kyber: Reference and Optimized Implementations of ML-KEM (CRYSTALS-Kyber)," GitHub Repository. https://github.com/pq-crystals/kyber

[pq-crystals-dilithium] CRYSTALS Team, "pq-crystals/dilithium: Reference and Optimized Implementations of ML-DSA (CRYSTALS-Dilithium)," GitHub Repository. https://github.com/pq-crystals/dilithium

[NielsenChuang10] M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2010. ISBN 978-1107002173.

Appendix: Quick Reference Cards

ML-KEM-768 Quick Reference

```
Key Sizes:
  Encapsulation Key (public):  1,184 bytes
  Decapsulation Key (private): 2,400 bytes
  Ciphertext:                  1,088 bytes
  Shared Secret:               32 bytes

Parameters:
  n = 256, k = 3, q = 3329
  η₁ = 2, η₂ = 2
  d_u = 10, d_v = 4

Security: NIST Level 3 (equivalent to AES-192)
```

ML-DSA-65 Quick Reference

```
Key Sizes:
  Verification Key (public):  1,952 bytes
  Signing Key (private):      4,032 bytes
  Signature:                  3,309 bytes

Parameters:
  n = 256, k = 6, l = 5, q = 8,380,417
  η = 4, τ = 49, γ₁ = 2^19, γ₂ = (q-1)/32

Security: NIST Level 3 (equivalent to AES-192)
```

ML-KEM Parameter Comparison

PARAMETER	ML-KEM-512	ML-KEM-768	ML-KEM-1024
NIST Level	1	3	5
Security	~AES-128	~AES-192	~AES-256
n	256	256	256
k	2	3	4
q	3329	3329	3329
η_1, η_2	3, 2	2, 2	2, 2
d_u, d_v	10, 4	10, 4	11, 5
Public Key	800 bytes	1,184 bytes	1,568 bytes
Private Key	1,632 bytes	2,400 bytes	3,168 bytes
Ciphertext	768 bytes	1,088 bytes	1,568 bytes
Shared Secret	32 bytes	32 bytes	32 bytes

ML-DSA Parameter Comparison

PARAMETER	ML-DSA-44	ML-DSA-65	ML-DSA-87
NIST Level	2	3	5
Security	~SHA-256 col.	~AES-192	~AES-256
n	256	256	256
k, l	4, 4	6, 5	8, 7
q	8,380,417	8,380,417	8,380,417
η	2	4	2
τ	39	49	60
γ_1	2^17	2^19	2^19
γ_2	(q-1)/88	(q-1)/32	(q-1)/32
Public Key	1,312 bytes	1,952 bytes	2,592 bytes
Private Key	2,560 bytes	4,032 bytes	4,896 bytes
Signature	2,420 bytes	3,309 bytes	4,627 bytes

SLH-DSA Parameter Comparison (Selected Variants)

PARAMETER	128F	128S	192F	192S	256F	256S
NIST Level	1	1	3	3	5	5
Public Key	32 B	32 B	48 B	48 B	64 B	64 B
Private Key	64 B	64 B	96 B	96 B	128 B	128 B
Signature	17,088 B	7,856 B	35,664 B	16,224 B	49,856 B	29,792 B
Sign Speed	Fast	Slow	Fast	Slow	Fast	Slow
Verify Speed	Fast	Fast	Fast	Fast	Fast	Fast

Note: "f" = fast signing, larger signatures; "s" = small signatures, slower signing

SLH-DSA Hash Function Variants

Each parameter set has both SHAKE and SHA2 variants:

- **SHAKE variants**: SLH-DSA-SHAKE-{128,192,256}{f,s}
- **SHA2 variants**: SLH-DSA-SHA2-{128,192,256}{f,s}

Performance is similar; SHAKE is recommended for new deployments due to consistent security properties.

X-Wing Hybrid KEM Quick Reference

```
Components:
  Classical:     X25519 (Curve25519 ECDH)
  Post-Quantum:  ML-KEM-768

Key Sizes:
  Public Key:    1,216 bytes (32 + 1,184)
  Secret Key:    2,432 bytes (32 + 2,400)
  Ciphertext:    1,120 bytes (32 + 1,088)
  Shared Secret: 32 bytes

Security: NIST Level 3 + classical ECDH security
Combiner: HKDF-based key derivation from both secrets
```

FALCON Quick Reference (FIPS 206 Draft, FN-DSA)

```
FALCON-512:
  Public Key:    897 bytes
  Secret Key:    1,281 bytes
  Signature:     ~666 bytes (average)
  Security:      NIST Level 1

FALCON-1024:
  Public Key:    1,793 bytes
  Secret Key:    2,305 bytes
  Signature:     ~1,280 bytes (average)
  Security:      NIST Level 5

Key Features:
  - Smallest signatures among lattice-based schemes
  - Based on NTRU lattices
  - Requires floating-point arithmetic or careful emulation
  - Variable signature size (average values shown)
```

Algorithm Selection Guide

USE CASE	RECOMMENDED ALGORITHM	NOTES
General KEM	ML-KEM-768	Good balance of security/performance
High Security KEM	ML-KEM-1024	For long-term sensitive data
Resource-Constrained	ML-KEM-512	When space is critical
General Signatures	ML-DSA-65	Good balance, fastest lattice sig
High Security Sig	ML-DSA-87	For critical infrastructure
Conservative Security	SLH-DSA-128s	Hash-only assumption
Hybrid KEX (TLS 1.3)	`X25519MLKEM768`	TLS-specific draft (`draft-ietf-tls-ecdhe-mlkem`) — what Chrome/Cloudflare ship today
Hybrid KEM (general)	X-Wing	Standalone hybrid KEM (`draft-connolly-cfrg-xwing-kem`) — different binding/KDF; not interchangeable with `X25519MLKEM768`
Small Signatures	FALCON-512	When sig size is critical

Algorithm Decision Table (by Requirement Class)

Practitioner cheat sheet for on-the-spot algorithm selection. Cells: ✓ recommended / ⚠ conditional / ✗ avoid. Read by row, applying your most-binding constraint first.

CONSTRAINT	ML-KEM-768	ML-KEM-1024	ML-DSA-65	ML-DSA-87	SLH-DSA-128S	SLH-DSA-128F	FALCON-512
Latency p99 < 5 ms (single op)	✓	✓	⚠ rejection tail	⚠	✗ slow sign	⚠	⚠ slow sign on FPU-less; verify <1 ms with FPU
Bandwidth < 5 KB / handshake	✓ (1.1 KB CT)	⚠ (1.5 KB CT)	⚠ (3.3 KB sig)	✗ (4.6 KB sig)	✗ (7.8 KB sig)	✗ (17 KB sig)	✓ (~666 B sig padded)
Memory < 16 KB stack (Cortex-M0)	⚠ tight (~14.2 KB ref; ~4 KB w/ `mlkem-c-embedded`)	✗	⚠ tight	✗	⚠ ok	✗	⚠ FP overhead
HNDL defense (long-term)	✓	✓ highest pure-PQ	N/A (sig)	N/A	N/A	N/A	N/A
FIPS 140-3 today	✓ FIPS 203	✓ FIPS 203	✓ FIPS 204	✓ FIPS 204	✓ FIPS 205	✓ FIPS 205	✗ FIPS 206 draft
Conservative crypto assumptions	⚠ MLWE	⚠ MLWE	⚠ MLWE	⚠ MLWE	✓ hash-only	✓ hash-only	⚠ NTRU
Code/ firmware signing (long-life)	N/A	N/A	⚠	⚠	✓	✓	⚠
TLS/protocol KEX	✓	✓	N/A	N/A	N/A	N/A	N/A
Cert chain < 10 KB	N/A	N/A	✓ leaf	⚠	✗	✗	✓

Worked examples:

- "*Web TLS server, 5 KB chain budget, FIPS-required, defend HNDL today*" → X-Wing (KEX) + ML-DSA-65 (cert) — only combo meeting all four.
- "*Embedded firmware signing, 30-year retention, Cortex-M4 verifier*" → SLH-DSA-128s (slow sign OK at build time, fast verify on device, hash-only is most conservative for 30-year horizon).
- "*Internal mTLS, latency-sensitive, FIPS optional*" → ML-KEM-768 + Ed25519 today; migrate signature to ML-DSA-65 once latency budget allows.
- "*Resource-constrained IoT (8 KB RAM)*" → SLH-DSA-128s for signing (small key, slow but tolerable for one-shot), ML-KEM-512 for KEM (smallest PQ KEM), accept reduced security category.

Master Parameter Reference (All Algorithms)

One-page lookup consolidating every parameter needed to implement or integrate the three FIPS PQC standards plus X-Wing.

ML-KEM (FIPS 203)

PARAMETER	ML-KEM-512	ML-KEM-768	ML-KEM-1024
NIST security level	1	3	5
Classical equivalent	~AES-128	~AES-192	~AES-256
n (ring degree)	256	256	256
q (modulus)	3329	3329	3329
k (module rank)	2	3	4
η_1 / η_2	3 / 2	2 / 2	2 / 2
d_u / d_v	10 / 4	10 / 4	11 / 5
Public key (bytes)	800	1,184	1,568
Secret key (bytes)	1,632	2,400	3,168
Ciphertext (bytes)	768	1,088	1,568
Shared secret (bytes)	32	32	32

ML-DSA (FIPS 204)

PARAMETER	ML-DSA-44	ML-DSA-65	ML-DSA-87
NIST security level	2	3	5
q (modulus)	8,380,417	8,380,417	8,380,417
k / l (matrix dims)	4 / 4	6 / 5	8 / 7
η (secret coeff bound)	2	4	2
τ (challenge weight)	39	49	60
β (= τ·η)	78	196	120
γ_1	2^17	2^19	2^19
γ_2	(q-1)/88	(q-1)/32	(q-1)/32
ω (hint bound)	80	55	75
c̃ bytes (λ/4)	32	48	64
Public key (bytes)	1,312	1,952	2,592
Secret key (bytes)	2,560	4,032	4,896
Signature (bytes)	2,420	3,309	4,627

SLH-DSA (FIPS 205) — 6 variants (f = fast signing, s = small signature)

PARAMETER	128F	128S	192F	192S	256F	256S
NIST security level	1	1	3	3	5	5
n (hash output bytes)	16	16	24	24	32	32
h (total tree height)	66	63	66	63	68	64
d (layers)	22	7	22	7	17	8
h' (= h/d)	3	9	3	9	4	8
a (FORS leaf height)	6	12	8	14	9	14
k (FORS trees)	33	14	33	17	35	22
w (WOTS+ Winternitz)	16	16	16	16	16	16
Public key (bytes)	32	32	48	48	64	64
Secret key (bytes)	64	64	96	96	128	128
Signature (bytes)	17,088	7,856	35,664	16,224	49,856	29,792

Each row has a SHA-2 variant (SLH-DSA-SHA2-*) and a SHAKE variant (SLH-DSA-SHAKE-*) with identical parameters but different underlying hash function.

X-Wing Hybrid KEM (draft-connolly-cfrg-xwing-kem)

PARAMETER	VALUE
Classical component	X25519 (Curve25519 ECDH)
Post-quantum component	ML-KEM-768
Public key (bytes)	1,216 (32 + 1,184)
Secret key (bytes)	2,432 (32 + 2,400)
Ciphertext (bytes)	1,120 (32 + 1,088)
Shared secret (bytes)	32
KDF label	`\.//^\` (hex `5c 2e 2f 2f 5e 5c`, 6 bytes)
KDF	SHA3-256(label ‖ ss_m ‖ ss_x ‖ ek_x ‖ pk_x)

FALCON / FN-DSA (FIPS 206 Draft)

PARAMETER	FALCON-512	FALCON-1024
NIST security level	1	5
Public key (bytes)	897	1,793
Secret key (bytes)	1,281	2,305
Signature (bytes, average)	~666	~1,280
Signature (bytes, max)	752	1,462

Protocol Overhead Cheat Sheet (Classical vs Hybrid PQ)

Impact of adding PQC to common protocols. Numbers are approximate and depend on exact key/signature encoding. Useful for quick impact assessment during migration planning.

PROTOCOL	CLASSICAL	HYBRID (CLASSICAL + PQ)	MULTIPLIER
TLS 1.3 handshake (ECDHE-P256 + ECDSA-P256)	~670 B	X25519+ML-KEM-768 + Ed25519+ML-DSA-65: ~2,500 B	~3.7×
TLS 1.3 handshake (ECDHE-P384 + ECDSA-P384)	~900 B	X448+ML-KEM-1024 + Ed448+ML-DSA-87: ~4,800 B	~5.3×
SSH transport (per connection)	~1.5 KB	sntrup761+x25519 + Ed25519+ML-DSA-65: ~4.8 KB	~3.2×
S/MIME email (per recipient, RSA-2048)	~256 B	ML-KEM-768 wrap: ~1.2 KB; +ML-DSA-65 sig: ~4.5 KB	~4-18×
X.509 certificate (ECDSA-P256 leaf)	~1 KB	ML-DSA-65 leaf: ~4.5 KB; +hybrid composite: ~5.5 KB	~4.5-5.5×
TLS 1.3 cert chain (3 certs, ECDSA)	~3 KB	ML-DSA-65 chain: ~13.5 KB	~4.5×
Code signing (Authenticode, ECDSA)	~1.2 KB	ML-DSA-65: ~5 KB; SLH-DSA-128s: ~9 KB	~4-7.5×
DNSSEC DNSKEY record (ECDSA-P256)	~100 B	ML-DSA-44 DNSKEY: ~1.5 KB	~15×

Takeaways for capacity planning:

- Budget **3-5× bandwidth increase** for TLS/SSH under hybrid PQ
- DNSSEC and cert chains see the largest multipliers — plan DNS UDP fallback and cert bundling carefully
- SLH-DSA signatures are **~7× larger than ML-DSA-65** — reserve for non-bandwidth-sensitive uses

---